

Introduction to MCU with MicroPython

A comprehensive course guide for sensing, automation, and integrated applications using the M5StickS3 platform.

IoT & Sensing | Automation | Edge AI

AUTHOR

Dr Yeoh Wui Keat, Ph.D

Introduction to MCU with MicroPython

Table of Contents

Course Summary	1
Topic Coverage and Learning Outcomes	2
1. Introduction	3
2. M5StickS3 and MicroPython Setup	3
2.1 Setting up Thonny IDE and Supporting Tools.....	4
2.2 Configuring the M5StickS3 Hardware	4
2.3 IDE Serial Communication Setting	6
2.4 Running an Application Program	9
3. MicroPython Programming Basics.....	10
3.1 Printing Output	11
3.2 Variables and Data Types.....	11
3.3 Conditional Statements	12
3.4 Loops.....	13
3.5 Functions.....	17
4. M5StickS3 Built-in Peripherals	21
4.1 LCD Display.....	22
4.2 Push-Buttons and Buzzer	26
4.3 Real-Time Clock.....	27
4.4 Microphone.....	30
4.5 Inertial Measurement Unit	32
5. Using M5StickS3 External Peripherals	36
5.1 Expanding GPIO and Communication Ports.....	36
5.2 GPIO-based External Peripherals	38
5.3 I2C-based External Peripherals	44
6. Integrated Examples.....	55
6.1 SecureHome RFID Motion Alarm System	55
6.2 Smart Bathroom Monitoring and Control.....	60
7. Edge Artificial Intelligence	65
7.1 How Edge-AI Works	65
7.2 Edge-AI Use Cases	68
8. Key Takeaways.....	76
9. Concluding Note.....	78
 Appendix A – MicroPython Examples.....	 79
Appendix B – List of MicroPython Commands	81
Appendix C – Hardware List.....	87

Course Summary

The course document introduces learners to microcontroller application development using the M5StickS3 and MicroPython. It begins by positioning the M5StickS3 as a compact ESP32-S3-based development device for portable IoT, sensing, automation, and embedded-system prototyping. Learners are guided to set up the software environment using Thonny IDE, configure the M5StickS3 with UIFlow 2.0 firmware using M5Burner, and establish serial communication between the computer and the MCU board.

After the setup activities, the document builds foundational MicroPython programming skills. Learners practice printing output, creating variables, using common data types, applying if/elif/else conditional statements, repeating tasks with for and while loops, adding delays, and organizing code using functions. These basic programming concepts are directly linked to MCU applications, where programs must continuously read inputs, process data, make decisions, and control outputs.

The next part of the training focuses on the built-in peripherals of the M5StickS3. Learners use the LCD display to show text, colors, and simple graphics; use push buttons for interaction; generate tones using the built-in speaker; use the real-time clock for timing applications; record and process sound through the microphone; and read motion information from the built-in IMU for tilt, vibration, and pedometer examples.

The document then expands beyond the built-in hardware by introducing Unit Hub, PaHUB, and PbHUB. These hub modules allow the M5StickS3 to connect to more external peripherals. Learners practice using GPIO-based modules such as dual buttons, PIR motion sensors, light sensors, angle sensors, and servo motors. They also use I2C-based modules such as the 4-Relay Unit, Color Sensor, Ultrasonic Distance Sensor, ENV-Pro environmental sensor, ADXL345 accelerometer, and RFID Unit.

The later sections combine multiple sensors and actuators into integrated applications. Examples include an RFID motion alarm system, a smart bathroom monitoring and control system, machine vibration monitoring, and thermal image learning and recognition. These examples show how an MCU can perform sensing, local processing, decision-making, actuator control, display updates, audible alerts, and simple Edge-AI learning and recognition without relying fully on cloud processing.

The final advanced topic introduces Edge-AI and prototype-based Neuromorphic AI. Learners are shown how local devices can process data, extract features, learn patterns, classify new inputs, and make real-time decisions. The vibration and thermal-imaging examples demonstrate how sensor data can be converted into feature vectors and used by a lightweight AI model for local pattern recognition on the M5StickS3.

Topic Coverage

Topic Area	Main Coverage
Introduction and setup	M5StickS3 overview, MicroPython motivation, Thonny IDE installation, M5Burner firmware configuration, UIFlow 2.0 setup, and serial communication with the MCU.
MicroPython programming basics	Printing output, variables, data types, conditional statements, loops, delays, functions, parameters, return values, and program structure.
Built-in M5StickS3 peripherals	LCD display, graphics, push buttons, speaker, real-time clock, microphone, sound recording, IMU tilt sensing, vibration sensing, and pedometer applications.
External GPIO peripherals	PbHUB-based dual-button input, PIR motion detection, light intensity measurement, angle sensor input, and servo motor control.
External I2C peripherals	PaHUB-based relay control, color sensing, ultrasonic distance measurement, ENV-Pro environmental monitoring, ADXL345 accelerometer sensing, and RFID reading.
Integrated applications	SecureHome RFID motion alarm, smart bathroom automation, environmental monitoring, fan and light control, and safety alert mechanisms.
Edge-AI and Neuromorphic AI	Edge-AI workflow, feature extraction, prototype-based learning, vibration monitoring, thermal image learning, category recognition, and distance-based interpretation.

Learning Outcomes

By the end of the course, learners should be able to:

1. Set up the M5StickS3 development environment and establish communication with the board using Thonny IDE.
2. Write and run MicroPython programs using the standard `setup()` and `loop()` structure.
3. Use basic Python constructs to implement decision-making, repetition, and reusable program functions.
4. Control the M5StickS3 LCD, speaker, buttons, RTC, microphone, and IMU for interactive embedded applications.
5. Connect and program external peripherals through PaHUB, PbHUB, and the I2C/GPIO expansion architecture.
6. Design simple automation and monitoring systems that combine sensors, actuators, display feedback, and alerts.
7. Explain how Edge-AI and prototype-based Neuromorphic AI can be implemented on an MCU for local learning and recognition.

1. Introduction

The **M5StickS3** (Figure 1-1) is a compact ESP32-S3-based microcontroller development device designed for portable IoT, sensing, automation, and embedded-system applications. It combines a color LCD display, programmable push buttons, microphone, speaker, inertial measurement unit, real-time clock, wireless connectivity, and expansion interfaces within a small handheld form factor. The device can operate as a standalone controller or connect to additional sensors and actuators through its expansion ports. Its built-in rechargeable battery supports mobile and untethered use, while Wi-Fi and Bluetooth connectivity enable communication with other devices, cloud platforms, and local applications. With support for MicroPython, the M5StickS3 provides a beginner-friendly programming environment while still offering sufficient capability for more advanced prototyping, making it suitable for classroom learning, rapid development, proof-of-concept projects, and portable smart-device applications.



Figure 1-1 - M5StickS3 MCU Module

2. M5StickS3 and MicroPython Setup

Traditionally, application programs for microcontroller systems were commonly written in assembly language or low-level C. While these languages provide detailed control over hardware resources, they may be difficult for beginners to learn and can require more development time. With the availability of MicroPython, learners can now develop microcontroller applications using a simpler and more readable Python-based programming approach.

MicroPython is a lightweight implementation of the Python programming language designed for microcontrollers and embedded systems. Its structured syntax and ease of use make it suitable for learning MCU application development, especially for beginners. It allows users to write programs that can read sensor inputs, process data, control output devices, and communicate with external modules using familiar Python-style commands.

Application development in this course will be carried out using the Thonny Integrated Development Environment (IDE). Thonny is a free and beginner-friendly development tool that provides a complete environment for writing, testing, and uploading MicroPython programs to the MCU hardware platform. It includes a text editor for writing code, a shell or serial console for interacting with the MCU, a message area for displaying errors or system messages, toolbars for common actions, and menus for configuring the development environment.

The Thonny IDE runs on a host computer and connects to the MCU board through a USB cable. This connection allows programs to be uploaded to the MCU and also enables communication between the computer and the board. The IDE supports basic syntax checking, program execution, file management, and serial communication with the MCU's onboard firmware or bootloader.

In this laboratory session, learners will familiarize themselves with the Thonny IDE, connect the M5StickS3 MCU board to the computer, run simple MicroPython commands, and prepare the hardware and software environment for subsequent MCU application development activities.

2.1 Setting up the Thonny IDE and supporting tools

Before creating MCU application software, learners are required to set up the necessary development tools and prepare a personal working directory. This working directory will be used as the main workspace for storing MicroPython programs, lab exercises, and project files throughout the course.

First, download Thonny IDE from <https://thonny.org/>. Learners should download **thonny-5.0.0-x64.exe** or a newer version, depending on availability. After the download is completed, open the PC's Downloads folder, locate the downloaded .exe file, and run the installer. Follow the on-screen instructions to complete the installation of Thonny IDE.

Next, download the required M5Stack supporting tools from <https://docs.m5stack.com/en/download>. Learners should download **M5Burner-v3-beta-win-x64.zip** and the USB Driver, if required. After the downloads are completed, open the Downloads folder and extract all downloaded .zip files. If the computer is unable to detect the M5StickS3 board, install the USB Serial Driver before continuing.

Learners should also clone or download the lab resources from the following GitHub repository: https://github.com/yeohwk/M5StickS3_MicroPython.git. These resources contain the sample programs, templates, and supporting files required for the lab activities. Save the downloaded files into the personal working directory created for this course.

2.2 Configure M5StickS3 MCU Development Hardware

MCU development hardware is required for developing, testing, and deploying MCU application programs. In this lab and subsequent labs, the **M5Stack M5StickS3 MCU module** will be used as the main development board.

Use the provided USB-C cable to connect the M5StickS3 to the development computer. Once connected, open the **M5Burner-v3-beta-win-x64** folder that was set up earlier and double-click **M5Burner.exe** to launch the M5Burner configuration program.

Before downloading firmware, the M5StickS3 must be placed into **Download Mode**. To do this, connect the M5StickS3 to the computer using the USB-C cable. Press and hold the reset button on the side of the device until the internal green LED flashes. When the green LED flashes, the device has successfully entered Download Mode.

In the M5Burner configuration program (Figure 2-2), complete the following steps:

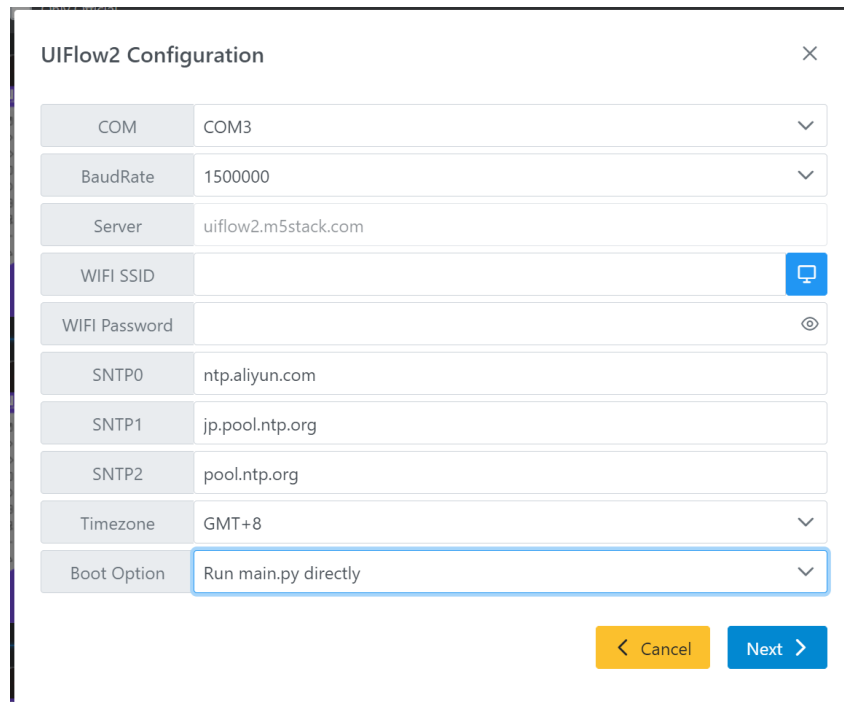
1. Select the correct MCU hardware model, **M5StickS3**.
2. Locate and download the latest **UIFLOW 2.0 (StickS3)** firmware.
3. Click **Configure** and a popup appears (Figure 2-3):
 - Select the correct communication **COM port** for connected M5StickS3.
 - Set the **baud rate** to 1500000 (default). Leave “Blank” for **WiFi** entries.
 - Set **Timezone** to GMT+8 and **Boot Option** 1500000 to Run main.py directly
4. Burn the firmware to the M5StickS3.

After the firmware burning process is completed, restart the M5StickS3 and confirm that the device boots correctly. If the computer is unable to detect the M5StickS3, check the USB cable, confirm that the correct USB driver has been installed, and verify that the correct COM port is selected.

The M5StickS3 has now been successfully configured. The steps in this section only need to be repeated if the M5StickS3 becomes unresponsive, fails to boot correctly, or cannot be detected by the development computer.



Figure 2-2 – M5 Burner Configuration Program: (1) Select the correct MCU hardware, (2) Locate and download the latest UIFLOW 2.0 (StickS3) firmware, (3) Set correct COM port, Baud rate, Timezone and Boot Option, (4) Burn the firmware to the M5StickS3.



The image shows the 'UIFlow2 Configuration' window. It contains the following fields and values:

Field	Value
COM	COM3
BaudRate	1500000
Server	uiflow2.m5stack.com
WiFi SSID	
WiFi Password	
SNTP0	ntp.aliyun.com
SNTP1	jp.pool.ntp.org
SNTP2	pool.ntp.org
Timezone	GMT+8
Boot Option	Run main.py directly

At the bottom right, there are two buttons: '< Cancel' (yellow) and 'Next >' (blue).

Figure 2-3 – UIFLOW 2.0 Configuration (with correct parameters).

2.3 IDE Serial Communication Interface Setting

Learners are now ready to explore the Thonny Integrated Development Environment (IDE). In Thonny, a Python script represents an MCU application program. It contains the instructions that will be uploaded to and executed on the M5StickS3 module.

When Thonny IDE is launched for the first time (Figure 2-4), communication with the M5StickS3 may not yet be established. Therefore, the IDE must be configured to communicate with the M5StickS3 through the correct serial communication port. This should be the same COM port that was used earlier in M5Burner.

From the menu bar, learners should click Tools and select Options (Figure 2-5). A small configuration window will appear with several tabs. Learners should select the Interpreter tab (Figure 2-6). Under the interpreter selection, MicroPython (ESP32) should be selected from the list. The correct COM port (Port or WebREPL) for the connected M5StickS3 should then be selected.

After completing the settings, learners should click OK. Thonny will attempt to establish communication with the M5StickS3. Activity should be observed in the Thonny Shell. When the connection is successful, the MicroPython prompt >>> will appear in the Shell window (Figure 2-7).

Once the >>> prompt is shown, the serial communication link between Thonny IDE and the M5StickS3 has been successfully configured. Learners may then proceed to write, upload, and run MicroPython programs on the M5StickS3.

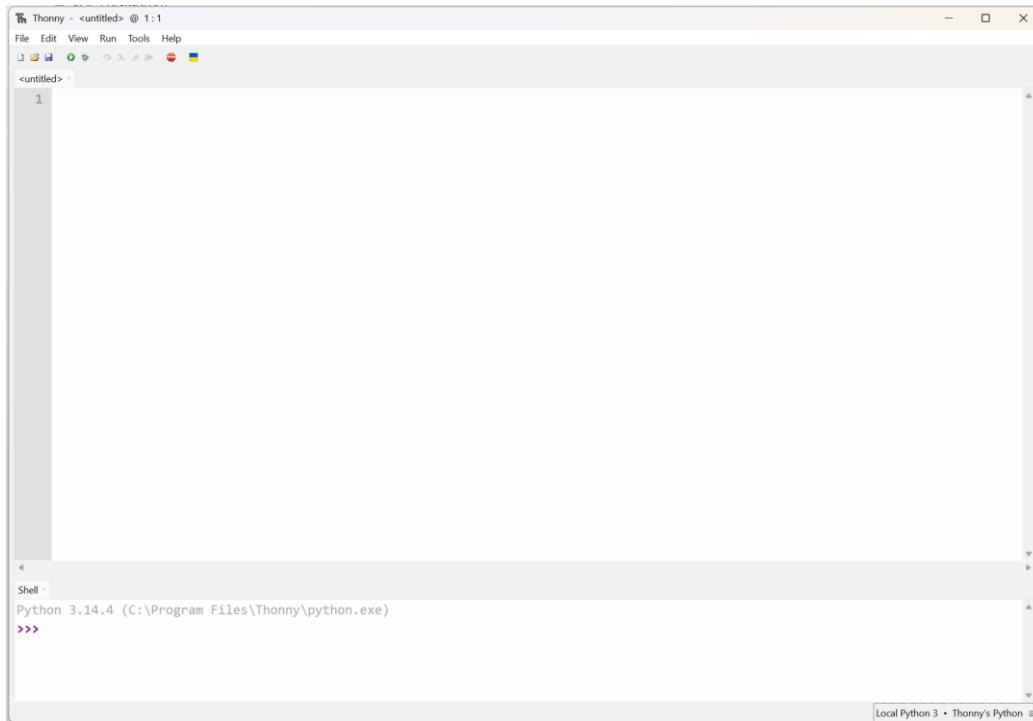
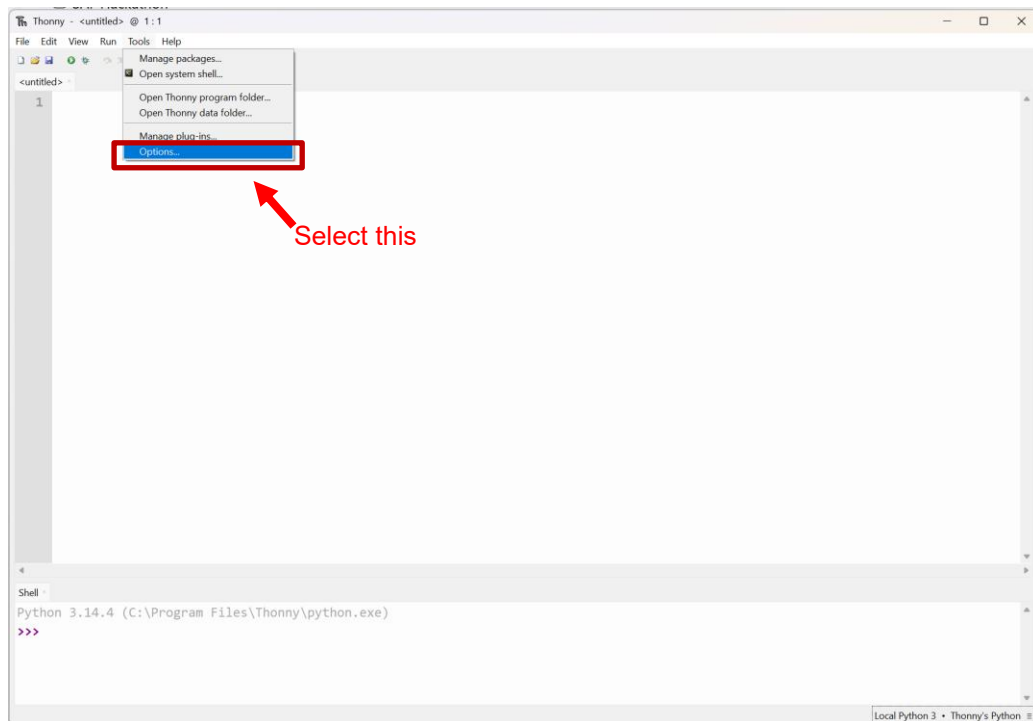
Figure 2-4 - Thonny IDE (1st Launch)

Figure 2-5 - Thonny IDE.

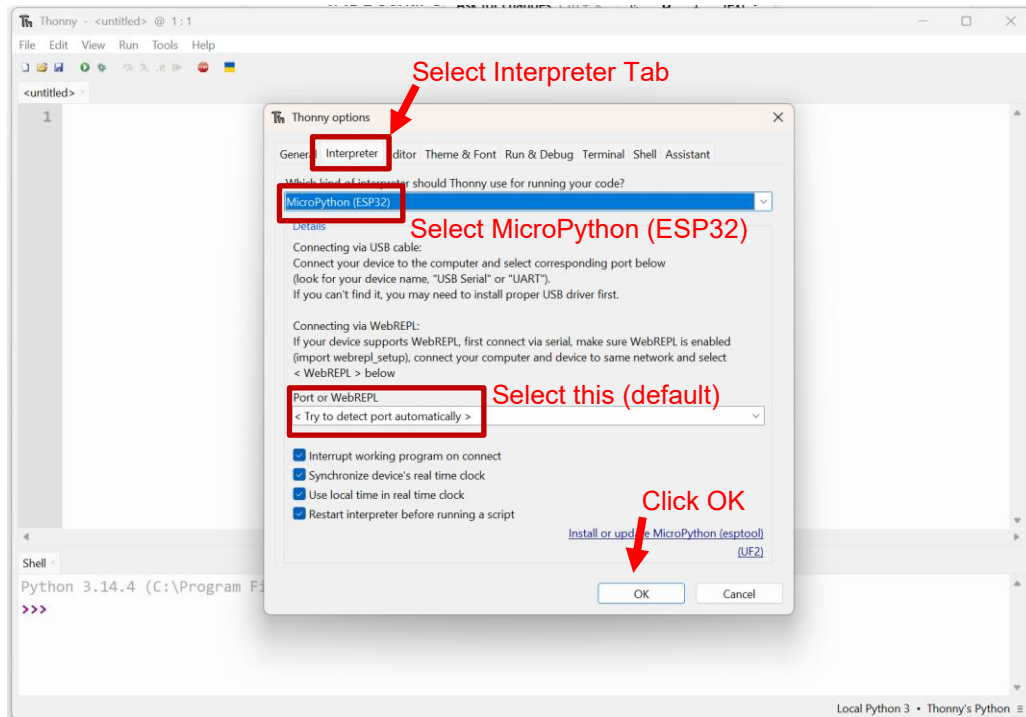


Figure 2-6 - Configure Thonny IDE to communicate with M5SickS3.

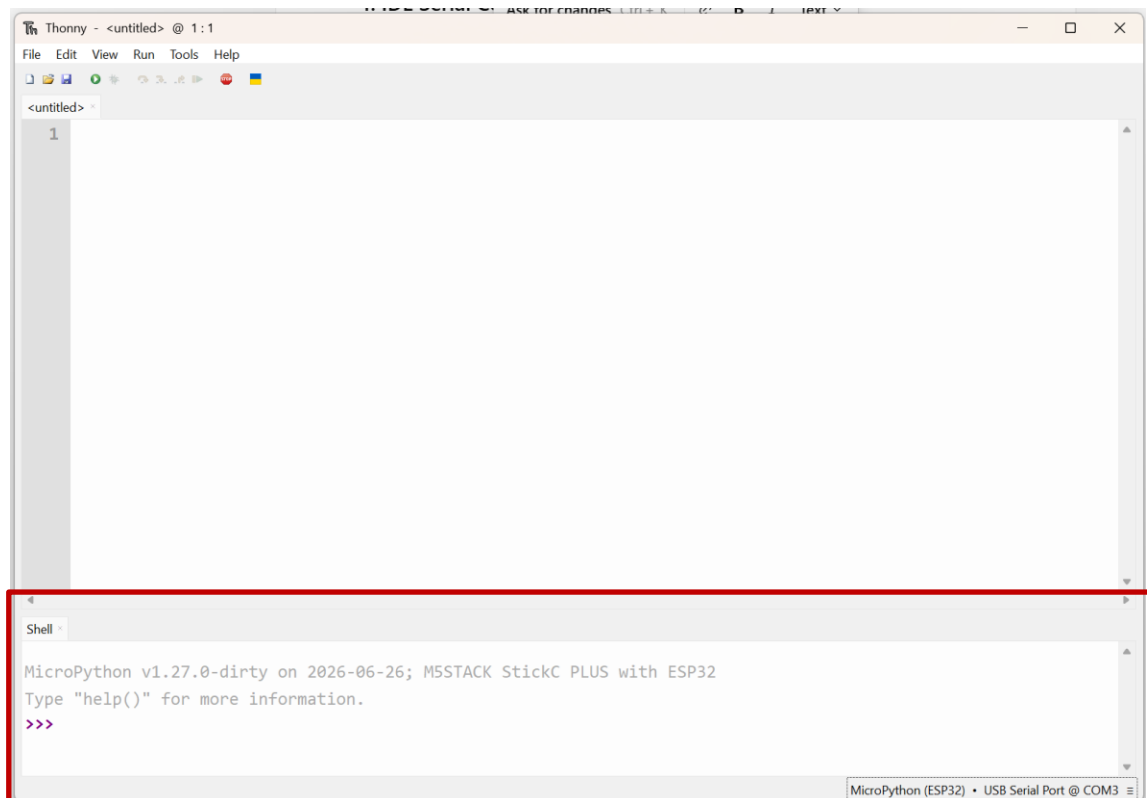


Figure 2-7 - Thonny IDE communicates with M5SickS3.

2.4 Running an Application Program on M5StickS3

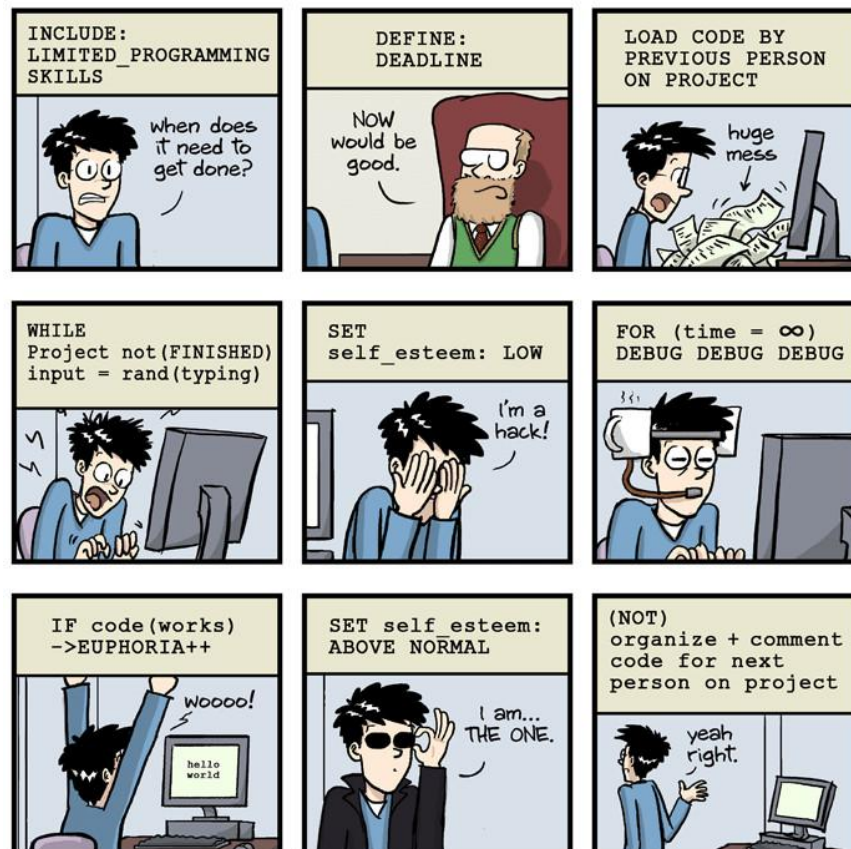
Learners will now load and run their first MicroPython program (Listing 2-1) on the M5StickS3. The application program is provided as a Python file and will be opened and executed using Thonny IDE.

The file **test1.py** is provided in the lab resources downloaded earlier and should already be saved in the personal working directory created for this course. Learners should launch Thonny IDE and open test1.py.

When a pop-up window appears asking “Where to open from?”, learners should select **This Computer**. They should then navigate to the folder containing **test1.py**, select the file, and click **Open**. Once the file is successfully loaded in Thonny, the Python code will appear in the editor window.

To run the program on the M5StickS3, learners should click the **Run** button in Thonny. They should observe the behavior of the M5StickS3 and note any output shown on the device or in the Thonny Shell.

To stop the running program, learners should click the **Stop** button in Thonny. This will interrupt the program running on the M5StickS3 and return control to the MicroPython prompt.



```

import os, sys, io
import M5
from M5 import *
import time

circle0 = None
circle1 = None
rect0 = None
triangle0 = None
label0 = None

#Initialization (Run once)
def setup():
    global circle0, circle1, rect0, triangle0, label0

    M5.begin()
    Widgets.setRotation(0)
    Widgets.fillScreen(0x000000)
    circle0 = Widgets.Circle(34, 56, 15, 0xffffffff, 0xea1313)
    circle1 = Widgets.Circle(91, 57, 15, 0xffffffff, 0xffffffff)
    rect0 = Widgets.Rectangle(35, 143, 60, 15, 0x19e240, 0x19e240)
    triangle0 = Widgets.Triangle(64, 90, 55, 118, 76, 118, 0xffffffff, 0xffffffff)
    label0 = Widgets.Label("Good Day !", 19, 198, 1.0, 0xffffffff, 0x222222, Widgets.FONTS.Montserrat18)
    Speaker.setVolumePercentage(100)

#Main Program (Run in a loop)
def loop():
    global circle0, circle1, rect0, triangle0, label0
    M5.update()
    Speaker.tone(4000, 200) #4KHz Tone, 200ms duration.
    time.sleep(1)          #Delay 1s.

#System Default (Do not modify)
if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")

```

Listing 2-1 - First MicroPython Program (test1.py).

3. MicroPython Programming Basics

MicroPython is a lightweight version of the Python programming language designed to run on microcontrollers. It allows learners to write simple and readable programs to control MCU hardware such as displays, buttons, sensors, relays, and motors.

In MCU applications, MicroPython is used to **read** input values, **make decisions**, **control output** devices, and repeat tasks continuously. For example, a program can read a distance sensor and trigger an alert when an object is too close.

3.1 Printing Output

The **print()** function is used to display messages or values in the Thonny Shell. It is useful for checking whether a program is running correctly and for debugging.

Learners should use Thonny to create a new Python file named **myprint.py**. Enter the Python program shown in Listing 3-1 into the editor window. Run the program and observe the output displayed in the Thonny Shell.

```
print("Hello from M5StickS3")
print("I am learning MicroPython")
```

Listing 3-1 - print() function.

After running the program, learners should observe that both messages are displayed line by line in the Thonny Shell.

Expected output:

```
Hello from M5StickS3
I am learning MicroPython
```

Exercise

Modify **myprint.py** so that it displays the learner's name (replaces <YOUR NAME>) and one additional message.

Expected output:

```
My name is <YOUR NAME>
This is my first MicroPython program
```

3.2 Variables and Data Types

A variable is used to store data in a program. The stored value can be used later for calculation, display, or decision-making.

Learners should use Thonny to create a new Python file named **mydata.py**. Enter the Python program shown in Listing 3-2 into the editor window. The program demonstrates how to create variables and display their values using the **print()** function. A variable is used to store data so that the value can be used later in the program. Run the program and observe the output displayed in the Thonny Shell.

```
device_name = "M5StickS3"
temperature = 28
distance = 15.5
system_ready = True

print(device_name)
print(temperature)
print(distance)
print(system_ready)
```

Listing 3-2 - Variables and Data.

Data Type	Example	Description
String	"M5StickS3"	Text
Integer	28	Whole number
Float	15.5	Decimal number
Boolean	True	True or false value

Table 3-1 - Data Types.

In Listing 3-2, four variables are created. The variable **device_name** stores the text value "M5StickS3". Text values are known as strings and must be placed inside quotation marks. The variable **temperature** stores the whole number 28, which is an integer. The variable **distance** stores the decimal value 15.5, which is a floating-point number. The variable **system_ready** stores the value True, which is a Boolean value used to represent true or false conditions. Table 3-1 depicts the data types used in the program.

The **print()** function is then used to display the value stored in each variable. When the program is executed, the values will be shown line by line in Thonny Shell. This allows learners to check that the variables have been assigned correctly and understand how different types of data can be stored and displayed in MicroPython.

Expected output:

```
M5StickS3
28
15.5
True
```

Exercise

Learners to modify Listing 3-3 to create and initialize variables for name, age, and favourite_activity, then print them.

```
name = "<YOUR NAME>"
age = <YOUR AGE>
favourite_activity = "<FAVOURITE ACTIVITY>"

print("Name:", name)
print("Age:", age)
print("Favourite Activity:", favourite_activity)
```

Listing 3-3 - Variables and Data Exercise.

3.3 Conditional Statements

Conditional statements allow a program to make decisions based on whether a condition is true or false. In MCU programming, this is important because the microcontroller often needs to respond differently depending on sensor readings, button inputs, or system status.

For example, a distance sensor may detect how far an object is from the device. If the object is near, the system may display a warning message or activate an alarm. If the object is far, the system may display that the area is clear.

A simple conditional statement uses **if** and **else**. In the example depicted in Listing 3-4, the variable distance stores the value 15. The program checks whether distance < 20 is true. Since 15 is less than 20, the condition is true and the message "Object is near" will be displayed in the Thonny Shell. If the value of distance is changed to a number greater than or equal to 20, the condition becomes false and the program will display "Object is far" instead.

```
distance = 15

if distance < 20:
    print("Object is near")
else:
    print("Object is far")
```

Listing 3-4 - Simple if and else conditional statement.

Expected output:

Object is near

A program can also check multiple conditions using **elif**, which means “else if”. This is useful when the system has more than two possible outcomes. In the example depicted in Listing 3-5, the program checks the conditions from top to bottom. Since distance is 35, the first condition distance < 15 is false. The second condition distance < 30 is also false. The third condition distance < 50 is true, so the program displays "Caution". Once a true condition is found, Python executes that block and skips the remaining conditions.

```
distance = 35

if distance < 15:
    print("Danger: Stop")
elif distance < 30:
    print("Warning: Object nearby")
elif distance < 50:
    print("Caution")
else:
    print("Safe")
```

Listing 3-5 - Conditional statement with elif.

Expected output:

Caution

This type of program structure is commonly used in sensor-based applications. For example, a car reverse sensor may show "Safe" when an object is far away, "Caution" when the object is getting closer, "Warning" when the object is nearby, and "Danger: Stop" when the object is too close.

Exercise 1

Change the value of **distance** in Listing 3-5 to the values provided in Table 3-2 and observe the output in Thonny Shell.

Distance Value	Expected Output
10	Danger: Stop
25	Warning: Object nearby
40	Caution
60	Safe

Table 3-2 - Distance value and expected output.

Exercise 2

Modify the program in Listing 3-4 so that it checks the temperature value instead of distance. The program should display:

- "Temperature is high" if the temperature is above 30
- "Temperature is normal" if the temperature is 30 or below

Learners should test the program using different temperature values and observe how the output changes.

3.4 Loops

Loops are used to repeat a set of instructions. In MCU programming, loops are very important because most microcontroller applications need to run continuously. For example, an MCU may need to keep reading a sensor, checking a button, updating a display, or controlling an output device until the system is turned off.

Without loops, the program would only run once and stop. With loops, the MCU can repeatedly perform the same task.

There are two common types of loops in MicroPython:

- **for** loop
- **while** loop

3.4.1 Using a for Loop

A **for** loop is used when the number of repetitions is known. For example, if the program needs to count from 0 to 4, a **for** loop can be used. In the example depicted in Listing 3-6, **range(5)** produces the values 0, 1, 2, 3, 4. The program repeats the `print()` statement five times. Each time the loop runs, the variable `count` takes the next value.

```
for count in range(5):  
    print("Count:", count)
```

Listing 3-6 - Example of **for** loop.

Expected output:

```
Count: 0  
Count: 1  
Count: 2  
Count: 3  
Count: 4
```

The number inside `range()` controls how many times the loop repeats. If **range(10)** is used, the loop will repeat ten times (Listing 3-7), starting from 0 and ending at 9.

```
for count in range(10):  
    print("Count:", count)
```

Listing 3-7 - Example of **for** loop

3.4.2 Using a while Loop

A **while** loop is used when the program should continue repeating as long as the condition remains true. In the example depicted in Listing 3-8, the loop continues running while `counter < 5` is true. The value of `counter` starts at 0. Each time the loop runs, the program displays the current value and then increases `counter` by 1. When `counter` reaches 5, the condition becomes false and the loop stops.

```
counter = 0  
while counter < 5:  
    print("Counter:", counter)  
    counter = counter + 1
```

Listing 3-8 - Example of **while** loop

Expected output:

```
Counter: 0  
Counter: 1  
Counter: 2  
Counter: 3  
Counter: 4
```

This line of code found in Listing 3-8 is important:

counter = counter + 1

It increases the value of counter each time the loop runs. Without this line, the value of counter would remain 0, and the condition `counter < 5` would always be true. This would cause an infinite loop.

3.4.3 Infinite Loops in MCU Programs

In many MCU applications, an infinite loop is commonly used because the microcontroller is expected to continue running until power is removed or the program is manually stopped. As shown in Listing 3-9, the condition **True** is always true. Therefore, the statements inside the loop will continue to execute repeatedly without ending.

```
while True:
    print("System running")
```

Listing 3-9 – Example of an infinite **while** loop

This type of loop is useful for MCU applications that need to continuously monitor inputs and respond to changes or events. For example, an infinite loop can be used to repeatedly read sensor values, detect button presses, update a display, or control output devices. Listing 3-10 shows how an infinite loop can be used to perform a continuous monitoring task, allowing the MCU to keep checking the system status throughout the operation of the device.

```
while True:
    temp_sensor = readTemp()
    print("Reading temperature: ", temp_sensor)
```

Listing 3-10 – Example of an infinite **while** loop reading sensor data.

3.4.4 Adding Delay in a Loop

When using loops, it is common to add a delay so that the program does not repeat too quickly. The `time.sleep()` function can be used to pause the program. In the example depicted in Listing 3-11, the message "Reading sensor..." is displayed five times. The program waits for 1 second between each message.

The **`time.sleep(1)`** statement pauses the program for 1 second. If **`time.sleep(0.5)`** is used, the program pauses for half a second.

```
Import time
for count in range(5):
    print("Reading sensor...")
    time.sleep(1)
```

Listing 3-11 – Delay in a loop.

Expected output:

```
Reading sensor...
Reading sensor...
Reading sensor...
Reading sensor...
Reading sensor...
```

Example: Simulated Sensor Reading

The example in Listing 3-12 simulates a microcontroller reading a sensor repeatedly. In this program, the distance value decreases by 10 each time the loop runs. This represents an object gradually moving closer to a sensor. The loop continues as long as the distance value is greater than 0. When the distance is no longer greater than 0, the loop ends and the program displays the message "Object is very close".

```
Import time
```

```
distance = 50
while distance > 0:
    print("Distance: ", distance, "cm")
    distance = distance - 10
    time.sleep(1)
print("Object is very close")
```

Listing 3-12 – Delay in a loop simulating a distance sensor.

Expected output:

Distance: 50 cm
Distance: 40 cm
Distance: 30 cm
Distance: 20 cm
Distance: 10 cm
Object is very close

Activity: Modify the Loop

Run the following program in Listing 3-13 and observe the output.

```
Import time

for count in range(5):
    print("Count: ", count)
    time.sleep(1)
```

Listing 3-13 – Delay in a loop.

Expected output:

Count: 0
Count: 1
Count: 2
Count: 3
Count: 4

Modify the program depicted in Listing 3-13 so that it counts from 0 to 9, and delay from 1 second to 0.5 second. Observe how the output changes. Listing 3-14 depicted the modified program.

```
import time

for count in range(9):
    print("Count:", count)
    time.sleep(0.5)
```

Listing 3-14 - Delay in a loop modification from **range(5)** to **range(9)**, and delay from **1s** to **0.5s**.**Expected output:**

Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6
Count: 7
Count: 8

Exercise

Complete the program in Listing 3-15 so that it displays "System checking..." five times.

```
import time

for count in range(<RANGE_VALUE>):
    print("System checking...")
    time.sleep(1)
```

Listing 3-15 - Delay in a loop. Determine <RANGE_VALUE>.

Expected output:

System checking...
System checking...
System checking...
System checking...
System checking...

Application Example

Loops are commonly used in MCU applications such as:

- Continuously checking a sensor value
- Repeating a buzzer beep
- Blinking an LED
- Updating a display
- Monitoring button presses
- Sending repeated data updates to another device

For example, a reverse sensor system may use a loop to continuously measure the distance between the vehicle and an obstacle. If the object gets too close, the program can display a warning or activate a buzzer.

3.5 Functions

A function is a reusable block of code that performs a specific task. Instead of writing the same instructions many times, the instructions can be placed inside a function and called whenever they are needed. Functions help make programs shorter, clearer, and easier to maintain.

In MCU programming, functions are useful because many tasks are repeated regularly. For example, a program may need to display a message, read a sensor, check a distance value, activate an output, or show an alert. By placing these tasks inside functions, the program becomes more organized.

3.5.1 Basic Function Structure

A function is created using the **def** keyword. The function name is followed by **brackets ()** and a **colon :**. The instructions inside the function must be indented.

```
def show_message():
    print("System Ready")
```

Listing 3-16 - Basis function.

The function in Listing 3-16 is named **show_message**. It contains one instruction that displays the message "System Ready" in the Thonny Shell. However, defining a function only tells the program what the function should do; it does not execute the function immediately.

To execute the function, it must be called by using its function name, as shown in Listing 3-17. When **show_message()** is called, the instruction inside the function will run and the message "System Ready" will be displayed in the Thonny Shell.

```
def show_message():
    print("System Ready")

show_message()
```

Listing 3-17 - Function call.

Expected output:

System Ready

Why Are Functions Useful?

Functions help to:

Benefit	Explanation
Reduce repeated code	Common instructions can be written once and reused.
Improve readability	Program is easier to understand when tasks are grouped clearly.
Simplify troubleshooting	Errors can be checked function by function.
Support larger programs	Complex applications can be divided into smaller parts.
Encourage reuse	The same function can be used in different parts of the program.

For example, instead of writing the same warning message many times, a function can be created to display the warning as depicted in Listing 3-18.

```
def show_warning():
    print("Warning: Object nearby")

show_warning()
show_warning()
show_warning()
```

Listing 3-18 - Multiple calls to the same function.

Expected output:

Warning: Object nearby
Warning: Object nearby
Warning: Object nearby

3.5.2 Functions with Input Values

A function can also receive input values. These input values are called parameters. Parameters allow the same function to work with different data.

In the example depicted in Listing 3-19, distance is a **parameter**. When **check_distance(15)** is called, the value 15 is passed into the function. Since 15 is less than 20, the output will be **"Object is near"**. When **check_distance(30)** is called, the output will be **"Object is far"**.

```
def check_distance(distance):
    if distance < 20:
        print("Object is near")
    else:
        print("Object is far")

check_distance(15)
check_distance(30)
```

Listing 3-19 - Same function call with different parameters.

Expected output:

Object is near
Object is far

3.5.3 Functions with Multiple Conditions

Functions can also contain conditional statements as depicted in Listing 3-20. This is useful when the MCU needs to classify sensor readings into different warning levels.

```
def check_distance(distance):  
    if distance < 15:  
        print("Danger: Stop")  
    elif distance < 30:  
        print("Warning: Object nearby")  
    elif distance < 50:  
        print("Caution")  
    else: print("Safe")  
  
check_distance(10)  
check_distance(25)  
check_distance(40)  
check_distance(60)
```

Listing 3-20 - Function with multiple conditions.

Expected output:

Danger: Stop
Warning: Object nearby
Caution
Safe

This type of function can be used in applications such as a car reverse sensor, where the warning message changes depending on how close the obstacle is.

3.5.4 Functions that Return Values

Some functions perform a calculation and return a result. The return keyword sends a value back to the part of the program that called the function as depicted in Listing 3-21.

```
def calculate_total(a, b):  
    total = a + b  
    return total  
  
result = calculate_total(5, 3)  
print("Result: ", result)
```

Listing 3-21 - Function that returns a value.

Expected output:

Result: 8

In MCU applications, a function can return a processed sensor value, a calculated average, or a system status, as depicted Listing 3-22.

```
def convert_to_percentage(value):  
    percentage = value * 100  
    return percentage  
  
light_level = 0.75  
light_percentage = convert_to_percentage(light_level)  
print("Light level: ", light_percentage, "%")
```

Listing 3-22 - Function that process sensor value and returns a processed value.

Expected output:

Light level: 75%

Example 1: Function for Temperature Checking

The following function in Listing 3-23 checks whether the temperature value is normal or high.

```
def check_temperature(temp):  
    if temp > 30:  
        print("Temperature is high")  
    else:  
        print("Temperature is normal")  
  
check_temperature(28)  
check_temperature(35)
```

Listing 3-23 - Function for temperature check.

Expected output:

Temperature is normal

Temperature is high

Example 2: Function Used in a Loop

Functions are often used together with loops in MCU programs. The loop keeps running, while the function performs a repeated task.

In the example shown in Listing 3-24, the program checks several distance values one at a time. The function **check_distance()** is called repeatedly inside the loop.

```
import time  
  
def check_distance(distance):  
    if distance < 15:  
        print("Danger: Stop")  
    elif distance < 30:  
        print("Warning")  
    else:  
        print("Safe")  
  
distances = [50, 35, 25, 10]  
  
for distance in distances:  
    print("Distance:", distance, "cm")  
    check_distance(distance)  
    time.sleep(1)
```

Listing 3-24 – Function called repeatedly within a **for** loop.**Expected output:**

Distance: 50cm

Safe

Distance: 35cm

Safe

Distance: 25cm

Warning

Distance: 10cm

Danger: Stop

Exercise

Complete the function in Listing 3-25 so that it checks the battery level. The program should display:

- "Battery low" if the battery level is below 20
- "Battery OK" if the battery level is 20 or above

```
def check_battery(level):  
    if level < <BATTERY_LEVEL>:  
        print("Battery low")  
    else:  
        print("Battery OK")  
  
check_battery(15)  
check_battery(80)
```

Listing 3-25 – Battery level checker. Replace <BATTERY_LEVEL> with appropriate value.

Expected output:

Battery low
Battery OK

Application Example

Functions are commonly used in MCU applications to organize tasks such as:

- Reading sensor values
- Checking thresholds
- Displaying messages
- Activating relays or buzzers
- Controlling servo motors
- Updating screen output
- Sending data to another device

For example, a smart access system may use one function to read an RFID tag, another function to check whether the tag is authorized, and another function to unlock the door. This makes the program easier to understand and easier to improve later.

4. M5StickS3 Built-in Peripherals

The M5StickS3 is a compact ESP32S3-based microcontroller module with a rich set of built-in peripherals that allow learners to create interactive MCU applications without relying heavily on external components. In this course, the focus will be on selected built-in peripherals, namely the **LCD display**, **Buzzer** or speaker, **Push-buttons**, **Real-time Clock**, **Microphone**, and **IMU**. The LCD display can be used to show text, sensor readings, status messages, menus, and simple graphics, while the buzzer or speaker can provide audio feedback such as alerts, confirmation tones, and warning sounds. The push buttons allow users to provide simple inputs to the program, such as selecting options, starting or stopping an operation, or changing system modes. The Real-time Clock provides accurate time-keeping and related functions.

The microphone and IMU further extend the sensing capability of the M5StickS3. The microphone can be used to detect sound levels or simple audio-triggered events, while the IMU can measure motion, tilt, orientation, and vibration. Together, these built-in peripherals provide a strong foundation for learning core MCU programming concepts such as input detection, output control, sensor reading, decision-making, and user interaction. These skills will also support learners when they progress to using external sensors and actuator modules in later course activities.

Listing 4-1 shows the MicroPython program template for the M5StickS3 using the M5Stack library. It provides the standard structure for writing an MCU application program.

The program begins by importing the required libraries. The **os**, **sys**, and **io** libraries provide general system-level support, while **M5** and **from M5 import *** provide access to the M5StickS3 hardware functions such as the display, buttons, buzzer, microphone, IMU and other built-in peripherals. The **time** library is used when timing delays are required in the program.

The **setup()** function is used to place initialization codes that should run once when the program starts. In this template, **M5.begin()** initializes the M5StickS3 hardware and prepares the built-in peripherals for use. Examples of initialization codes include setting up the LCD display, configuring variables, preparing sensors, or displaying a startup message.

The **loop()** function contains the main application codes that should run repeatedly. The statement **M5.update()** refreshes the device status and should be called regularly, especially when working with buttons and other M5StickS3 functions. Application codes such as reading button status, updating the LCD display, checking sensor values, or controlling outputs should be placed inside this function.

The final section of the program should not be modified. It ensures that the **setup()** function runs once, followed by the **loop()** function running continuously. The **while True:** statement creates an infinite loop so that the M5StickS3 keeps executing the application. The **try** and **except** structure is used for error handling. If an error occurs, the program will attempt to display an error message. If the required error utility is not available, it will show the message "please update to latest firmware".

```
#Imports (Place library imports HERE!)
import os, sys, io
import M5
from M5 import *
import time

def setup():
    M5.begin()
    #Place initialization codes HERE!

def loop():
    M5.update()
    #Place application codes HERE!

#Do not modify.
if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")
```

Listing 4-1 - basic MicroPython program template for the M5StickS3 (app_template.py)

4.1 LCD Display

The M5StickS3 is equipped with a built-in 1.14-inch LCD display with a resolution of 135 × 240 pixels. The LCD provides a local visual interface for MCU applications, allowing the device to show text output, sensor readings, status messages, warning indicators, and simple graphical elements directly on the module without requiring an

external monitor. From a technical perspective, the LCD is controlled through the M5StickS3 firmware and MicroPython display libraries, allowing learners to write programs that update the screen dynamically during execution. This makes it useful for debugging, user interaction, menu display, real-time monitoring, and standalone embedded applications where system information needs to be shown directly on the device.

4.1.1 Basic LCD Control

Listing 4-2 shows a basic LCD control program **lcd_basic.py** that demonstrates how to configure the M5StickS3 LCD display. The program allows the LCD background color, brightness level, and display orientation to be set according to the intended application.

```
import os, sys, io
import M5
from M5 import *
import time

def setup():
    M5.begin()
    Widgets.setBrightness(150)           #Set LCD brightness (0 - 255)
    Widgets.fillScreen(0xffff00)         #Set LCD background color: 0xRRGGBB
    Widgets.setRotation(0)               #Set LCD screen rotation (0, 1, 2, 3) => 0, 90, 180, 270

def loop():
    M5.update()

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")
```

Listing 4-2 - Basic LCD control program (lcd_basic.py).

The statement **Widgets.setBrightness(150)** sets the brightness level of the LCD backlight. The input parameter for **setBrightness()** ranges from **0** to **255**, where a lower value makes the display dimmer and a higher value makes the display brighter. In this example, the brightness is set to **150**, which provides a moderate brightness level. This setting is useful for improving screen visibility under different lighting conditions or reducing power consumption when a lower brightness level is sufficient.

The statement **Widgets.fillScreen(0xFFFF00)** fills the LCD screen with a yellow colour. The input parameter for **fillScreen()** is based on the RGB hexadecimal color format **0xRRGGBB**, where **RR** represents the red value, **GG** represents the green value, and **BB** represents the blue value. Each color component ranges from **00** to **FF**. In this example, **0xFFFF00** means **red** is at maximum, **green** is at maximum, and **blue** is zero, which produces **yellow**. Since this instruction appears after the earlier black screen command, the final screen color displayed by the program will be **yellow**.

The statement **Widgets.setRotation(0)** sets the orientation of the LCD display. The input parameter for **setRotation()** can be **0**, **1**, **2**, or **3** corresponding to **0**, **90**, **180**, or **270** rotation angle, representing the screen rotation angle in degrees. A rotation value of **0** displays the screen in its default orientation. Changing this value

allows the LCD output to be rotated to match how the M5StickS3 is held or mounted. For example, **Widgets.setRotation(90)** rotates the display by **90** degrees, while **Widgets.setRotation(180)** and **Widgets.setRotation(270)** rotate the display further accordingly.

4.1.2 LCD Text Display

Listing 4-3 shows **lcd_write_text.py** program which displays text directly on the M5StickS3 LCD using defined text colors. Instead of showing output only in the Thonny Shell, the program sends text to the device screen, allowing the M5StickS3 to function as a simple standalone display.

In this example, **label0** and **label1** are first created using **Widgets.Label()**. Both labels are initially assigned the placeholder text "TEXT". The **x, y** position values **10, 40** and **10, 80** specify where the two text labels will appear on the LCD screen. **label0** uses a font scale of **1.5** and displays text in red using the RGB hexadecimal color code **0xFF0000**, while **label1** uses a font scale of **1.0** and displays text in blue using **0x0000FF**. Both labels use a black background color, represented by **0x000000**, and the **Montserrat18** font.

The **setText()** function is then used to update the displayed text for each label. **label0.setText(str('M5StickS3'))** changes **label0** to display "M5StickS3", while **label1.setText(str('Nanyang Polytechnic'))** changes **label1** to display "Nanyang Polytechnic". This shows how labels can be created first and updated later during program execution.

```

import os, sys, io
import M5
from M5 import *

label0 = None
label1 = None

def setup():
    global label0, label1
    M5.begin()
    Widgets.setBrightness(150)           #Set LCD brightness to 150 (out of 255).
    Widgets.fillScreen(0x000000)         #Set LCD background color to 0x000000 (Black)
    Widgets.setRotation(1)               #Rotate LCD display by 90 degree.
    #Initialize text label0 and label1
    label0 = Widgets.Label("TEXT", 10, 40, 1.5, 0xFF0000, 0x000000, Widgets.FONTS.Montserrat18)
    label1 = Widgets.Label("TEXT", 10, 80, 1.0, 0x0000FF, 0x000000, Widgets.FONTS.Montserrat18)

def loop():
    M5.update()
    #Display texts at label0 ND and label1
    label0.setText(str('M5StickS3'))
    label1.setText(str('Nanyang Polytechnic'))

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")

```

Listing 4-3 - lcd_write_text.py program

4.1.3 LCD Graphic Display

Listing 4-4 shows **lcd_draw.py** which demonstrates how simple graphics can be drawn on the M5StickS3 LCD using the Widgets graphics functions. In this example, a title and several circles are created to form a simple “Crazy Face” display. The **Widgets.Title()** function creates a title bar at the top of the screen with the text "Crazy Face". The value **3** sets the title position or style parameter, **0xFFFFF** sets the title text color to **white**, **0x0000FF** sets the title background color to blue, and **Widgets.FONTS.Montserrat18** selects the font type used for the title.

The **Widgets.Circle()** function is then used to draw circular shapes on the LCD. **circle0** and **circle1** create the two larger white circles that represent the eyes of the face. For example, **Widgets.Circle(62, 87, 33, 0xFFFFF, 0xFFFFF)** draws a circle with its center at **x-coordinate 62** and **y-coordinate 87**, with a radius of **33 pixels**. The first color value sets the circle outline color, while the second color value sets the filled color. Since both values are **0xFFFFF**, the circle appears fully white. Similarly, **circle1** draws another white eye at a different screen location.

The smaller circles, **circle2** and **circle3**, are drawn inside or near the larger white circles to represent the pupils. These circles use a smaller radius of **15 pixels** and darker fill colors such as **0x181717** and **0x0D0C0C**. This creates a simple cartoon-style face on the LCD. Through this listing, learners can see how basic graphical objects such as

titles and **circles** can be positioned using **x-y coordinates**, sized using radius values, and customized using hexadecimal RGB color codes.

```
import os, sys, io
import M5
from M5 import *

title0 = None
circle0 = None
circle1 = None
circle2 = None
circle3 = None

def setup():
    global title0, circle0, circle1, circle2, circle3

    M5.begin()
    Widgets.setRotation(1)
    Widgets.fillScreen(0x7F7F00)
    title0 = Widgets.Title("Crazy Face", 3, 0xFFFFFFFF, 0x0000FF, Widgets.FONTS.Montserrat18)
    circle0 = Widgets.Circle(62, 87, 33, 0xFFFFFFFF, 0xFFFFFFFF)
    circle1 = Widgets.Circle(173, 71, 33, 0xFFFFFFFF, 0xFFFFFFFF)
    circle2 = Widgets.Circle(54, 78, 15, 0xFFFFFFFF, 0x181717)
    circle3 = Widgets.Circle(186, 74, 15, 0xFFFFFFFF, 0x0D0C0C)

def loop():
    global title0, circle0, circle1, circle2, circle3
    M5.update()

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")
```

Listing 4-4 - lcd_draw.py program

Self-explore

Explore <https://uiflow2.m5stack.com/> for programming shortcuts to create graphics on LCD.

4.2 Push-Buttons and Buzzer

Listing 4-5 shows **buzzer_buttons.py** which demonstrates how the built-in push buttons on the M5StickS3 can be used to trigger audio output through the built-in speaker. The program defines two button event functions: **btnA_wasPressed_event()** and **btnB_wasPressed_event()**. When **Button A** is pressed, the function **btnA_wasPressed_event()** is called and the speaker generates a tone at **2000 Hz** for **100 ms** using **Speaker.tone(2000, 100)**. When **Button B** is pressed, the function **btnB_wasPressed_event()** is called and the speaker generates a higher tone at **4000 Hz** for **100 ms** using **Speaker.tone(4000, 100)**.

The statements **BtnA.setCallback()** and **BtnB.setCallback()** are used to register callback functions for the button press events. The callback type **WAS_PRESSED** means that the assigned function will run once when the respective button is pressed. The statement **Speaker.setVolumePercentage(100)** sets the speaker volume to

100%. Overall, this listing shows how button inputs can be used to produce different audio feedback tones, forming a simple event-driven interaction on the M5StickS3.

```
import os, sys, io
import M5
from M5 import *

def btnA_wasPressed_event(state):
    Widgets.fillScreen(0x00FF00)

def btnB_wasPressed_event(state):
    Widgets.fillScreen(0xFF0000)

def setup():
    M5.begin()
    Widgets.setRotation(1)
    Widgets.fillScreen(0x000000)
    BtnA.setCallback(type=BtnA.CB_TYPE.WAS_PRESSED, cb=btnA_wasPressed_event)
    BtnB.setCallback(type=BtnB.CB_TYPE.WAS_PRESSED, cb=btnB_wasPressed_event)

def loop():
    M5.update()

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")
```

Listing 4-5 - buzzer_buttons.py program

4.3 Real-Time Clock

The M5StickS3 includes a Real-Time Clock (RTC) that allows the microcontroller to keep track of the current date and time. The RTC can be used in applications that require timestamps, scheduled operations, alarms, data logging, or time-based control. For example, an environmental monitoring system can record the date and time together with each sensor reading. The RTC operates independently from the main application timing functions, making it more suitable for maintaining calendar time than using delays or elapsed-time counters. However, the RTC must first be set to the correct date and time, either manually or by synchronizing it through a network time service when Wi-Fi is available.

4.3.1 Alarm Clock

The **alarm_clock.py** program turns the M5StickS3 into a simple 24-hour digital clock with an alarm function. The current date and time are stored and maintained using the Real-Time Clock, while the LCD displays the time, date, alarm setting, and alarm status. Button A enables the alarm, while Button B disables the alarm and stops the alarm sound.

The program begins by importing the required libraries based on previously introduced template **app_template.py**. The **M5** library provides access to the M5StickS3 hardware. In this section, **RTC** from the machine library is added to manage the date and time. The **time** library provides timing functions such as **ticks_ms()** and **sleep_ms()**.

The initial date and time are hardcoded using variables such as **INITIAL_YEAR**, **INITIAL_MONTH**, **INITIAL_DAY**, **INITIAL_HOUR**, **INITIAL_MINUTE**, and **INITIAL_SECOND**. The weekday is also provided, where **0** represents **Monday**. These values are used to set the **RTC** when the program starts.

The alarm time is hardcoded using **ALARM_HOUR** and **ALARM_MINUTE**. Since the clock uses the 24-hour format, an alarm hour of **14** represents **2:00 PM**. Learners can modify these two values to change the alarm time.

The variables **alarm_enabled** and **alarm_ringing** are used to control the alarm. **alarm_enabled** indicates whether the alarm function is active, while **alarm_ringing** indicates whether the current time matches the alarm time and the speaker should produce a sound.

The function **btnA_wasPressed_event()** is called when Button A is pressed. It sets **alarm_enabled** to **True**, clears the ringing state, and updates the LCD to show that the alarm is enabled. The text color is changed to **green** to provide a clear visual indication.

The function **btnB_wasPressed_event()** is called when Button B is pressed. It disables the alarm, clears the ringing state, stops the speaker, and updates the LCD to show that the alarm is disabled. The status text is displayed in **red**.

The function **update_clock_display()** formats the current time as **HH:MM:SS** and the date as **DD/MM/YYYY**. The format specifier **{:02d}** ensures that single-digit values are displayed with a leading zero. For example, **8:05:03** is shown as **08:05:03**.

The function **check_alarm()** compares the current hour and minute with the hardcoded alarm time. If the alarm is enabled and the current time matches the alarm time, **alarm_ringing** is set to **True**. The alarm remains active for the matching minute. Once the minute changes, **alarm_ringing** is set back to **False**.

The function **sound_alarm()** produces the alarm sound. It checks whether **alarm_ringing** is **True** and uses **time.ticks_ms()** to generate one short beep every **500 milliseconds**. The statement **Speaker.tone(2500, 200)** produces a 2500 Hz tone lasting 200 milliseconds.

In the **setup()** function, **M5.begin()** initializes the M5StickS3 hardware. The LCD rotation, background color, and brightness are then configured. The built-in speaker is initialized and set to 80% volume.

The **RTC** is set using:

```
rtc.datetime((
    INITIAL_YEAR,
    INITIAL_MONTH,
    INITIAL_DAY,
    INITIAL_WEEKDAY,
    INITIAL_HOUR,
    INITIAL_MINUTE,
    INITIAL_SECOND,
    0
))
```

This tuple contains the year, month, day, weekday, hour, minute, second, and subsecond values. After this statement runs, the **RTC** continues counting time automatically.

The **setup()** function creates four LCD labels. **label_time** displays the current time, **label_date** displays the current date, **label_alarm** displays the configured alarm time, and **label_status** shows whether the alarm is enabled or disabled. The Button A and Button B callback functions are then registered.

The **loop()** function runs continuously. The current date and time are read using **rtc.datetime()**, and the individual values are extracted from the returned tuple.

The LCD is updated only when the second value changes. This reduces unnecessary screen updates and helps prevent flickering. During each update, the program displays the new time and checks whether the alarm time has been reached. The **sound_alarm()** function is then called repeatedly to generate the alarm beeps when required.

4.3.2 Countdown Timer

The **countdown_timer.py** program turns the M5StickS3 into a simple preset timer with an alarm. Button A is used to increase the timer setting in 5-second steps, while Button B starts or stops the countdown. When the timer reaches zero, the built-in **speaker** produces an alarm sound. Pressing Button B while the alarm is ringing stops the alarm and restores the original preset time.

The program begins by importing the required libraries. In this section, the **RTC** class is added to provide the real-time clock object, and the **time** library is used for timing functions such as **time.time()**, **time.ticks_ms()**, and **time.sleep_ms()**.

The constants **TIME_INCREMENT** and **MAX_TIME** control how the timer is set. **TIME_INCREMENT** is set to 5, so each press of Button A adds 5 seconds. **MAX_TIME** is set to **180**, which limits the timer to a maximum of 180 seconds, or 3 minutes.

The variable **preset_seconds** stores the original timer value selected by the user. The variable **remaining_seconds** stores the amount of time still left during the countdown. The Boolean variable **counting** indicates whether the timer is currently running, while **alarm_ringing** indicates whether the alarm is active.

The variable **countdown_end_time** stores the exact future time when the countdown should end. **last_display_second** is used to avoid updating the LCD unnecessarily when the displayed second has not changed. **last_beep_time** controls the timing between repeated alarm sounds.

The function **format_time()** converts a value in seconds into the **MM:SS** format. Integer division using **//** calculates the number of whole minutes, while the remainder operator **%** calculates the remaining seconds. For example, 125 seconds is displayed as 02:05.

The function **btnA_wasPressed_event()** runs when Button A is pressed. The timer setting cannot be changed while the countdown is running or while the alarm is sounding. If the preset has already reached 180 seconds, the next press resets it to zero. Otherwise, 5 seconds is added. The selected value is copied to **remaining_seconds** and displayed on the LCD.

The timer-setting sequence therefore cycles as follows:

```
00:00 → 00:05 → 00:10 → ... → 02:55 → 03:00 → 00:00
```

The function **btnB_wasPressed_event()** performs different actions depending on the timer state. If the alarm is ringing, Button B stops the speaker, clears the alarm state, and restores **remaining_seconds** to the original **preset_seconds** value. This allows the same timer setting to be used again.

If the countdown is currently running, Button B stops it and keeps the remaining time on the screen. If the timer is not running and the remaining time is greater than zero, Button B starts the countdown. The program calculates the finishing time by adding the remaining seconds to the current time:

```
countdown_end_time = time.time() + remaining_seconds
```

The function **update_countdown()** calculates the remaining time by subtracting the current time from the stored ending time:

```
remaining_seconds = int(countdown_end_time - time.time())
```

Using an end time is more reliable than simply subtracting one every loop because the loop may not run at exactly one-second intervals. When the calculated time reaches zero, the countdown stops, **alarm_ringing** becomes **True**, and the LCD displays TIME UP!.

The function **sound_alarm()** generates a repeating alarm tone. It checks whether at least 500 milliseconds have passed since the previous beep. If so, the speaker produces a 2500 Hz tone lasting 200 milliseconds. This creates a repeated beeping effect without blocking the rest of the program.

In the **setup()** function, the **RTC** is initialized with hardcoded initial date and time. Although the timer calculation mainly uses **time.time()**, setting the **RTC** provides a valid system time reference for the device.

Four LCD labels are created. **label_title** displays the application title, **label_time** displays the remaining time, **label_status** shows the current timer state, and **label_instruction** displays the button instructions. Callback functions are then assigned to Button A and Button B.

The **loop()** function runs continuously. If the countdown is running, **update_countdown()** calculates the remaining time. The LCD is updated only when the number of remaining seconds changes. The **sound_alarm()** function is called during every loop so the alarm can continue beeping when required.

4.4 Microphone

The M5StickS3 is equipped with a built-in microphone that allows the device to capture sound for audio-related applications. The microphone can be used to measure sound levels, detect claps or voice activity, record short audio clips, and analyze sound signals in the time or frequency domain. The captured audio data can be processed by the microcontroller, displayed on the LCD, or played back through the built-in speaker. This makes the microphone useful for developing simple sound meters, voice-triggered systems, sound recorders, and basic audio analysis projects.

4.4.1 Sound Pressure Measurement

The **simple_spl_meter.py** program creates a simple sound pressure level meter application using the built-in **microphone** and LCD on the M5StickS3. The program uses the microphone to collect sound samples, estimates the sound level in decibels, and displays the result on the LCD. Button A is used to start the measurement, while Button B is used to stop the measurement.

The program begins by importing the required libraries. In this section, the **math** library is added to calculate the estimated dB value. The **time** library is used to add a short delay between readings, and the **array** library is used to create a storage buffer for microphone samples.

The variable **measuring** is used to control whether sound measurement is active or stopped. When **measuring** is set to **True**, the program will continuously read sound data from the microphone. When it is set to **False**, the program stops updating the sound level. The labels **label_title**, **label_db**, and **label_status** are used to display the title, measured dB value, and sound status on the LCD.

The function **estimate_db()** processes the microphone samples stored in the audio buffer. It first calculates the RMS value of the sound samples. RMS, or root mean square, gives an estimate of the average sound signal strength. The program then converts this RMS value into an estimated dB value using a logarithmic calculation. This reading is suitable for demonstration purposes, but it is not a calibrated sound pressure level measurement. For accurate SPL measurement, the M5StickS3 microphone must be calibrated against a proper sound level meter.

The function **btnA_wasPressed_event()** is called when Button A is pressed. It sets **measuring** to **True**, clears the LCD screen, and updates the display to show that measurement has started. The function **btnB_wasPressed_event()** is called when Button B is pressed. It sets **measuring** to **False**, clears the LCD screen, and updates the display to show that measurement has stopped.

In the **setup()** function, the program registers the callback functions for Button A and Button B. The microphone is initialized, and the sampling rate is set so that the device is ready to collect sound samples.

The **loop()** function runs repeatedly after setup is completed. If **measuring** is **True**, the program creates an audio buffer, records microphone samples, calculates the estimated dB value, and updates the LCD display. The program also classifies the sound level as **Low**, **Medium**, or **High** based on the estimated dB value.

4.4.2 Sound Recorder and Playback

The **sound_recorder.py** program uses the M5StickS3 built-in **microphone** to record sound and the built-in **speaker** to play back the recorded sound. The program also uses the built-in LCD to show the current system status. Button A is used to start sound recording, while Button B is used to play or stop the recorded sound.

The program begins by importing the required libraries. In this section, the **time** library is added to create a short delay in the main loop. Several global variables are then created. The variable **recording** is used to indicate whether the microphone is currently recording. The variable **playing** indicates whether the speaker is currently playing back audio. The variable **recorded** indicates whether a valid recording has been completed and is available for playback. The variable **rec_data** is used to store the recorded sound data.

The audio settings are defined using **SAMPLE_RATE**, **MAX_RECORD_TIME**, and **BUFFER_SIZE**. In this program, **SAMPLE_RATE** is set to **8000**, which means the microphone samples sound at 8000 samples per second. **MAX_RECORD_TIME** is set to **10**, so the maximum recording duration is 10 seconds. **BUFFER_SIZE** is calculated using **SAMPLE_RATE * MAX_RECORD_TIME**, giving the program enough memory space to store up to **10** seconds of recorded sound.

The function **btnA_wasPressed_event()** runs when Button A is pressed. If the system is already recording or playing audio, the function will return and no new action will be taken. This prevents recording and playback

from happening at the same time. If the system is ready, the program sets **recorded** to **False** and **recording** to **True**, clears the LCD screen, and updates the display to show that recording has started. A new **bytearray** is created for **rec_data**, which means each new recording overwrites the previous recording. The microphone is then started using **Mic.begin()**, and **Mic.record(rec_data, SAMPLE_RATE, False)** records sound into the **rec_data** buffer. The third parameter **False** is used to set the recording mode to non-blocking. This means the microphone starts recording into the **rec_data** buffer, but the program does not pause and wait at that line until the full recording is completed.

The function **btnB_wasPressed_event()** runs when Button B is pressed. If the system is still recording, the function returns and playback will not start. If no recording has been completed, the LCD displays "No recording" and prompts the learner to press Button A first. If a recording is available and playback is not currently active, the program starts the speaker, sets the volume to 80%, and plays the recorded sound using **Speaker.playRaw(rec_data, SAMPLE_RATE)**. If playback is already active and Button B is pressed again, the program stops the speaker and updates the LCD to show that playback has stopped.

In the **setup()** function, three text labels are created on the LCD. These labels are used to show the application title, current status, and user instruction. A **bytearray** buffer is created to prepare memory space for sound recording. The program then registers Button A and Button B callback functions, allowing the device to respond when the buttons are pressed.

The **loop()** function runs continuously after setup is completed. If the system is recording, the program checks **Mic.isRecording()** to determine whether the recording process has finished. Once recording is completed, the program sets **recording** to **False**, sets **recorded** to **True**, ends microphone operation using **Mic.end()**, and updates the **LCD** to show that the recording has been saved. If the system is playing audio, the program checks **Speaker.isPlaying()** to determine whether playback has finished. When playback ends, the program resets the playing state, ends speaker operation using **Speaker.end()**, and updates the display to show that playback is complete.

4.5 Inertial Measurement Unit

The M5StickS3 includes a built-in Inertial Measurement Unit (IMU) that can detect movement, acceleration, tilt, orientation, and vibration. The IMU combines **accelerometer** and **gyroscope** sensing, allowing the microcontroller to measure motion along the x, y, and z axes as well as changes in rotational movement. The sensor data can be used to determine whether the device is tilted, shaken, rotated, or kept stable. This makes the IMU suitable for applications such as tilt meters, motion detectors, posture monitoring, vibration sensing, gesture control, and simple activity-tracking systems.

4.5.1 Tilt Meter

The **tilt_meter.py** implements a simple IMU Tilt Meter program for the M5StickS3. The program reads acceleration values from the built-in IMU sensor and uses the values to determine whether the device is tilted left, right, up, down, or kept flat. The tilt direction, acceleration readings, estimated tilt angle, and a simple moving dot indicator are displayed on the LCD.

The program begins by importing the required libraries. In this section, the **time** library is added to control how often the screen is updated, while the **math** library is used to calculate the estimated tilt angle.

The variables **label_status**, **label_ax**, **label_ay**, **label_az**, and **label_angle** are used to store LCD label objects. These labels are created in the **setup()** function and later updated in the **loop()** function. The variable **last_time** is used to keep track of the last update time so that the screen is updated about 10 times per second instead of updating too quickly.

The function **get_tilt_direction(ax, ay)** determines the tilt direction of the M5StickS3. It uses a threshold value of **0.35** to decide whether the tilt is significant enough to be detected. If the acceleration value on the x-axis is greater than 0.35, the program returns "Tilt Right". If it is less than -0.35, it returns "Tilt Left". Similarly, the y-axis value is used to detect "Tilt Up" and "Tilt Down". If both x-axis and y-axis readings are within the threshold range, the device is considered "Flat / Stable".

The function **get_tilt_angle(ax, ay, az)** estimates the tilt angle of the device using the acceleration values from the x, y, and z axes. The x-axis and y-axis values are first combined to calculate the horizontal acceleration. The program then compares this horizontal value with the z-axis value to estimate the tilt angle. The result is converted from radians to degrees so that it is easier for learners to understand.

The function **draw_tilt_indicator(ax, ay)** creates a simple visual indicator on the LCD. It first draws a black rectangle at the lower part of the screen to clear the previous dot position. The center point is set at $x = 120$ and $y = 108$. The acceleration values ax and ay are then converted into screen movement. When the device is tilted, the green dot moves away from the center point. The program also limits the dot position so that it does not move outside the visible screen area.

In the **setup()** function, several labels are created on the LCD to display the tilt status, acceleration readings, and estimated angle. The variable **last_time** is then set to the current time using **time.time()**.

The **loop()** function runs continuously after setup is completed. The program then checks whether 100 milliseconds have passed since the last update. This gives an update rate of about 10 times per second. If enough time has passed, the program reads the accelerometer values using **Imu.getAccel()**. The values are stored as ax , ay , and az , representing acceleration along the x-axis, y-axis, and z-axis respectively.

After reading the IMU values, the program calls **get_tilt_direction(ax, ay)** to determine the tilt direction and **get_tilt_angle(ax, ay, az)** to estimate the tilt angle. The LCD labels are then updated to show the current status, acceleration readings, and angle. Finally, **draw_tilt_indicator(ax, ay)** is called to update the moving dot indicator on the screen.

4.5.2 Vibration Sensor

The **vibration_sensor.py** program uses the M5StickS3 built-in **IMU** to detect vibration and activate an alarm when the measured vibration exceeds a preset threshold. Button A is used to increase the threshold in steps of 0.1g, while Button B starts or stops vibration measurement. If an alarm is triggered, pressing Button B stops the alarm and ends the measurement.

The program begins by importing the required libraries. In this section, the **time** library is added to control sampling and alarm timing, and the **math** library is used to calculate the overall vibration value from the acceleration changes on the x, y, and z axes.

The constants define the operating limits and timing settings. **THRESHOLD_INCREMENT** is set to **0.1**, so each press of Button A increases the vibration threshold by 0.1g. **MIN_THRESHOLD** is 0.1g, while **MAX_THRESHOLD** is

16.0g. SAMPLE_INTERVAL_MS sets the **IMU** sampling interval to 50 milliseconds. The alarm frequency, tone duration, and interval between beeps are also defined as constants.

The variable **vibration_threshold** stores the current vibration threshold. The Boolean variable **measuring** indicates whether vibration measurement is active, while **alarm_ringing** indicates whether the vibration alarm has been triggered. The variables **previous_ax**, **previous_ay**, and **previous_az** store the previous acceleration values so that the program can calculate how much the acceleration has changed between two readings.

The function **calculate_vibration()** calculates the vibration level. It subtracts the previous acceleration values from the current values to obtain the changes on the three axes:

```
delta_x = ax - previous_ax
delta_y = ay - previous_ay
delta_z = az - previous_az
```

These changes are combined using the three-dimensional magnitude formula:

```
vibration = math.sqrt(
    (delta_x * delta_x) +
    (delta_y * delta_y) +
    (delta_z * delta_z)
)
```

A small value indicates little movement, while a larger value indicates stronger vibration or sudden movement. After the calculation, the current readings are saved for use during the next measurement.

The function **btnA_wasPressed_event()** runs when Button A is pressed. The threshold cannot be changed while measurement is active or while the alarm is ringing. If the current threshold has reached **16.0g**, the next press resets it to **0.1g**. Otherwise, the threshold increases by **0.1g**. The **round()** function is used to reduce floating-point rounding errors.

The threshold therefore cycles as follows:

```
0.1g → 0.2g → 0.3g → ... → 15.9g → 16.0g → 0.1g
```

The function **btnB_wasPressed_event()** controls the measurement and alarm states. If the alarm is ringing, Button B stops the speaker, clears the alarm state, and stops measurement. If measurement is already active, Button B stops it. If neither condition is active, Button B reads the current **IMU** acceleration values, stores them as the starting reference, and begins vibration measurement.

The function **measure_vibration()** reads the latest acceleration values from the **IMU** using **Imu.getAccel()**. It then calls **calculate_vibration()** and displays the calculated vibration value on the LCD. If the vibration level is equal to or greater than the selected threshold, measurement stops and the alarm state becomes active.

The function **sound_alarm()** produces a repeating alarm sound. It checks whether the alarm is active and whether enough time has passed since the previous beep. When the interval reaches 500 milliseconds, the speaker generates a 2500 Hz tone lasting 200 milliseconds. This creates a repeated warning beep without blocking the main program.

In the **setup()** function, Several LCD labels are created to display the application title, vibration threshold, measured vibration level, system status, and button instructions. The callback functions for Button A and Button B are also registered. This allows the program to respond immediately when either button is pressed. The initial sampling and alarm timing values are then stored using **time.ticks_ms()**.

The **loop()** function runs continuously. If measurement is active, the program checks whether 50 milliseconds have passed since the previous sample. When the interval is reached, **measure_vibration()** is called. The **sound_alarm()** function is also called during every loop so that the alarm continues beeping when required.

4.5.3 Pedometer

The **pedometer.py** program uses the M5StickS3 built-in **IMU** to detect walking steps and estimate the number of calories burned. Button A starts or stops step monitoring, while Button B resets the step count and calorie value.

The program begins by importing the required libraries. In this section, the **time** library is added to control the sampling interval, and the **math** library is used to calculate the combined acceleration magnitude.

The constants define how the pedometer operates. **SAMPLE_INTERVAL_MS** is set to 50 milliseconds, so the **IMU** is read about 20 times per second. **STEP_THRESHOLD** sets the movement level required to detect a possible step, while **STEP_RESET_LEVEL** determines when the movement has fallen enough for the next step to be detected. **MIN_STEP_INTERVAL_MS** prevents steps from being counted too quickly. **CALORIES_PER_STEP** provides a simple calorie estimate of **0.04 kcal** for each detected step.

The variable **monitoring** indicates whether step monitoring is active. **step_count** stores the total number of detected steps, while **calories_burned** stores the estimated calorie value. The variable **step_peak_detected** is used to prevent one movement peak from being counted more than once.

The function **calculate_magnitude()** combines the acceleration values from the x, y, and z axes:

```
magnitude = math.sqrt((ax * ax) + (ay * ay) + (az * az))
```

This produces one overall acceleration value regardless of how the M5StickS3 is positioned.

The function **detect_step()** determines whether the current movement represents a step. Since a stationary device normally measures about 1g because of gravity, the program removes this gravity component using:

```
movement = abs(magnitude - 1.0)
```

If the movement is greater than or equal to **STEP_THRESHOLD**, and a step has not already been detected, the program checks that at least 300 milliseconds have passed since the previous step. If these conditions are met, the function returns **True**.

After a step is detected, **step_peak_detected** remains **True** until the movement falls below **STEP_RESET_LEVEL**. This ensures that one walking motion is counted only once.

The function **update_display()** updates the LCD with the current number of steps and estimated calories. The calorie value is displayed to two decimal places.

The function **btnA_wasPressed_event()** starts or stops monitoring. When monitoring begins, the step-detection state and timing variables are reset. The LCD status changes to "Monitoring". When Button A is pressed again, monitoring stops and the status changes to "Stopped".

The function **btnB_wasPressed_event()** resets the step count and calorie estimate to zero. It also clears the step-detection state and updates the LCD. The reset can be performed whether monitoring is active or stopped.

The function **monitor_steps()** reads the current accelerometer values using **Imu.getAccel()**. It sends the x, y, and z values to **detect_step()**. If a step is detected, **step_count** increases by one and the calorie estimate is recalculated using:

```
calories_burned = step_count * CALORIES_PER_STEP
```

The **setup()** function creates labels for the title, step count, calorie estimate, monitoring status, and button instructions. It also registers the Button A and Button B callback functions.

The **loop()** function runs continuously. When monitoring is active, the program checks whether 50 milliseconds have passed since the previous **IMU** reading. If so, it calls **monitor_steps()** to check for a new step.

5. Using M5StickS3 External Peripherals

The M5StickS3 can be expanded using external peripherals to support a wider range of sensing, identification, control, and automation activities. In this course, PaHub, PbHub, and Unit Hub modules are used to increase the number of available connection ports, allowing several sensors and output devices to be connected to the M5StickS3 at the same time. The Ultrasonic Sensor can be used to measure distance, while the Light Sensor detects ambient brightness and the Color Sensor identifies color information from nearby objects. The ENV-Pro Sensor provides environmental measurements such as temperature, humidity, air pressure, and gas-related data, while the Angle Sensor allows learners to detect the position of a rotary control. The PIR Sensor detects human movement, the Dual Button module provides additional push-button inputs, and the RFID Unit can read RFID tags for identification, access control, attendance tracking, and object-tagging applications.

Other external modules extend the M5StickS3 with advanced motion sensing and actuator control. The ADXL345 Accelerometer can measure acceleration, tilt, orientation, and vibration along three axes, making it suitable for movement and condition-monitoring applications. The Servo Motor can be controlled to rotate to a required position or perform simple mechanical actions, while the 4 Relay module allows the M5StickS3 to switch several external electrical loads independently. Together, these peripherals allow learners to practice connecting multiple devices, reading sensor and RFID data, processing conditions, controlling outputs, and designing integrated MCU systems that respond to users, tagged objects, and changes in the surrounding environment.

5.1 Expanding M5StickS3 GPIO and Communication Ports

The M5StickS3 has a limited number of built-in expansion ports, so hub modules are used when multiple external peripherals need to be connected within the same system. In this configuration, the Unit Hub is connected directly to the M5StickS3 to provide additional expansion points. The M5Stack PaHUB and PbHUB are then connected through the Unit Hub, forming a structured expansion arrangement that supports a wider range of sensors, input devices, and actuator modules. Figure 5-1 illustrates a typical port expansion connection diagram showing how the Unit Hub, PaHUB, and PbHUB are interconnected with the M5StickS3.

The PaHUB expands the M5StickS3 I2C communication interface by providing up to six I2C ports for compatible digital sensors and modules. The PbHUB complements this expansion by providing six additional analog and digital GPIO ports for peripherals such as push buttons, angle sensors, PIR sensors, and other input or output devices. Together, the Unit Hub, PaHUB, and PbHUB significantly increase the number of available connections, enabling the M5StickS3 to support up to six I2C devices and six analog or digital GPIO devices. This arrangement provides greater flexibility for developing more advanced MCU applications that integrate multiple sensors, user inputs, and actuator outputs within a single system.

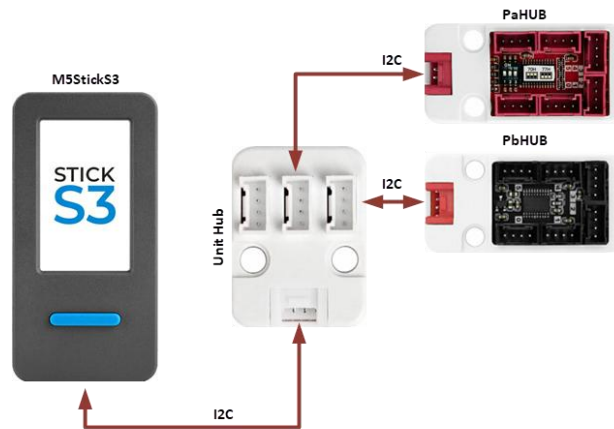


Figure 5-1 - M5StickS3 Ports Expansion

Listing 5-1 presents the modified application program template that extends the basic M5StickS3 program structure to support both the PaHUB and PbHUB expansion modules. In addition to the standard M5StickS3 libraries, the program imports the **Pin** and **I2C** classes from the hardware library, together with the **PAHUBUnit** and **PBHUBUnit** libraries required to control the two hub modules. The variables **i2c0**, **pahub_0**, and **pbhub_0** are declared as global objects so that they can be accessed throughout the program.

Within the **setup()** function, **GPIO Pin 10** is configured as the **I2C clock** line, while **GPIO Pin 9** is configured as the **I2C data** line. The I2C bus operates at a clock frequency of **100 kHz**. The PaHUB is then initialized on Channel 0 of the I2C bus, while the PbHUB is initialized using its assigned I2C address of 0x61. These additional initialization steps allow the M5StickS3 to communicate with external sensors, input devices, and actuator modules connected through the expanded I2C and GPIO interfaces. The modified template therefore serves as the starting point for subsequent applications that require multiple peripherals to operate within the same system.

```

import os, sys, io
import M5
from M5 import *
from hardware import Pin
from hardware import I2C          #I2C Library
from unit import PAHUBUnit        #PaHUB Library (Include if PaHUB is connected)
from unit import PBHUBUnit        #PbHUB Library (Include if PbHUB is connected)

i2c0 = None                        #Added to support external peripherals connecting to I2C Bus
pahub_0 = None                    #Added for PaHUB
pbhub_0 = None                    #Added for PbHUB

def setup():
    global i2c0, pahub_0, pbhub_0

    M5.begin()
    Widgets.setRotation(1)
    Widgets.fillScreen(0x000000)

    #I2C peripherals Initialization codes
    #Set M5StickS3 GPIO Pin 9 and 10 to be I2C Bus port with Bus clock of 100KHz
    i2c0 = I2C(0, scl=Pin(10), sda=Pin(9), freq=100000)
    pahub_0 = PAHUBUnit(i2c=i2c0, channel=0)      #PaHUB Initialization
    pbhub_0 = PBHUBUnit(i2c0, 0x61)              #PbHUB Initialization

def loop():
    global i2c0, pahub_0, pbhub_0
    M5.update()

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("please update to latest firmware")

```

Listing 5-1 – Application template with expanded I2C Bus and GPIO support (app_template_exp.py)

5.2 GPIO based External Peripherals

The PbHUB is used to expand the M5StickS3 with additional analog and digital GPIO ports for connecting external input and output devices. It provides multiple selectable ports that allow several peripherals to be monitored or controlled through a single I2C connection. In this course, learners explore the M5Stack **Dual-Button Unit, Passive IR (PIR) Sensor Unit, Light Sensor Unit, Angle Sensor Unit, and 360-degree servo motor**. These peripherals provide practical experience in reading digital button inputs, detecting human movement, measuring ambient light levels, obtaining analog position values, and controlling continuous rotational movement. Together, they support the development of applications such as user-input systems, motion-triggered automation, automatic lighting, manual speed or direction control, and simple robotic or actuator-based systems.

5.2.1 Dual-Button

The **M5Stack Dual-Button Unit** (Figure 5-2) consists of two independent momentary push-button switches, identified by their red and blue button caps. Each button produces a digital output only while it is being pressed, allowing the M5StickS3 to detect separate user actions through the PbHUB. The two buttons can be assigned different functions, such as using the red button to start a process and the blue button to stop it, increasing or decreasing a value, selecting between operating modes, moving a servo motor in opposite directions, switching different relays, confirming or cancelling a menu option, or controlling the direction of a simple robot. The Dual-Button Unit is useful for applications that require clear manual input, including machine control panels, counters, menu navigation, game controls, access systems, and emergency start-and-stop demonstrations.



Figure 5-2 - M5Stack Mini Dual Button Unit

The **dual_button_terminal.py** program uses the M5StickS3 together with the PbHUB and M5Stack Dual-Button Unit to detect button presses. In this setup, the Dual-Button Unit is connected to **PbHUB Port 0**. When the red or blue button is pressed, the program prints the corresponding message to the terminal.

The program begins by importing the required libraries. The **PBHUBUnit** library is added to provide the functions required to read the two digital button signals through the PbHUB. The **time** library is used to control how often the buttons are checked.

The variables **BUTTON_PORT**, **RED_BUTTON_PIN**, and **BLUE_BUTTON_PIN** identify the PbHUB port and the two signal pins used by the Dual-Button Unit. The variable **BUTTON_PORT** is set to **0** because the unit is connected to PbHUB Port 0. The red and blue buttons use separate digital channels (channel 0 and channel 1 respectively) on the same port.

The variables **previous_red** and **previous_blue** store the previous state of each button. They are initially set to 1, which represents the released state. These variables allow the program to detect a new button press and prevent the same message from being printed repeatedly while a button remains held down.

In the **setup()** function, the I2C bus is created, initialized, and operates at 100 kHz. The PbHUB is then initialized using its I2C address:

```
pithub_0 = PBHUBUnit(i2c0, 0x61)
```

The program also prints a title and status messages to the terminal to show that the Dual-Button Unit is connected to PbHUB Port 0 and is waiting for user input.

The **loop()** function runs continuously and reads the red and blue button states using:

```
red_state = pbhub_0.digital_read(BUTTON_PORT, RED_BUTTON_PIN)
blue_state = pbhub_0.digital_read(BUTTON_PORT, BLUE_BUTTON_PIN)
```

The buttons use active-low logic. A value of 0 means the button is being pressed, while a value of 1 means it is released.

The program checks whether the red button has changed from released to pressed:

```
if red_state == 0 and previous_red == 1:
    print("Red button pressed")
```

The same check is performed for the blue button:

```
if blue_state == 0 and previous_blue == 1:
    print("Blue button pressed")
```

By comparing the current state with the previous state, the program prints only one message for each button press instead of printing continuously while the button is held down.

At the end of each loop, the current button states are stored as the previous states for the next reading:

```
previous_red = red_state
previous_blue = blue_state
```

The short 50-millisecond delay reduces unnecessary processor activity and also helps to minimise the effect of button bouncing while keeping the program responsive.

5.2.2 Passive IR (PIR) Sensor

The **M5Stack Passive Infrared (PIR) Motion Sensor Unit** (Figure 5-3) is used to detect movement by sensing changes in infrared radiation emitted by people and other warm objects within its detection area. When motion is detected, the unit produces a digital output that can be read by the M5StickS3 through the PbHUB, allowing the program to trigger an appropriate response. Unlike a camera, the PIR sensor does not capture images or identify individuals; it only indicates whether movement has been detected. The unit can be used for operations such as automatically switching on lights when someone enters a room, activating an alarm or buzzer when movement is detected in a restricted area, opening an automatic door, starting a ventilation system when a space becomes occupied, recording room-occupancy events, waking a device from standby mode, or sending a remote motion alert. It is suitable for security systems, energy-saving applications, smart-room automation, visitor detection, and other projects that require simple presence or movement detection.



Figure 5-3 - M5Stack PIR Motion Sensor Unit

The **pir_motion_terminal.py** program uses the M5StickS3 together with the PbHUB and M5Stack PIR Motion Sensor Unit to detect human movement. In this setup, the PIR sensor is connected to **PbHUB Port 1**. The program continuously checks the sensor and prints either "Motion detected" or "No motion detected" to the terminal.

The program begins by importing the required libraries. The **PBHUBUnit** library provides the functions required to read the PIR sensor through the PbHUB, while the **time** library is used to control how often the sensor is checked.

The constants identify the PbHUB connection used by the sensor:

```
PIR_PORT = 1
PIR_SIGNAL_PIN = 1
```

PIR_PORT = 1 means that the PIR sensor is connected to PbHUB Port 1. PIR_SIGNAL_PIN = 1 selects the second signal line of that port (channel 1), which is used to read the PIR sensor output.

In the **setup()** function, the I2C bus is created, initialized, and operates at 100 kHz. The PbHUB is then initialized using its I2C address (0x61).

The program then prints several messages to the terminal to indicate that the PIR sensor is connected to PbHUB Port 1 and is ready to detect motion.

The **loop()** function runs continuously. The PIR sensor output is then read using:

```
motion_state = pbhub_0.digital_read(PIR_PORT, PIR_SIGNAL_PIN)
```

If the returned value is 1, the program assumes that motion has been detected and prints:

```
print("Motion detected")
```

If the returned value is 0, the program prints:

```
print("No motion detected")
```

The program does not store the previous detection state. Therefore, it prints the current sensor result every time the loop runs, even if the motion condition has not changed. The 500-millisecond causes the sensor to be checked every 500 milliseconds

5.2.3 Light Sensor

The **M5Stack Light Sensor Unit** (Figure 5-4) is used to measure the surrounding light intensity through an analog output. The sensor produces a value that changes according to the amount of light falling on it, allowing the M5StickS3 to determine whether the environment is bright, dim, or dark. The reading can be displayed on the LCD, printed to the terminal, or compared with a threshold to trigger other devices. The Light Sensor Unit can be used for applications such as automatically switching on lamps when the room becomes dark, adjusting screen brightness according to ambient lighting, controlling window blinds, monitoring lighting conditions in a greenhouse, activating security lighting, detecting whether an enclosure has been opened, or recording changes in light levels over time.



Figure 5-4 - M5Stack Light Sensor Unit

The **light_intensity_terminal.py** program uses the M5StickS3 together with the PbHUB and M5Stack Light Sensor Unit to measure environmental light intensity. In this setup, the Light Sensor Unit is connected to **PbHUB Port 2**. The program reads the analog sensor value, converts it into an estimated lux value, and prints both results to the terminal every 500 milliseconds.

The program begins by importing the required libraries. The **PBHUBUnit** library provides the functions required to read the Light Sensor Unit through the PbHUB, while the **time** library is used to control how often the sensor is read.

The constant **LIGHT_PORT** is set to **2** because the Light Sensor Unit is connected to PbHUB Port 2. The calibration values **DARK_RAW**, **BRIGHT_RAW**, and **MAX_LUX** define the range used to convert the analog reading into an estimated light intensity. In this program, a value of 0 represents darkness, a value of 4095 represents maximum brightness, and the maximum estimated light intensity is set to 1,000 lux.

The **convert_to_lux()** function converts the analog sensor reading into an estimated lux value. It first ensures that the reading remains within the calibration range from 0 to 4095. Any value below the minimum is limited to 0, while any value above the maximum is limited to 4095.

The lux value is calculated using:

```
lux = ((raw_value - DARK_RAW) / (BRIGHT_RAW - DARK_RAW)) * MAX_LUX
```

This calculation maps the analog range from 0 - 4095 to an estimated light-intensity range from 0 - 1,000 lux. The result is then returned to the main program.

In the **setup()** function, the I2C bus is created, initialized, and operates at 100 kHz. The PbHUB is then initialized using its I2C address (0x61).

The program then prints several messages to the terminal to indicate that the Light Sensor Unit is connected to PbHUB Port 2 and is ready to measure environmental light intensity.

The **loop()** function runs continuously to read from the Light Sensor Unit. The sensor produces a high reading in darkness and a low reading in bright conditions. The program reverses this relationship using:

```
raw_value = 4095 - pbhub_0.analog_read(LIGHT_PORT)
```

After this conversion, a low value represents darkness and a high value represents brightness.

The analog reading is then passed to the `convert_to_lux()` function:

```
light_lux = convert_to_lux(raw_value)
```

The program prints the raw value and estimated light intensity to the terminal:

```
print("Raw: {} | Light Intensity: {:.1f} lux" .format(raw_value, light_lux))
```

The lux value is formatted to one decimal place. The program waits 500 milliseconds before taking the next reading. This causes the light level to be measured and printed approximately twice per second. The estimated lux value is based on a simple linear conversion and should be calibrated against a reference lux meter if greater accuracy is required.

5.2.4 Angle Sensor and Servo Motor

The **M5Stack Angle Sensor Unit** (Figure 5-5a) is a compact analog input module that uses a rotary potentiometer to produce a variable voltage according to the position of its control knob. When connected through the PbHUB, the M5StickS3 can read the analog value and convert it into a percentage, approximate angle, speed setting, or other control value required by the application. The unit is suitable for manually adjusting parameters such as LED brightness, motor speed, servo position, alarm thresholds, audio volume, menu values, and timing intervals. Its simple rotary control makes it useful for interactive control panels, user-input systems, calibration activities, and applications that require smooth adjustment rather than fixed button selections.

The **M5Stack 360-Degree Servo Motor** (Figure 5-5b) is a continuous-rotation actuator that can rotate clockwise or anticlockwise instead of moving to a fixed angular position like a standard positional servo. A control signal determines its direction and rotational speed, with a centre or neutral command used to stop the motor. When connected through the PbHUB, it can be controlled by the M5StickS3 using inputs from devices such as the Mini Angle Unit, Dual-Button Unit, or PIR sensor. The servo motor is suitable for applications such as driving small robot wheels, rotating display platforms, operating conveyor mechanisms, opening and closing lightweight mechanisms, turning camera mounts, and creating other continuous-motion automation systems.

The `angle_servo_control.py` program uses the M5StickS3 together with the PbHUB, M5Stack Angle Sensor Unit, and a 360-degree servo motor to create a simple manual servo-control system. In this setup, the Angle Sensor Unit is connected to **PbHUB Port 3**, while the servo motor is connected to **PbHUB Port 5**. Rotating the Angle Unit changes the servo command from 0° to 180°, and the analog reading and calculated servo angle are printed to the terminal.

The program begins by importing the required libraries. The **PbHUBUnit** library provides the functions required to read the Angle Unit and control the servo motor through the PbHUB. The **time** library is used to control how often the sensor is read and the servo is updated.

The constants **ANGLE_PORT = 3** and **SERVO_PORT = 5** identify the PbHUB ports used by the two peripherals. **ANGLE_PORT = 3** indicates that the Angle Sensor Unit is connected to PbHUB Port 3, while **SERVO_PORT = 5** indicates that the servo motor is connected to PbHUB Port 5.

The constant **ADC_MAX = 4095** defines the maximum expected analog reading and represents the upper limit of the 12-bit analog range used to convert the Mini Angle Unit's sensor value into a servo angle.



Figure 5-5a - M5Stack Angle Sensor Unit



Figure 5-5b - M5Stack 360-Degree Servo Motor

In the **setup()** function, the I2C bus is created, initialized, and operates at 100 kHz. The PbHUB is then initialized using its I2C address (0x61). The program then prints the application title and connection details to the terminal.

The **loop()** function runs continuously to read from the Angle Sensor Unit connected to PbHUB Port 3 using:

```
raw_value = pbhub_0.analog_read(ANGLE_PORT)
```

The program then limits the analog value to the expected range from 0 to 4095. Values below 0 are set to 0, while values above 4095 are limited to 4095.

The analog reading is converted into a servo command between 0° and 180° using:

```
servo_angle = int((raw_value / ADC_MAX) * 180)
```

A low analog reading produces an angle close to 0°, while a high reading produces an angle close to 180°.

The servo motor connected to PbHUB Port 5 is controlled using:

```
pbhub_0.set_servo_angle(SERVO_PORT, 0, servo_angle)
```

The first argument selects PbHUB Port 5, the second argument selects the servo signal channel on that port, and the third argument provides the calculated servo command.

The program prints the raw analog value and calculated servo angle to the terminal:

```
print("Raw: {} | Servo Angle: {} degrees".format(raw_value, servo_angle))
```

The short 100-millisecond delay causes the Angle Unit and servo to be updated approximately 10 times per second, allowing the servo to respond smoothly as the knob is rotated.

If an error or keyboard interruption occurs, the program sends a value of 90° to the servo:

```
pbhub_0.set_servo_angle(SERVO_PORT, 0, 90)
```

For a 360-degree continuous-rotation servo, this neutral value is normally used to stop the motor. Values below 90° typically rotate the motor in one direction, while values above 90° rotate it in the opposite direction, with the distance from 90° affecting the rotational speed.

5.3 I2C based External Peripherals

The PaHUB is used to interface the M5SticksS3 with multiple external peripherals that communicate through the I2C protocol. It expands a single I2C connection into several selectable ports, allowing different I2C-based units to be connected and controlled within the same system. In the program, the **PAHUBUnit** library provides the

functions required to initialize the PaHUB and select the port connected to each peripheral. In this course, learners explore the use of the M5Stack **4-Relay Unit**, **Color Sensor RGB Unit**, **Ultrasonic Distance Sensor Unit**, **Environmental Sensor (ENV Pro) Unit**, **3-Axis Digital Accelerometer Unit**, and **RFID Unit**. These peripherals provide practical experience in controlling actuators, detecting color, measuring distance, monitoring environmental conditions, analyzing vibration and motion, and implementing RFID-based identification and access-control applications.

5.3.1 Relay

The **M5Stack 4-Relay Unit** (Figure 5-6) is an I2C-controlled output module that allows the M5StickS3 to switch up to four external electrical loads independently. Each relay channel can be turned on or off through software, making the unit suitable for controlling devices such as lamps, fans, pumps, indicators, and other low-power electrical equipment. Because the relays provide electrical isolation between the microcontroller and the connected load circuit, the unit can be used in automation and control applications that require safer switching of external devices. Its four-channel design also allows several outputs to be managed within the same system through a single I2C connection.

The **simple_relay.py** program operates a M5Stack 4-Relay Unit connected to **PaHUB Port 5**. The program switches Relay 1 through Relay 4 on in sequence. Each relay remains on for one second, switches off, and then remains off for two seconds before the next relay is activated. The relay indicator LEDs follow the relay states, so the corresponding LED turns on and off together with each relay channel.

The program begins by importing the required libraries. It follows the previously introduced template for expanded I2C bus and GPIO support. In this section, the **Relay4Unit** library is added to provide the functions required to control the 4-Relay Unit and its indicator LEDs. The **time** library is used to manage relay switching intervals.

The function **operate_relay(channel)** controls one relay channel at a time. The statement below switches on the selected relay:

```
relay4_0.set_relay_state(channel, 1)
```

The value **channel** identifies the relay number, while **1** represents the ON state. The LCD is updated to show which relay is active, and **time.sleep(1)** keeps the relay on for one second.



Figure 5-6 - M5Stack 4-Relay Unit

The relay is then switched off using:

```
relay4_0.set_relay_state(channel, 0)
```

The value 0 represents the OFF state. The program updates the LCD and waits for two seconds before proceeding to the next relay.

In the **setup()** function, The I2C bus is created, initialized and operates at 100 kHz. The following statement selects **Port 5** of the **PaHUB**:

```
pahub_5 = PAHUBUnit(i2c=i2c0, channel=5)
```

The parameter **channel=5** is important because it tells the program that the connected peripheral is located on **PaHUB Port 5**. The 4-Relay Unit must therefore be physically connected to Port 5 of the PaHUB for the program to communicate with it correctly.

The 4-Relay Unit is then initialized through the PaHUB Port 5 connection:

```
relay4_0 = Relay4Unit(pahub_5)
```

The program sets the relay module to synchronous mode so that each relay's indicator LED follows its relay state. When a relay is switched on, its corresponding LED also lights up. When the relay is switched off, the LED turns off.

The following statement ensures that all relay channels are initially switched off:

```
relay4_0.set_relay_all(0)
```

In the **loop()** function, a **for loop** controls Relay 1 through Relay 4:

```
for channel in range(1, 5):  
    operate_relay(channel)
```

The values produced by **range(1, 5)** are 1, 2, 3, and 4. Each value is passed to **operate_relay()**, causing the four relays to operate sequentially. After Relay 4 completes its cycle, the main loop repeats the sequence continuously.

5.3.2 Color Sensor

The **M5Stack Color Sensor RGB Unit** (Figure 5-7) is an external I2C sensor module used to detect and measure the red, green, blue, and clear-light components of a nearby object or surface. It can provide raw color-channel values that the M5StickS3 processes to identify dominant colors, estimate color intensity, or classify objects based on predefined ranges. The unit is useful for applications such as color sorting, object identification, quality checking, lighting analysis, and interactive displays. When connected through the PaHUB or directly to an available I2C port, the sensor can be integrated with other peripherals to support more advanced multi-sensor MCU applications.

The **color_sensor.py** program uses the M5StickS3 together with the PaHUB and M5Stack Color Sensor Unit to measure red, green, and blue color components and reproduce the detected color on the LCD. In this setup, the Color Sensor Unit is connected to **PaHUB Port 0**, allowing the M5StickS3 to communicate with the sensor through the expanded I2C interface. The program also displays numerical RGB values so that learners can compare the sensor readings with the color shown on the screen.

The program begins by importing the required libraries. In this section, the **ColorUnit** library is added to provide the functions required to control the Color Sensor RGB Unit. The **time** library is used to control how often the sensor is read.

The label variables store the LCD text objects used to display the application title and the red, green, and blue readings. The variable **last_update_time** is used to manage the sensor update interval.



Figure 5-7 - M5Stack Color Sensor RGB Unit

The function **create_rgb_color()** combines the separate red, green, and blue values into one 24-bit colour value that can be used by the LCD:

```
color = (red << 16) | (green << 8) | blue
```

The red value is shifted left by 16 bits, placing it in the upper color byte. The green value is shifted left by 8 bits, placing it in the middle byte. The blue value remains in the lowest byte. The bitwise OR operator **|** then combines the three components into the standard **0xRRGGBB** color format.

The function **display_detected_color()** calls **create_rgb_color()** and uses the resulting value to draw a filled rectangle on the lower section of the LCD. The rectangle starts at the left edge of the screen and covers the area below the RGB text readings. This allows the M5StickS3 to visually reproduce the color measured by the sensor.

In the **setup()** function, four labels are created to display the title and RGB values. Each RGB label uses a matching text color: red for the red reading, green for the green reading, and blue for the blue reading. The I2C bus is created, initialized and operates at 100 kHz. The PaHUB is then initialized on the I2C bus. The following statement selects **PaHUB Port 0**:

```
pahub_1 = PAHUBUnit(i2c=i2c0, channel=0)
```

The parameter **channel=0** identifies PaHUB Port 0. The sensor is then initialized through the PaHUB Port 0 connection:

```
color_0 = ColorUnit(pahub_0)
```

The **loop()** function runs continuously and checks whether 200 milliseconds have passed since the previous sensor update. This means the RGB values are refreshed about five times per second, which is fast enough to provide a responsive display without updating the LCD unnecessarily.

The statement below reads the processed red, green, and blue values from the Color Sensor Unit:

```
rgb_values = color_0.get_color_rgb_bytes()
```

The returned values are stored as a sequence. The program extracts them individually:

```
red = rgb_values[0]
green = rgb_values[1]
blue = rgb_values[2]
```

Each value normally ranges from 0 to 255. A value closer to 255 represents a stronger contribution from that color component, while a value closer to 0 represents a weaker contribution.

The LCD labels are updated using three-digit formatting:

```
label_red.setText("R: {:03d}".format(red))
```

The `{:03d}` format ensures that each value always occupies three digits. For example, a red value of 8 is displayed as 008, while a value of 125 is displayed as 125. The same formatting is applied to the green and blue readings.

After displaying the numerical values, the program calls **display_detected_color(red, green, blue)**. This combines the RGB components and fills the lower portion of the LCD with the corresponding color. As the sensor is moved toward differently colored objects or surfaces, both the numerical readings and the displayed color change.

5.3.3 Ultrasonic Range Sensor

The **M5Stack Ultrasonic Distance Unit** (Figure 5-8) is an I2C-based sensor module used to measure the distance between the sensor and a nearby object without physical contact. It transmits ultrasonic sound waves and calculates distance from the time taken for the reflected signal to return. The I2C interface allows the M5StickS3 to obtain distance readings through a simple digital communication connection. The measured distance can be displayed on the LCD or used to control alarms, indicators, relays, and other output devices. This unit is suitable for applications such as obstacle detection, proximity monitoring, liquid-level estimation, parking assistance, and automatic object-triggered systems.

The **measure_distance.py** program uses the M5StickS3 together with the PaHUB and M5Stack I2C Ultrasonic Distance Unit to measure the distance between the sensor and a nearby object. In this setup, the Ultrasonic Distance Unit is connected specifically to **PaHUB Port 1**. The measured distance is displayed in centimeters on the M5StickS3 LCD, together with a simple status message indicating whether an object is detected or the reading is out of range.



Figure 5-8 - M5Stack Ultrasonic Distance Unit

The program begins by importing the required libraries. In this section, the **UltrasonicI2CUnit** library is added to provide the functions required to control the Ultrasonic Distance Unit. The **time** library is used to control how often the sensor is read.

The label variables are used to display the application title, measured distance, and sensor status on the LCD. The variable **last_update_time** keeps track of the previous sensor-reading time.

In the **setup()** function, three labels are created to display the title, distance value, and measurement status. The I2C bus is created, initialized and operates at 100 kHz. The following statement selects **PaHUB Port 1**:

```
pahub_1 = PAHUBUnit(i2c=i2c0, channel=1)
```

The parameter **channel=1** identifies PaHUB Port 1. The sensor is then initialized through the PaHUB Port 1 connection:

```
ultrasonic_0 = UltrasoundI2CUnit(pahub_1)
```

This creates the ultrasonic sensor object that is later used to obtain distance measurements.

The **loop()** function runs continuously and checks whether 200 milliseconds have passed since the previous reading. This causes the sensor to be read approximately five times per second.

The distance is obtained using:

```
distance = ultrasonic_0.get_target_distance()
```

If the returned distance is greater than zero, the value is displayed on the LCD to one decimal place, and the status is updated to "Object Detected". If the value is zero or invalid, the program displays "Distance: -- cm" and changes the status to "Out of Range".

The short delay at the end of the loop reduces unnecessary processor activity while allowing the application to remain responsive.

5.3.4 Environment and Gas Sensor

The **M5Stack ENV Pro Unit** (Figure 5-9) is an environmental sensing module that measures temperature, relative humidity, atmospheric pressure, and gas resistance through an I2C interface. It allows the M5StickS3 to monitor surrounding environmental conditions and display the readings on the LCD or use them as inputs for control and alert functions. Temperature and humidity readings can support comfort and indoor-condition monitoring, while atmospheric pressure can be used to observe weather-related changes or estimate altitude trends. The gas sensor provides a resistance value that changes in response to volatile compounds in the surrounding air and can be used to observe relative changes in air quality. The unit is suitable for applications such as indoor environmental monitoring, ventilation control, weather observation, and data-logging systems.

The **env_gas_monitor.py** program uses the M5StickS3 together with the PaHUB and M5Stack ENV-Pro Unit to measure temperature, humidity, atmospheric pressure, and relative air-quality changes. In this setup, the ENV-Pro Unit is connected specifically to **PaHUB Port 2**. The measured values are displayed on the M5StickS3 LCD and updated once every second.

The program begins by importing the required libraries. In this section, the **ENVPROUnit** library is added to provide the functions required to read temperature, humidity, atmospheric pressure, and gas resistance from the ENV-Pro Unit. The **time** library is used to control how often the sensor is read.

The label variables are used to display the application title, temperature, humidity, pressure, and estimated VOC value on the LCD. The variable **last_update_time** keeps track of the previous sensor-reading time. The variables **gas_baseline**, **baseline_total**, and **baseline_samples** are used to establish an initial clean-air reference from the first 10 valid gas-resistance readings.



Figure 5-9 - M5Stack ENV Pro Unit

The **estimate_voc_ppm()** function compares the current gas-resistance reading with the clean-air baseline. A lower gas resistance generally indicates a stronger response to volatile compounds. The calculated value is limited to a range between 0 and 999 ppm. This value is an estimated relative VOC reading for observing changes in air conditions and is not a calibrated VOC measurement.

In the **setup()** function, five labels are created to display the title and environmental measurements. The I2C bus is created, initialized, and operates at 100 kHz. The following statement selects **PaHUB Port 2**:

```
pahub_2 = PAHUBUnit(i2c=i2c0, channel=2)
```

The parameter **channel=2** identifies PaHUB Port 2. The sensor is then initialized through the PaHUB Port 2 connection:

```
envpro_0 = ENVPROUnit(pahub_2)
```

This creates the ENV-Pro sensor object that is later used to obtain environmental measurements.

The **loop()** function runs continuously and checks whether 1,000 milliseconds have passed since the previous reading. This causes the sensor to be read approximately once every second.

The environmental measurements are obtained using:

```
temperature = envpro_0.get_temperature()
humidity = envpro_0.get_humidity()
pressure_hpa = envpro_0.get_pressure()
gas_ohm = envpro_0.get_gas_resistance()
```

The temperature, humidity, and atmospheric pressure values are displayed directly on the LCD. The pressure reading is displayed in hectopascals because **get_pressure()** already returns the value in hPa.

During the first 10 valid gas-resistance readings, the program calculates an average baseline value. While the baseline is being established, the LCD displays "VOC: Calibrating...". After calibration, the current gas-resistance reading is passed to the **estimate_voc_ppm()** function:

```
voc_ppm = estimate_voc_ppm(gas_ohm)
```

The estimated VOC value is then displayed in ppm. The readings are formatted to one decimal place before being shown on the LCD.

The short delay at the end of the loop reduces unnecessary processor activity while allowing the application to remain responsive.

5.3.5 Accelerometer

The M5Stack 3-Axis Digital Accelerometer Unit (Figure 5-10) uses the ADXL345 sensor to measure acceleration along the X, Y, and Z axes through an I2C interface. Compared with the built-in M5StickS3 IMU, the ADXL345 is particularly useful for machine vibration monitoring because it can be mounted directly on or close to the equipment being monitored, while the M5StickS3 remains in a more accessible or protected location. This allows the sensor to capture vibration at the actual source, such as a motor, pump, fan, gearbox, or rotating shaft. The ADXL345 can detect changes in vibration level, repeated motion, sudden shocks, and abnormal movement patterns during operation. These readings can be collected over time to establish the machine's normal vibration behavior, while significant deviations may indicate developing problems such as looseness, imbalance, misalignment, bearing wear, or mechanical deterioration. The acceleration data can also be analyzed in the time or frequency domain to identify vibration trends and dominant frequencies associated with specific machine conditions. This makes the ADXL345 suitable for condition-based monitoring applications, where maintenance decisions are based on the actual measured condition of the equipment rather than a fixed maintenance schedule, helping to support early fault detection, reduce unexpected failures, improve maintenance planning, and minimize unnecessary downtime. In comparison, the built-in M5StickS3 IMU requires no external wiring and can measure both linear acceleration and rotational movement, making it more suitable for orientation, gesture, tilt, and motion-tracking applications.

The **machine_vibration.py** program uses the M5StickS3 together with the PaHUB and M5Stack 3-Axis Digital Accelerometer (ADXL345) Unit to monitor machine vibration. In this setup, the ADXL345 Unit is connected specifically to **PaHUB Port 3**. The program reads acceleration along the X, Y, and Z axes, calculates the amount of change between consecutive readings, and displays the vibration level and machine status on the M5StickS3 LCD.

The program begins by importing the required libraries. In this section, the **AccelUnit** library is added to provide the functions required to read acceleration data from the ADXL345 Unit. The **time** library is used to control how often the sensor is read, while the **math** library is used to calculate the combined vibration level across the three axes.

The variables **previous_x**, **previous_y**, and **previous_z** store the previous acceleration readings. The variable **last_sample_time** keeps track of the previous sensor-reading time, while **first_reading** ensures that the first measurement is not compared against the initial zero values. The warning and alert thresholds define the vibration levels used to classify the machine condition.

In the **setup()** function, six labels are created to display the title, X-axis acceleration, Y-axis acceleration, Z-axis acceleration, vibration level, and machine status. The I2C bus is created, initialized, and operates at 100 kHz. The following statement selects **PaHUB Port 3**:

```
pahub_3 = PAHUBUnit(i2c=i2c0, channel=3)
```

The parameter **channel=3** identifies PaHUB Port 3. The ADXL345 Unit is then initialized through the PaHUB Port 3 connection:

```
accel_0 = AccelUnit(pahub_3, 0x53)
```



Figure 5-10 - M5Stack 3-Axis Digital Accelerometer Unit (ADXL345)

This creates an accelerometer object that is later used to obtain acceleration measurements. The value 0x53 is the I2C address of the ADXL345 sensor.

The **loop()** function runs continuously and checks whether 50 milliseconds have passed since the previous reading. This causes the accelerometer to be read approximately 20 times per second, allowing the program to detect rapid changes in machine movement.

The acceleration values are obtained using:

```
acceleration = accel_0.get_accel()
```

The returned data contains the X, Y, and Z acceleration values:

```
x = acceleration[0]
y = acceleration[1]
z = acceleration[2]
```

For the first reading, the vibration value is set to zero because there is no previous measurement available for comparison. For later readings, the program calculates the change in acceleration along each axis:

```
change_x = x - previous_x
change_y = y - previous_y
change_z = z - previous_z
```

The individual changes are then combined into one vibration value using:

```
vibration = math.sqrt(change_x ** 2 + change_y ** 2 + change_z ** 2)
```

A larger value indicates a greater change in movement between consecutive readings and therefore a higher vibration level. The current X, Y, Z, and vibration values are displayed on the LCD to two decimal places.

The program then compares the vibration value against the configured thresholds. If the value is below **WARNING_THRESHOLD**, the status is displayed as "Normal" in green. If it reaches the warning threshold, the status changes to "Warning" in yellow. If it reaches or exceeds **ALERT_THRESHOLD**, the status changes to "High Vibration" in red.

The current acceleration readings are stored as the previous readings before the next cycle begins. The short delay at the end of the loop reduces unnecessary processor activity while allowing the program to remain responsive. The threshold values are demonstration values and should be adjusted after observing the normal vibration behavior of the actual machine.

5.3.6 RFID

The M5Stack RFID 2 Unit (Figure 5-11) is a contactless identification module that operates at 13.56 MHz and communicates with the M5StickS3 through an I2C interface. It can detect compatible RFID cards, key fobs, and tags placed near the reader and retrieve their unique identification numbers for processing by the program. The detected identification data can be displayed on the LCD, stored for record keeping, or compared with a list of authorized IDs to determine whether access should be granted or rejected. The unit can also be integrated with other output devices, such as relays, buzzers, indicator lights, or electronic locks, to create a complete automated response after a valid or invalid tag is detected. This makes the RFID 2 Unit suitable for applications such as access control, attendance recording, equipment checkout, user authentication, visitor registration, asset tracking, cashless identification, and automated locker or door-control systems.

The `rfid_security_access.py` program uses the M5StickS3 together with the PaHUB, M5Stack RFID 2 Unit, and 4-Relay Unit to create a simple access-control system. In this setup, the RFID 2 Unit is connected to **PaHUB Port 4**, while the 4-Relay Unit is connected to **PaHUB Port 5**. Button A is used to switch between enrolment mode and security mode. A valid RFID tag activates the lock actuator for five seconds before the system returns to the locked state.

The program begins by importing the required libraries. In this section, the **RFIDUnit** library is added to provide the functions required to detect an RFID tag and read its UID, while the **Relay4Unit** library provides the functions required to control the 4-Relay Unit. The **time** library is used to measure the five-second relay activation period and provide short delays during program operation.



Figure 5-11 - M5Stack RFID 2 Unit

The label variables are used to display the application title, operating mode, access status, detected UID, and lock-actuator state on the LCD. The variable **enrolment_mode** identifies whether the system is in enrolment mode or security mode. The variable **authorised_uid** stores the UID of the enrolled RFID tag. The variables **relay_active** and **relay_start_time** keep track of whether the relay is active and when it was switched on. The variable **card_latched** prevents the same tag from being processed repeatedly while it remains near the reader.

The **uid_to_text()** function converts the detected RFID UID into readable text. The UID may be returned as an integer, string, or sequence of byte values. The function converts these different formats into a consistent hexadecimal string so that the UID can be displayed and compared with the enrolled UID.

The **activate_relay()** function switches on the relay channel assigned to the lock actuator:

```
relay4_0.set_relay_state(RELAY_UNLOCK, 1)
```

It records the activation time and updates the LCD to show that the lock actuator is off, which represents the unlocked state. The **deactivate_relay()** function switches the relay off:

```
relay4_0.set_relay_state(RELAY_UNLOCK, 0)
```

It updates the LCD to show that the lock actuator is on, which represents the locked state.

The **change_mode()** function switches between enrolment mode and security mode whenever Button A is pressed. In enrolment mode, the LCD displays "Mode: ENROLMENT" and prompts the user to present an RFID tag. In security mode, the LCD displays "Mode: SECURITY" and waits for an authorised tag. The relay is also deactivated whenever the operating mode changes.

The **process_rfid_tag()** function checks whether a new RFID tag is present. The tag UID is obtained using:

```
uid = rfid_0.read_card_uid()
```

The UID is then converted into readable text:

```
uid_text = uid_to_text(uid)
```

In enrolment mode, the detected UID is stored as the authorized UID:

```
authorised_uid = uid_text
```

The LCD then displays "Tag enrolled". Only one UID is stored, so enrolling another tag replaces the previously authorized tag.

In security mode, the detected UID is compared with the enrolled UID. If no tag has been enrolled, the LCD displays "No enrolled tag". If the detected UID matches the authorised UID, the program displays "Access granted" and calls **activate_relay()** to unlock the actuator. If the UID does not match, the program displays "Access denied" and the lock remains active.

In the **setup()** function, five labels are created to display the title, operating mode, status, UID, and lock-actuator state. The I2C bus is created, initialized, and operates at 100 kHz. The following statement selects **PaHUB Port 4** for the RFID 2 Unit:

```
pahub_4 = PAHUBUnit(i2c=i2c0, channel=4)
```

The parameter **channel=4** identifies PaHUB Port 4. The RFID 2 Unit is then initialized through the PaHUB Port 4 connection:

```
rfid_0 = RFIDUnit(pahub_4)
```

This creates the RFID reader object that is later used to detect tags and read their UIDs.

The following statement selects **PaHUB Port 5** for the 4-Relay Unit:

```
pahub_5 = PAHUBUnit(i2c=i2c0, channel=5)
```

The parameter **channel=5** identifies PaHUB Port 5. The relay unit is then initialized through the PaHUB Port 5 connection:

```
relay4_0 = Relay4Unit(pahub_5)
```

The relay module is placed in synchronous mode, and all relay channels are initially switched off:

```
relay4_0.set_mode(Relay4Unit.SYNC_MODE)
relay4_0.set_relay_all(0)
```

The **loop()** function runs continuously and begins by calling `M5.update()` so that Button A events can be detected. When Button A is pressed, the following statement calls the mode-change function:

```
if BtnA.wasClicked():
    change_mode()
```

The program then calls **process_rfid_tag()** to detect and process any RFID tag presented to the reader. When the tag is removed, **card_latched** is reset so that another tag can be read.

When a valid tag activates the relay, the program checks whether 5,000 milliseconds have passed:

```
if time.ticks_diff(time.ticks_ms(), relay_start_time) >= RELAY_ON_TIME:
```

After five seconds, **deactivate_relay()** is called to switch off the relay and return the lock actuator to the locked state. The LCD then displays "Present authorised tag" and waits for the next valid RFID tag.

The short delay at the end of the loop reduces unnecessary processor activity while allowing the access-control system to remain responsive. The enrolled RFID UID is stored only in memory, so it is cleared when the M5StickS3 is restarted or powered off.

6. Integrated Examples

6.1 SecureHome RFID Motion Alarm System

(a) System Overview

The **SecureHome RFID Motion Alarm System**, shown in Figure 6-1, uses the M5StickS3 as the central controller for a simple home security application. The Dual-Button Unit is connected to PbHUB Port 0, with the red button used to arm the system and the blue button used to enter RFID enrolment mode. A PIR Motion Sensor Unit connected to PbHUB Port 1 detects human movement, while an RFID 2 Unit connected to PaHUB Port 4 is used to enroll and authenticate authorized tags. A 4-Relay Unit connected to PaHUB Port 5 controls an external high-power alarm through Relay 1, and the M5StickS3 embedded buzzer provides an additional local audible warning.

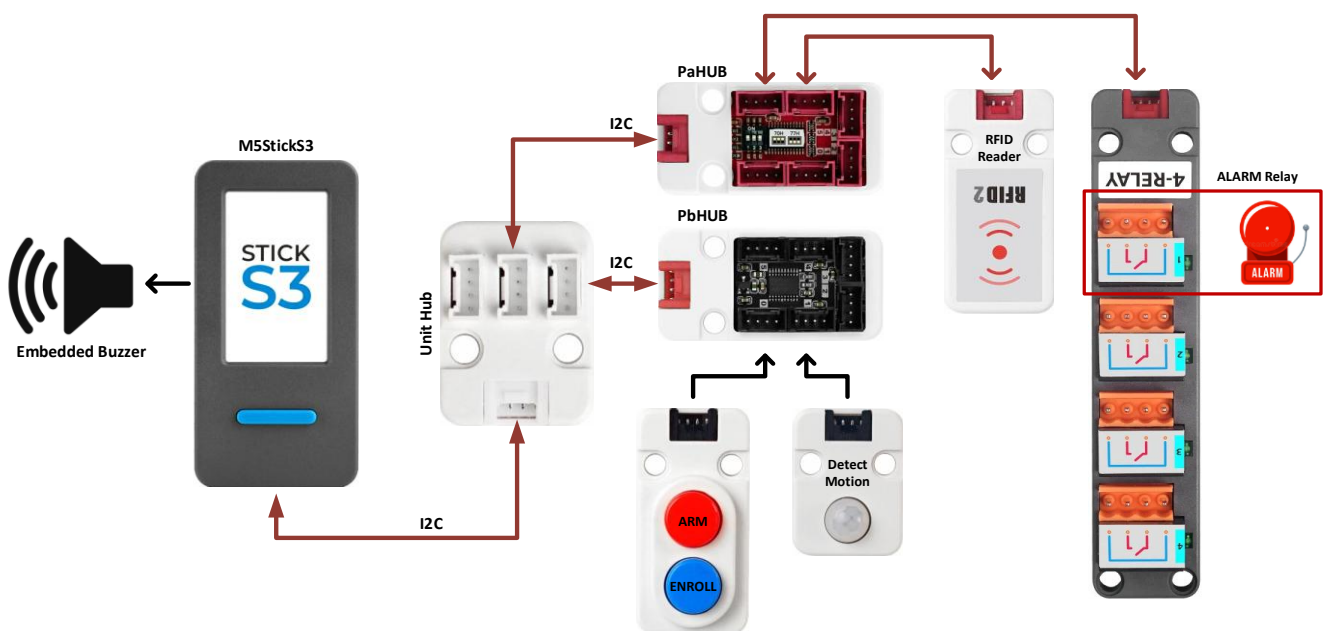


Figure 6-1 - SecureHome RFID Motion Alarm System

The system begins in the disarmed mode. It supports the enrolment of up to two RFID tags, and new tags can be enrolled only while the system is disarmed. Pressing the blue button places the system into enrolment mode, after which the next RFID tag presented to the reader is registered as an authorized tag. The display confirms whether the tag has been enrolled successfully, is already registered, or cannot be added because the maximum of two tags has been reached. After the enrolment attempt, the system automatically returns to the disarmed mode.

The security system cannot be armed unless at least one RFID tag has been enrolled. While the system is disarmed, pressing the red button changes the operating mode to armed and begins monitoring the PIR sensor for motion. The LCD displays the current system mode, operating status, countdown information, number of enrolled tags, and alarm state. The terminal also reports important events such as successful enrolment, arming, motion detection, invalid RFID tags, alarm activation, and disarming.

When the system is armed and the PIR sensor detects movement, the system enters countdown mode and starts a 10-second entry delay. The remaining time is updated once per second on the LCD and printed to the terminal. During this countdown, the user can present an authorized RFID tag to cancel the countdown and return the system to the disarmed mode. Presenting an invalid tag does not stop the countdown and results in an access-denied message.

If the 10-second countdown expires without successful RFID authentication, the system enters alarm mode. Relay 1 is activated to switch on the external high-power alarm, and the M5StickS3 embedded buzzer produces a repeating 2,000 Hz warning tone. The buzzer sounds for 400 milliseconds at approximately 600-millisecond intervals while the alarm remains active. The LCD displays an intruder warning and prompts the user to present an authorized tag. Both the relay-controlled alarm and embedded buzzer remain active until a valid enrolled RFID tag is detected.

An authorized RFID tag can disarm the system while it is armed, during the countdown, or after the alarm has been activated. When a valid tag is presented, the relay is switched off, the buzzer is stopped, the alarm state is cleared, and the system returns to disarmed mode. The RFID card-latch logic prevents the same tag from being processed repeatedly while it remains near the reader, and the button-state logic ensures that each button press is recognized only once. If a runtime error or keyboard interruption occurs, the program switches off all relay outputs and stops the buzzer to leave the system in a safer state.

(b) Software Implementation

The **securehome.py** program uses the **M5StickS3** together with the **PaHUB**, **PbHUB**, **Dual-Button Unit**, **PIR Motion Sensor Unit**, **RFID 2 Unit**, **4-Relay Unit**, and **embedded buzzer** to create a simple home security system.

The program begins by importing the required libraries. The **PAHUBUnit** and **PBHUBUnit** libraries provide support for the two hub modules. The **RFIDUnit** library provides the functions required to detect RFID tags and read their UIDs, while the **Relay4Unit** library provides the functions required to control the external alarm through the 4-Relay Unit. The **time** library is used to manage the alarm countdown, buzzer timing, button delays, and other time-based operations.

The hardware connection constants identify the ports and signal pins used by the system. The Dual-Button Unit uses PbHUB Port 0, with signal pin 0 assigned to the red button and signal pin 1 assigned to the blue button. The

PIR sensor uses PbHUB Port 1 and signal pin 1. The RFID 2 Unit and 4-Relay Unit are assigned to PaHUB Ports 4 and 5 respectively.

```
DUAL_BUTTON_PORT = 0
RED_BUTTON_PIN = 0
BLUE_BUTTON_PIN = 1
PIR_PORT = 1
PIR_SIGNAL_PIN = 1
RFID_PAHUB_PORT = 4
RELAY_PAHUB_PORT = 5
```

The constants **MAX_TAGS**, **COUNTDOWN_TIME**, **BUZZER_FREQUENCY**, and **BUZZER_INTERVAL** define the main operating settings. A maximum of two RFID tags can be enrolled. The alarm countdown is set to 10,000 milliseconds, which is equal to 10 seconds. The embedded buzzer operates at 2,000 Hz and produces a repeating tone at the specified interval.

```
MAX_TAGS = 2
COUNTDOWN_TIME = 10000
BUZZER_FREQUENCY = 2000
BUZZER_INTERVAL = 600
```

The system uses five operating modes: **disarmed**, **enrolment**, **armed**, **countdown**, and **alarm**. The variable **system_mode** stores the current operating state and initially places the system in disarmed mode.

```
MODE_DISARMED = 0
MODE_ENROLMENT = 1
MODE_ARMED = 2
MODE_COUNTDOWN = 3
MODE_ALARM = 4
system_mode = MODE_DISARMED
```

The list **authorised_uids** stores the enrolled RFID tag identifiers. The variables **countdown_start_time** and **last_countdown_second** are used to manage and display the 10-second entry countdown. The variables **alarm_active** and **last_buzzer_time** control the external alarm and repeating buzzer. The previous button-state variables detect new button presses, while **card_latched** prevents the same RFID tag from being processed repeatedly while it remains near the reader.

The **uid_to_text()** function converts an RFID UID into a consistent readable string. Depending on the RFID driver, the UID may be returned as an integer, string, byte array, or list. The function converts the UID into hexadecimal text so that it can be displayed, stored, and compared reliably.

The **update_tag_display()** function updates the LCD with the number of enrolled RFID tags:

```
label_tags.setText("Tags enrolled: {}/{}".format(len(authorised_uids), MAX_TAGS))
```

The **activate_alarm()** function switches on the selected relay channel and starts the alarm condition:

```
relay4_0.set_relay_state(ALARM_RELAY, 1)
```

It sets **alarm_active** to True, updates the LCD to show "Mode: ALARM" and "INTRUDER ALERT", and prints an alarm message to the terminal. The embedded buzzer is produced separately by the **process_buzzer()** function while the alarm remains active.

The **deactivate_alarm()** function switches off the relay and stops the embedded buzzer:

```
relay4_0.set_relay_state(ALARM_RELAY, 0)
Speaker.stop()
```

It also changes **alarm_active** to False and updates the LCD to show that the alarm is off.

The **disarm_system()** function calls **deactivate_alarm()** and returns the system to disarmed mode. It resets the countdown display, updates the LCD, and provides instructions showing that the red button is used to arm the system and the blue button is used to enrol a tag.

The **arm_system()** function first checks whether at least one RFID tag has been enrolled. If no authorised tag is available, the system refuses to arm and displays "Enrol RFID tag first". If a tag has been enrolled, the system changes to armed mode and begins monitoring the PIR sensor for movement.

The **start_enrolment()** function allows a new RFID tag to be registered. Enrolment can begin only while the system is disarmed. If two tags have already been enrolled, the program displays "Maximum 2 tags reached" and does not accept another tag. Otherwise, the system enters enrolment mode and waits for an RFID tag to be presented.

The **start_countdown()** function is called when the PIR sensor detects movement while the system is armed. It changes the system mode to countdown mode, records the starting time, resets the countdown display, and informs the user that motion has been detected.

The **is_authorised()** function checks whether a detected UID is stored in the list of authorized tags:

```
def is_authorised(uid_text):  
    return uid_text in authorised_uids
```

The **process_rfid_tag()** function detects and processes RFID tags. It first checks whether the current card has already been handled and whether a new card is present. The UID is then read and converted into text:

```
uid = rfid_0.read_card_uid()  
uid_text = uid_to_text(uid)
```

In enrolment mode, the tag is added to **authorised_uids** if it is not already enrolled and the maximum of two tags has not been reached. The tag count is then updated, and the system returns to disarmed mode.

In armed, countdown, or alarm mode, the detected UID is compared with the enrolled tags. If the UID is authorized, the system is disarmed. This cancels the countdown or switches off both the relay alarm and embedded buzzer. If the UID is not authorized, the LCD displays "Invalid RFID tag" and the system remains in its current security state.

The **process_buttons()** function reads both switches on the Dual-Button Unit:

```
red_state = pbhub_0.digital_read(DUAL_BUTTON_PORT, RED_BUTTON_PIN)  
blue_state = pbhub_0.digital_read(DUAL_BUTTON_PORT, BLUE_BUTTON_PIN)
```

The buttons use **active-low** logic, so a value of 0 means that a button is pressed. The previous states are compared with the current states so that each press is processed only once. Pressing the red button arms the system when it is disarmed. Pressing the blue button starts RFID enrolment when the system is disarmed.

The **process_motion_sensor()** function reads the PIR Motion Sensor Unit from PbHUB Port 1, signal pin 1:

```
motion_state = pbhub_0.digital_read(PIR_PORT, PIR_SIGNAL_PIN)
```

If the system is armed and the PIR sensor returns 1, the program calls **start_countdown()**.

The **process_countdown()** function manages the 10-second delay before the alarm is activated. The function calculates the elapsed time since the countdown started and determines how many seconds remain:

```
elapsed = time.ticks_diff(time.ticks_ms(), countdown_start_time)
remaining_ms = COUNTDOWN_TIME - elapsed
remaining_seconds = (remaining_ms + 999) // 1000
```

The remaining time is displayed only when the second value changes. When the elapsed time reaches **10 seconds**, the system changes to alarm mode and activates the alarm:

```
if elapsed >= COUNTDOWN_TIME:
    system_mode = MODE_ALARM
    activate_alarm()
```

The **process_buzzer()** function produces the repeating alarm tone. It runs only when **alarm_active** is True. At each buzzer interval, the following statement generates a 2,000 Hz tone for 400 milliseconds:

```
Speaker.tone(BUZZER_FREQUENCY, 400)
```

The **reset_card_latch()** function clears **card_latched** after the RFID tag has been removed. This allows the same tag or another tag to be read during the next presentation.

In the **setup()** function, the M5StickS3 and LCD are initialized. Six labels are created to display the product title, system mode, operating status, countdown information, number of enrolled tags, and alarm state.

The shared I2C bus is created using GPIO 10 as SCL and GPIO 9 as SDA, operating at 100 kHz:

```
i2c0 = I2C(0, scl=Pin(10), sda=Pin(9), freq=100000)
```

The PbHUB is initialized using its I2C address:

```
pbrush_0 = PBHUBUnit(i2c0, 0x61)
```

The RFID 2 Unit is connected through **PaHUB Port 4**:

```
pahub_4 = PAHUBUnit(i2c=i2c0, channel=RFID_PAHUB_PORT)
rfid_0 = RFIDUnit(pahub_4)
```

The 4-Relay Unit is connected through **PaHUB Port 5**:

```
pahub_5 = PAHUBUnit(i2c=i2c0, channel=RELAY_PAHUB_PORT)
relay4_0 = Relay4Unit(pahub_5)
```

The relay module is placed in synchronous mode, and all relays are initially switched off:

```
relay4_0.set_mode(Relay4Unit.SYNC_MODE)
relay4_0.set_relay_all(0)
```

The embedded buzzer is then initialized, its volume is set, and it is stopped to ensure that no sound is produced during startup:

```
Speaker.begin()
Speaker.setVolume(180)
Speaker.stop()
```

The **loop()** function runs continuously and performs all system operations in sequence:

```
process_buttons()
process_motion_sensor()
process_rfid_tag()
process_countdown()
process_buzzer()
reset_card_latch()
```

This allows the program to respond to button presses, detect movement, process RFID tags, update the countdown, operate the buzzer, and prepare the RFID reader for the next tag. The 50-millisecond delay reduces unnecessary processor activity while keeping the security system responsive.

The final section starts the program by calling **setup()** once and then repeatedly calling **loop()**. If an error or keyboard interruption occurs, all relay outputs are switched off, and the embedded buzzer is stopped before the error message is displayed. This helps leave the alarm outputs in a safe state.

The enrolled RFID tags are stored only in memory. Restarting or powering off the M5StickS3 clears the authorized-tag list, so the tags must be enrolled again after each restart.

6.2 Smart Bathroom

(a) System Overview

The **Smart Bathroom System**, shown in Figure 6-2, uses the M5StickS3 as the central controller to monitor environmental conditions and operate the bathroom light, exhaust fan, display, and warning buzzer automatically. The M5Stack Environment and Gas Sensor (ENV-Pro) Unit measures ambient temperature, relative humidity, atmospheric pressure, and gas resistance, while the Light Sensor Unit measures the surrounding illumination level. A PIR Motion Sensor Unit detects human movement, and the program treats the bathroom as occupied for 30 seconds after the most recent motion detection. Motion events are also counted over one-minute intervals to provide a simple indication of activity within the room.

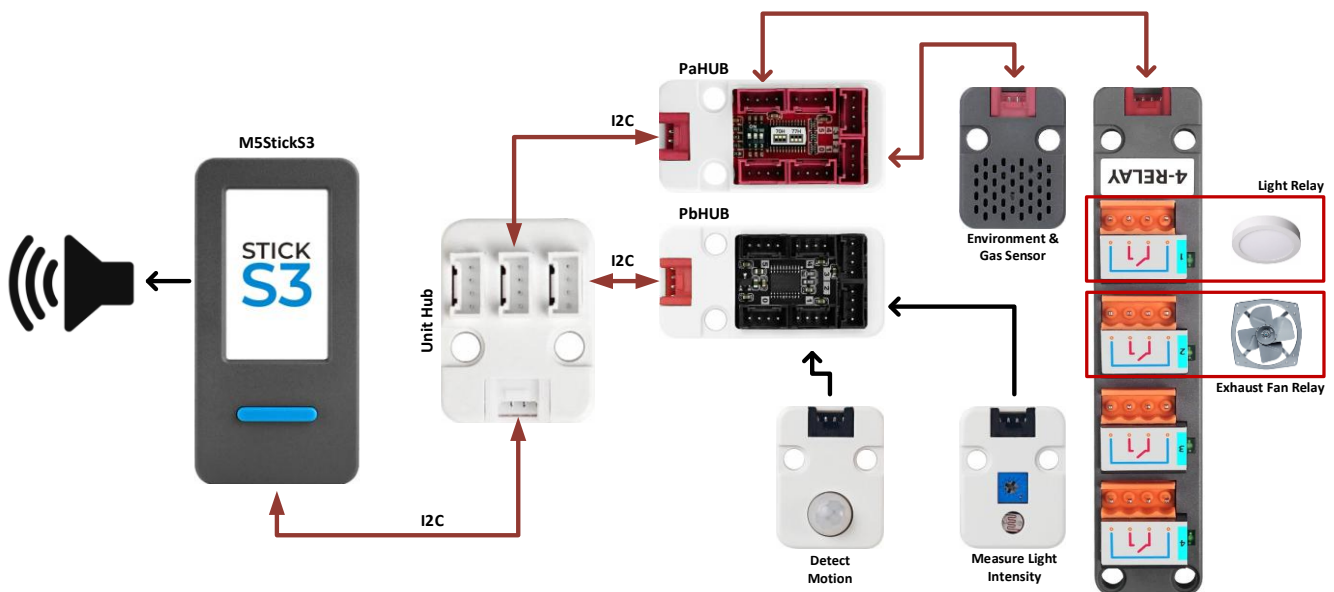


Figure 6-2 - Smart Bathroom System

The controller operates a 4-Relay Unit connected to the bathroom light and exhaust fan. While the bathroom is occupied, the program evaluates the light intensity once every second. If the measured value falls below the configured light-on threshold, the controller switches on the bathroom light. If sufficient natural light later becomes available and the reading rises above the separate light-off threshold, the controller switches the light off. The use of different on and off thresholds helps prevent rapid relay switching when the measured light level fluctuates near a single threshold.

When the bathroom changes from occupied to vacant, the system does not switch off an active bathroom light immediately. Instead, it starts a 10-second delay. If motion is detected again before the countdown expires, the countdown is cancelled and normal light-level evaluation resumes. If the bathroom remains vacant for the full 10 seconds, the controller switches off the bathroom light.

While the bathroom is occupied, the controller monitors temperature and relative humidity. If either value reaches its configured high threshold, the exhaust fan is activated to improve air circulation, reduce humidity, and help lower the room temperature. Separate recovery thresholds are used to switch off the fan only after both temperature and humidity have returned to acceptable levels. This hysteresis reduces unnecessary relay switching when readings fluctuate close to the activation thresholds. When the bathroom is vacant, the fan is switched off unless an air-quality alert remains active.

Air-quality monitoring operates regardless of whether the bathroom is occupied. During startup, the system averages the first 20 valid gas-resistance readings to establish a normal baseline. If the current gas resistance falls below the configured percentage of this baseline, the system interprets the change as abnormal air quality and immediately activates the exhaust fan. At the same time, the M5StickS3 embedded buzzer produces a warning tone at five-second intervals, and the display shows a poor-air-quality warning. The fan, buzzer, and warning message are cleared automatically only after the gas resistance recovers above the configured recovery level.

The ENV-Pro gas sensor provides a general gas-resistance response rather than a selective, calibrated measurement of ammonia or individual VOC concentrations. Therefore, the program uses the reading as a relative air-quality indicator and calculates a relative VOC response for observation. The gas baseline and alert thresholds should be established and adjusted through testing under normal bathroom conditions. A dedicated ammonia sensor would be required where reliable ammonia concentration measurement is necessary.

(b) Software Implementation

The **smart_bathroom.py** program uses the **M5StickS3** together with the **PaHUB**, **PbHUB**, **Environment and Gas Unit (ENV-Pro)**, **PIR Motion Sensor Unit**, **Light Sensor Unit**, **4-Relay Unit**, and **embedded buzzer** to create an automated bathroom monitoring and control system.

The program begins by importing the required libraries. The **PAHUBUnit** and **PBHUBUnit** libraries provide support for connected hub modules. The **ENVPROUnit** library provides the functions required to read temperature, humidity, pressure, and gas resistance from the ENV-Pro Unit, while the **Relay4Unit** library provides the functions required to control the bathroom light and exhaust fan. The **time** library is used to manage sensor update intervals, occupancy timing, the delayed light switch-off, motion-activity measurement, and buzzer timing.

The variables **i2c0**, **pahub_2**, **pahub_5**, and **pbhub_0** store the shared I2C bus and the hub connections. The variables **envpro_0** and **relay4_0** store the ENV-Pro and 4-Relay device objects. The label variables are used to display the system title, occupancy status, environmental readings, light intensity, air-quality condition, and output states on the M5StickS3 LCD.

The hardware constants identify where each peripheral is connected:

```
ENVPRO_PAHUB_PORT = 2
RELAY_PAHUB_PORT = 5
PIR_PORT = 1
PIR_SIGNAL_PIN = 1
LIGHT_SENSOR_PORT = 2
```

ENVPRO_PAHUB_PORT = 2 indicates that the ENV-Pro Unit is connected to PaHUB Port 2, while **RELAY_PAHUB_PORT = 5** identifies PaHUB Port 5 for the 4-Relay Unit. The PIR sensor uses PbHUB Port 1 and signal pin 1, while the Light Sensor Unit uses PbHUB Port 2.

The relay assignments are defined using:

```
LIGHT_RELAY = 1
FAN_RELAY = 2
```

Relay 1 controls the bathroom light, while Relay 2 controls the exhaust fan.

The light-sensor settings define the analog range and estimated lux range:

```
ADC_MAX = 4095
MAX_LUX = 1000.0
```

The separate light thresholds are:

```
LIGHT_ON_THRESHOLD_LUX = 250.0
LIGHT_OFF_THRESHOLD_LUX = 350.0
```

When the room is occupied, the bathroom light is switched on if the measured light intensity falls below 250 lux. If the light level later rises to 350 lux or above, the bathroom light is switched off. Using separate on and off thresholds prevents the relay from rapidly switching when the light level fluctuates near a single value.

The environmental thresholds define when the exhaust fan should be activated and when it may be switched off:

```
EMPERATURE_HIGH = 30.0
TEMPERATURE_RECOVERY = 27.0
HUMIDITY_HIGH = 75.0
HUMIDITY_RECOVERY = 68.0
```

While the bathroom is occupied, the fan is switched on if the temperature reaches 30°C or the humidity reaches 75%. It is switched off only after the temperature falls to 27°C or below and the humidity falls to 68% or below. These separate activation and recovery values provide hysteresis and reduce unnecessary relay switching.

The gas settings define how the program establishes a normal air-quality reference:

```
GAS_BASELINE_SAMPLES = 20
GAS_ALERT_RATIO = 0.70
GAS_RECOVERY_RATIO = 0.80
```

The first 20 valid gas-resistance readings are averaged to create the baseline. If the current gas resistance falls to 70% or less of this baseline, the program treats the change as abnormal air quality. The alert is cleared only after the gas resistance recovers to at least 80% of the baseline.

The timing settings control how frequently the system performs its operations:

```
SENSOR_UPDATE_INTERVAL = 1000
LIGHT_OFF_DELAY = 10000
OCCUPANCY_HOLD_TIME = 30000
ACTIVITY_WINDOW = 60000
```

Environmental sensors are read once every second. The bathroom light remains on for 10 seconds after the room becomes vacant. Since a PIR sensor detects movement rather than continuous presence, the bathroom remains classified as occupied for 30 seconds after the most recent motion detection. Motion events are counted over a 60-second window to provide a simple activity value in events per minute.

The **convert_to_lux()** function converts the Light Sensor Unit's analog value into an estimated lux reading:

```
return (raw_value / ADC_MAX) * MAX_LUX
```

The function first limits the reading to the expected range from 0 to 4095, then maps it linearly to an estimated range from 0 to 1,000 lux.

The **estimate_relative_voc()** function compares the current gas resistance with the established baseline:

```
ratio = gas_baseline / gas_resistance  
relative_voc = (ratio - 1.0) * 100.0
```

A lower gas-resistance reading produces a higher relative VOC response. The result is limited to a range from 0 to 999. This is a relative indicator for observing changes in air conditions and is not a calibrated VOC concentration.

The **set_light()** function controls the bathroom light through Relay 1:

```
relay4_0.set_relay_state(LIGHT_RELAY, 1)
```

A value of 1 switches the relay on, while a value of 0 switches it off. The function checks the existing light_on state before changing the relay so that repeated commands are avoided.

The **set_fan()** function works in the same way for Relay 2. It switches the exhaust fan on or off and updates the fan_on variable and terminal output.

The **start_light_off_countdown()** function starts the delayed light switch-off when the bathroom changes from occupied to vacant:

```
light_off_countdown = True  
light_off_start_time = time.time()
```

The start time is recorded so that the program can determine when 10 seconds have passed.

The **process_occupancy()** function reads the PIR sensor from PbHUB Port 1, signal pin 1:

```
motion_state = pbhub_0.digital_read(PIR_PORT, PIR_SIGNAL_PIN)
```

When motion is detected, the current time is stored in **last_motion_time**. A new motion event is counted only when the PIR signal changes from 0 to 1. If the bathroom was previously vacant, it is changed to occupied. Any pending light-off countdown is also cancelled.

If no motion is detected for 30 seconds, the bathroom changes to vacant. If the bathroom light is currently on, the program starts the 10-second delayed switch-off.

The **process_light_control()** function evaluates the light intensity only while the bathroom is occupied. If the bathroom is dark and the light is off, it switches the light on. If sufficient natural light becomes available while the room remains occupied, it switches the light off. This function is called whenever a new light-sensor reading is obtained, so changing light conditions are evaluated periodically rather than only when occupancy first changes.

The **process_light_off_countdown()** function manages the delayed switch-off after vacancy. If the bathroom becomes occupied again before the 10-second delay expires, the countdown is cancelled. If the room remains vacant for the full delay, the bathroom light is switched off.

The **process_motion_activity()** function calculates a simple activity value. It counts new PIR motion events for 60 seconds, stores the total as **average_motion_activity**, prints the number of events per minute, resets the counter, and starts a new activity window.

The **process_gas_baseline()** function collects the first 20 valid gas-resistance readings:

```
gas_baseline_total += gas_resistance
gas_baseline_count += 1
```

After 20 readings, it calculates the average:

```
gas_baseline = (gas_baseline_total / GAS_BASELINE_SAMPLES)
```

This average becomes the normal reference used for later air-quality comparisons.

The **process_air_quality()** function compares the current gas resistance with the baseline. If the ratio falls to 0.70 or below, **gas_alert** becomes True. If the ratio later recovers to 0.80 or above, the alert is cleared and the buzzer is stopped. The separate alert and recovery ratios help prevent rapid switching near one threshold.

The **process_fan_control()** function gives the air-quality alert the highest priority. If **gas_alert** is active, the fan is switched on regardless of whether the bathroom is occupied. When no gas alert exists, temperature and humidity control applies only while the bathroom is occupied. If the bathroom becomes vacant, the fan is switched off unless an air-quality alert is still active.

The **process_buzzer()** function produces the air-quality warning tone. When **gas_alert** is active, the buzzer sounds at 2,000 Hz for 500 milliseconds every five seconds:

```
Speaker.tone(BUZZER_FREQUENCY, 500)
```

The interval is controlled using **last_buzzer_time**, allowing the rest of the program to continue running while the warning repeats.

The **read_sensors()** function obtains new sensor readings once every second. The ENV-Pro Unit provides temperature, humidity, pressure, and gas resistance:

```
temperature = envpro_0.get_temperature()
humidity = envpro_0.get_humidity()
pressure_hpa = envpro_0.get_pressure()
gas_resistance = envpro_0.get_gas_resistance()
```

The Light Sensor Unit is read from PbHUB Port 2:

```
light_raw = (ADC_MAX - pbhub_0.analog_read(LIGHT_SENSOR_PORT))
```

The subtraction reverses the sensor relationship so that lower values represent darker conditions and higher values represent brighter conditions. The value is then converted into estimated lux.

After reading the sensors, the program establishes or updates the gas baseline, evaluates air quality, checks the light level, controls the exhaust fan, calculates the relative VOC response, and prints the latest readings to the terminal.

The **update_display()** function updates the LCD with the current bathroom status. It shows whether the room is occupied or vacant, the activity count, temperature, humidity, light intensity, air-quality condition, and the

current states of the bathroom light and exhaust fan. The air-quality label changes to red during an alert and green when conditions are normal.

In the **setup()** function, the M5StickS3 and LCD are initialized. Six labels are created for the title and system information. The shared I2C bus is configured using GPIO 10 as SCL and GPIO 9 as SDA, operating at 100 kHz:

```
i2c0 = I2C(0, scl=Pin(10), sda=Pin(9), freq=100000)
```

The PbHUB is initialized using its default I2C address:

```
pbhub_0 = PBHUBUnit(i2c0, 0x61)
```

The ENV-Pro Unit is initialized through PaHUB Port 2:

```
pahub_2 = PAHUBUnit(i2c=i2c0, channel=ENVPRO_PAHUB_PORT)
envpro_0 = ENVPROUnit(pahub_2)
```

The 4-Relay Unit is initialized through PaHUB Port 5:

```
pahub_5 = PAHUBUnit(i2c=i2c0, channel=RELAY_PAHUB_PORT)
relay4_0 = Relay4Unit(pahub_5)
```

The relay module is placed in synchronous mode, and all relay outputs are initially switched off:

```
relay4_0.set_mode(Relay4Unit.SYNC_MODE)
relay4_0.set_relay_all(0)
```

The embedded buzzer is then initialized, its volume is set, and it is stopped to ensure that no warning tone is produced during startup.

The **loop()** function runs continuously and performs the main operations in sequence:

```
process_occupancy()
process_motion_activity()
read_sensors()
process_light_off_countdown()
process_buzzer()
update_display()
```

This allows the program to monitor occupancy, calculate motion activity, read environmental conditions, control the light and fan, manage the delayed light-off function, produce air-quality warnings, and update the LCD. The 50-millisecond delay reduces unnecessary processor activity while keeping the system responsive.

The final section starts the program by calling **setup()** once and repeatedly calling **loop()**. If an error or keyboard interruption occurs, all relay outputs are switched off and the embedded buzzer is stopped before the error message is displayed. This helps leave the bathroom light, exhaust fan, and warning output in a safe state.

7. Edge Artificial Intelligence

7.1 How Edge-AI Works

Figure 7-1 shows the basic workflow of an Edge AI system. Instead of sending all raw data to the cloud for processing, Edge AI allows data to be collected, analyzed, and acted on directly at the device level. This makes the system faster, more responsive, and less dependent on continuous internet connectivity.

The first stage is **Data In**. Data is collected from devices and sensors such as cameras, temperature sensors, wireless sensors, robotic systems, vehicles, or other smart equipment. These devices capture information from the real-world environment and prepare it for local processing.

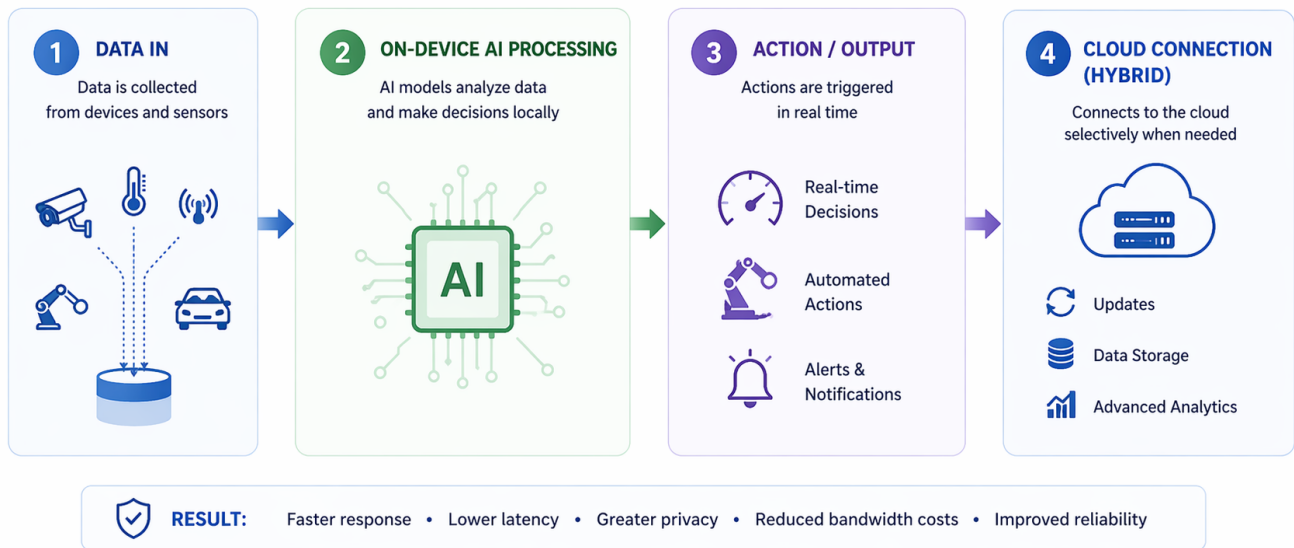


Figure 7-1 – How Edge-AI Works

The second stage is **On-Device AI Processing**. The AI model runs directly on the edge device, such as a microcontroller, embedded system, camera, or industrial controller. The device analyses the incoming data, extracts useful features, recognizes patterns, and makes decisions locally. For example, it may detect an object, recognize abnormal temperature readings, identify machine faults, or classify a sensor pattern.

The third stage is **Action or Output**. Once the AI model makes a decision, the device can trigger an immediate response. This may include real-time decisions, automated actions, alerts, or notifications. For example, a system may switch on a fan, activate an alarm, stop a machine, update a display, or send a warning message when an abnormal condition is detected.

The fourth stage is **Cloud Connection**, which is **optional** and **selective**. Edge AI systems can still connect to the cloud when needed, but they do not need to send all raw data continuously. The cloud may be used for software updates, long-term data storage, dashboard reporting, advanced analytics, or model improvement. This is known as a hybrid approach, where immediate decisions are handled locally while the cloud supports larger-scale analysis and management.

Overall, Figure 7-1 highlights the key benefits of Edge AI: faster response, lower latency, greater privacy, reduced bandwidth cost, and improved reliability. By processing data close to where it is collected, Edge AI enables smart devices to sense, understand, decide, and act locally in real time.

On the software side, Edge AI is supported by lightweight AI frameworks that allow trained models to run efficiently on local devices. Frameworks such as **TensorFlow Lite**, **ONNX Runtime**, and **OpenVINO** help developers deploy AI models across different types of hardware, from smartphones and embedded boards to cameras and industrial machines. These tools are designed to make AI inference faster and more efficient on devices with limited processing power, memory, and energy. In addition, real-time operating systems and middleware help manage device tasks reliably, so that AI-powered features can run consistently even when the system is handling sensor readings, display updates, communication, and control actions at the same time.

Another approach that is highly suitable for microcontrollers is **Neuromorphic AI**. Neuromorphic AI is inspired by how the human brain processes information. Instead of relying only on large, computation-heavy neural networks, neuromorphic systems can learn and recognize patterns using compact representations and efficient matching methods. This makes them useful for MCU-based applications where memory, processing power, and battery capacity are limited. For example, a microcontroller can learn sensor patterns, thermal images, vibration signals, or sound signatures locally and recognize them later without needing a cloud connection.

The main advantage of Neuromorphic AI on MCUs is that it can support fast, low-power, and on-device learning or recognition. Unlike many conventional AI models that are trained on powerful computers before deployment, some neuromorphic approaches allow learning to happen directly on the device. This is useful for practical Edge AI applications where the system needs to adapt to new users, new environments, or new sensor patterns. In a thermal-image application, for example, the MCU can learn several thermal patterns and later recognize which learned category a live thermal image is closest to, while showing the recognition result immediately on the device.

Prototype-based Neuromorphic AI (Figure 7-2) is a pattern-recognition approach that is well suited for Edge AI and microcontroller-based applications. It is inspired by the way biological systems recognize familiar patterns from past experiences. Instead of using a large, computation-heavy neural network, this approach learns examples as **prototypes**. Each prototype represents a known pattern, such as a thermal image, sound signature, vibration profile, or sensor condition.

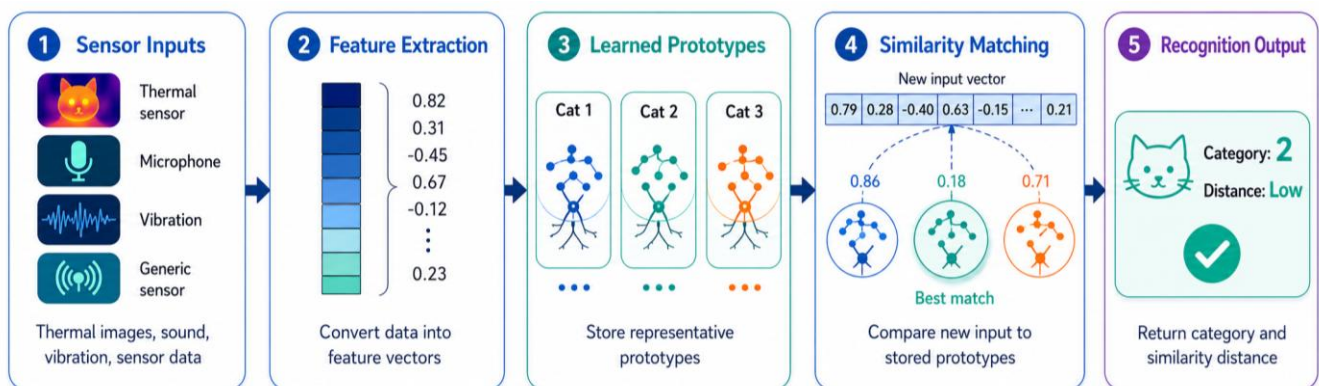


Figure 7-2 – Prototype-based Neuromorphic AI

In a prototype-based system, sensor data is first converted into a feature vector. This feature vector may come from images, thermal pixels, sound, vibration signals, or other sensor readings. During learning, the system stores the feature vector together with a category label. During recognition, a new feature vector is compared against the learned prototypes. The system then returns the closest matching category and a distance value. A smaller distance means that the live input is more similar to the learned prototype.

This approach is useful for microcontrollers because it is lightweight and does not require heavy cloud-based training. In many cases, learning can happen directly on the device. For example, an MCU can learn a thermal image pattern when a button is pressed, assign it a category, and later recognized whether a live thermal image is similar to one of the learned patterns. This makes prototype-based Neuromorphic AI suitable for applications

where the device needs to adapt to new examples in the field without retraining a large AI model on a computer.

Prototype-based Neuromorphic AI is also useful for **few-shot learning**, where the system learns from a small number of examples. This is different from many conventional AI models that require large datasets and long training time. For embedded systems, this is a major advantage because users can teach the device a small set of meaningful patterns, such as normal and abnormal machine conditions, familiar thermal profiles, specific sound signatures, or known sensor states.

Another advantage is that the recognition result is easy to interpret. The system can return both the recognized category and the distance. The category tells us which learned prototype is the closest match, while the distance gives an indication of how similar the current input is to that prototype. This makes it useful for classroom demonstrations, prototyping, and practical Edge AI applications because learners can observe how the device makes a recognition decision.

7.2 Edge-AI Use Cases

7.2.1 Smart Bathroom

(a) Program Overview

This program `smart_bathroom_ai.py` implements a **Smart Bathroom Edge-AI control system** using the M5StickS3, ENV-Pro sensor, PIR motion sensor, Light Sensor, 4-Relay Unit, and embedded buzzer. Its main purpose is to monitor bathroom occupancy, lighting condition, temperature, humidity, pressure, and relative air-quality changes, then automatically control the bathroom light and exhaust fan. It also includes a lightweight Edge-AI classifier that interprets the bathroom condition as **vacant normal**, **occupied normal**, **steamy / humid**, or **poor air**.

The system does not rely only on fixed sensor thresholds. It combines conventional rule-based control with a simple prototype-based AI model. Sensor readings are converted into six normalised features: temperature, humidity, gas response, light intensity, motion activity, and occupancy. These features are compared against predefined class prototypes to estimate the current bathroom condition. The AI result is then used together with safety rules and environmental thresholds to control the exhaust fan.

(b) Program Operational Logic

During setup, the program first initialises the M5StickS3 and configures the shared I2C bus. The PbHUB is initialised for the PIR and light sensor, while the PaHUB is used to connect the ENV-Pro Unit on Port 2 and the 4-Relay Unit on Port 5. Relay 1 is assigned to the bathroom light, and Relay 2 is assigned to the exhaust fan. The relay unit is placed in synchronous mode, and all relays are switched off at startup. The embedded buzzer is also initialised and stopped so that the system starts in a safe state.

The program continuously runs inside the main loop. Each cycle processes occupancy, motion activity, sensor readings, Edge-AI classification, fan control, delayed light-off logic, and buzzer alerts. Timing is handled using `time.ticks_ms()` and `time.ticks_diff()` so that different tasks can run at different intervals without blocking the whole system.

Occupancy is determined using the PIR motion sensor connected through the PbHUB. When motion is detected, the bathroom is marked as occupied, the last motion time is updated, and a motion event is counted. If no

motion is detected for the defined hold time, the room is treated as vacant. This prevents the system from immediately declaring the room vacant when the PIR sensor momentarily stops detecting movement.

The motion event count is also used to estimate human activity. The program counts new motion events over a one-minute activity window. At the end of each window, the count is stored as **average_motion_activity** and then reset. This value becomes one of the AI input features and helps the system distinguish between low activity, normal occupancy, and more active bathroom usage.

Sensor readings are updated at a fixed interval. The ENV-Pro Unit provides temperature, humidity, pressure, and gas resistance readings. The Light Sensor is read through the PbHUB, but its signal is inverted in the program before being converted into an estimated lux value. The lux value is then used to decide whether artificial lighting is needed when the room is occupied.

The bathroom light is controlled using occupancy and light level. While the bathroom is occupied, the light is switched on when the measured lux falls below the lower threshold. If there is enough ambient light and the lux value rises above the upper threshold, the light is switched off. The use of separate on and off thresholds provides hysteresis, which prevents the light from repeatedly toggling when the brightness is near the threshold.

When the bathroom becomes vacant, the light is not switched off immediately. Instead, the program starts a delayed light-off countdown. If motion is detected again during the countdown, the countdown is cancelled and the room is treated as occupied again. If no motion is detected until the countdown expires, the light is switched off. This provides a more practical and user-friendly lighting behaviour.

The gas sensor uses a baseline calibration stage. During startup, the program collects a defined number of gas resistance samples and calculates their average as the gas baseline. After calibration, the current gas resistance is compared with this baseline. A significant drop in gas resistance indicates a possible increase in VOC or poor-air condition. If the gas ratio falls below the alert threshold, the program sets **gas_alert** to true. When the gas ratio recovers above the recovery threshold, the alert condition is cleared.

The Edge-AI logic is based on predefined class prototypes. Each prototype represents a typical bathroom condition using six normalised features: temperature, humidity, gas response, light intensity, motion activity, and occupancy. The program extracts the current feature vector from live sensor values and compares it with each prototype using squared Euclidean distance. The class with the smallest distance becomes the AI classification. Confidence is estimated by comparing the best distance with the second-best distance.

The AI classifier does not run until gas baseline calibration is complete. Before that, the classification remains in the calibrating state. Once calibration is complete, the AI model runs periodically and updates the current bathroom classification. The possible classifications are vacant normal, occupied normal, steamy / humid, and poor air.

Fan control uses both safety rules and AI classification. The highest priority is the direct air-quality safety override. If **gas_alert** is active, the fan is switched on immediately regardless of other conditions. If the AI classifier detects a poor-air pattern, the fan is also switched on. If the room is occupied and the AI classifier detects a steamy or humid condition, the fan is switched on as well.

If none of the AI or gas-alert conditions apply, the fan falls back to conventional threshold-based control. While occupied, the fan switches on when temperature or humidity exceeds the high thresholds. It switches off only

after temperature and humidity fall below their recovery thresholds and the AI classifier is no longer detecting a steamy / humid condition. When the bathroom is vacant, the fan is switched off unless an air-quality alert condition keeps it on.

The buzzer is used as an air-quality warning output. It only operates when **gas_alert** is active. Instead of sounding continuously, the program produces a periodic warning tone based on a buzzer interval. This gives an audible alert while avoiding constant sound output.

The program also includes a safety shutdown mechanism. If an exception or keyboard interrupt occurs, the relay outputs are switched off and the buzzer is stopped. This helps ensure that the connected light, fan, and warning output do not remain unintentionally active after a program error.

7.2.2 Vibration Monitoring

(a) Program Overview

Machine vibration is commonly used in **machine condition-based monitoring** because changes in vibration patterns can indicate changes in machine health. A normal machine usually produces a consistent vibration signature during operation, while faults such as imbalance, looseness, misalignment, bearing wear, or abnormal mechanical contact may produce different vibration patterns. By monitoring vibration regularly, the system can help detect early signs of abnormal machine behavior before a failure occurs.

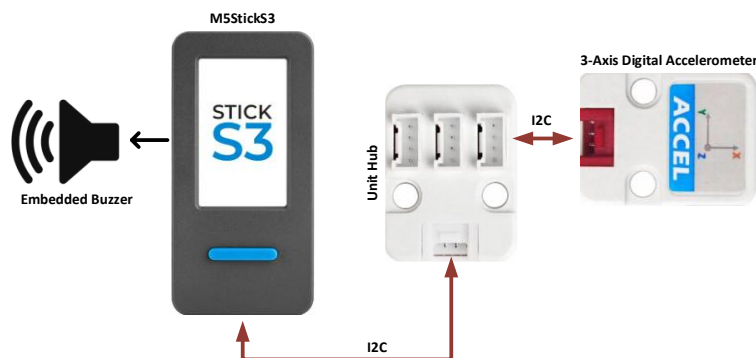


Figure 7-3 – Vibration monitoring system

This program **vibration_mon_ai.py** implements an **NMAI-based vibration monitoring system** using the M5StickS3, a 3-axis digital accelerometer unit (ADXL345), the built-in speaker, and the custom **nmaicore.py** Edge-AI library. The system as shown in Figure 7-3 collects vibration data from the accelerometer, converts the 3-axis readings into a single vibration magnitude signal, transforms the time-domain vibration signal into frequency-domain data, normalizes it into a **256-byte vector**, and uses NMAI to learn and recognize vibration patterns locally on the microcontroller.

The system starts in **monitoring mode**. In this mode, it periodically captures a new vibration signal, converts it into a feature vector, and compares it against trained vibration prototypes stored in the NMAI model. Button A is used to train the current vibration pattern, while Button B clears all trained prototypes and resets the category numbering.

(b) Required Custom Library

The key custom library required by this program is:

```
from nmaicore import NMAICore, NORM_L1, CLASSIFIER_RBF
```

The library, **nmaicore.py**, provides the NMAICore class, which is used as the **prototype-based Neuromorphic AI model** in this vibration monitoring system. The AI network is configured with a network size of **32 neurons** and a vector size of **256 bytes**, matching the 256-byte frequency-domain vibration vector generated from the accelerometer data.

The program configures the NMAI model using:

```
ai.set_context(  
    context=1,  
    norm=NORM_L1,  
    minif=MINIF,  
    maxif=MAXIF  
)  
ai.set_classifier_type(CLASSIFIER_RBF)
```

The model uses **NORM_L1 distance measurement** and the **CLASSIFIER_RBF classifier**. NORM_L1 compares vibration vectors by measuring the total difference between their feature values, while CLASSIFIER_RBF allows the model to recognise whether a new vibration vector falls within the influence range of a learned prototype. The **MINIF** and **MAXIF** values control the minimum and maximum influence range used during learning and recognition.

In this setup, each learned vibration pattern becomes a stored prototype with an assigned category. For example, the first trained vibration vector is stored as Category 1, the second as Category 2, and so on. During monitoring, the live vibration vector is compared against the stored prototypes, and the model returns the closest matching category and its distance. A smaller distance indicates that the current vibration pattern is more similar to the learned prototype.

(c) Program Operational Logic

Accelerometer and Sampling

The program includes an internal driver class for the ADXL345-based 3-axis accelerometer unit. The accelerometer is connected through I2C using address 0x53. During setup, the program configures the accelerometer into measurement mode, full-resolution mode, and $\pm 16g$ range. Each raw X, Y, and Z acceleration reading is converted into g-force using the scale factor 0.0039.

For each vibration sample, the program calculates the acceleration magnitude:

```
magnitude =  $\sqrt{x^2 + y^2 + z^2}$ 
```

This combines the X, Y, and Z acceleration values into one single vibration value. The program collects **256 magnitude samples** for each vibration vector. With a sample interval of 5 ms, one acquisition window takes about:

```
256 samples  $\times$  5ms = 1280ms
```

This produces a short time-domain vibration signal representing the recent vibration behaviour of the target object.

Noise Reduction

A stationary accelerometer still produces small fluctuations due to sensor noise, electrical noise, and minor mechanical disturbances. To reduce this effect, the program first calculates the average magnitude of the 256 samples and subtracts it from each sample. This removes the static gravity component and allows the system to focus on vibration changes instead of constant background acceleration.

After mean removal, the program applies a small deadband:

```
NOISE_DEADBAND_G = 0.015
```

Any vibration change smaller than 0.015 g is treated as zero. This helps suppress small stationary noise and produces a cleaner vibration signal for display and feature extraction.

Frequency-Domain Feature Extraction

After collecting and filtering the 256 time-domain vibration samples, the program converts the signal into frequency-domain data using a 256-point FFT. The FFT changes the representation from “how vibration changes over time” to “what frequency components are present in the vibration”.

The program then calculates the magnitude spectrum from the real and imaginary FFT results. The DC component is set to zero so that the feature vector focuses on vibration frequency content rather than constant offset.

The spectrum is then normalised into a byte range from 0 to 255. The final result is a **256-byte vibration feature vector**, which is suitable for NMAI learning and recognition.

Training

When Button A is pressed, the program captures a fresh vibration signal and converts it into a 256-byte frequency-domain vector. This vector is then passed into the NMAI model using:

```
ai.learn(vibration_vector, category)
```

Each trained vibration vector is assigned a category automatically. The first trained vector is assigned category 1, the second is assigned category 2, and so on. After each training operation, the program increments both the learned count and the next category number.

The system can train up to:

```
MAX_PROTOTYPES = 32
```

Once 32 vibration prototypes have been trained, no further prototypes are added unless the model is cleared.

Forget

When Button B is pressed, the program clears all trained vibration prototypes using:

```
ai.forget()
```

It then resets:

```
earned_count = 0  
next_category = 1
```

This returns the system to its initial learning state, allowing the user to train a new set of vibration patterns starting again from category 1.

Monitoring and Recognition

The system periodically performs monitoring based on:

```
MONITORING_INTERVAL_MS = 1000
```

During each monitoring cycle, the program captures a new vibration signal, converts it into a 256-byte frequency-domain vector, and classifies it using:

```
result = ai.classify(vibration_vector)
```

The NMAI model compares the current vibration vector against all trained prototypes. It returns the closest matching category, the distance, and recognition status. The category indicates which trained prototype is closest to the current vibration pattern. The distance indicates how similar or different the current pattern is compared with the trained prototype. A smaller distance means a closer match.

(d) Overall Operational Flow

```

Read X, Y, Z acceleration
      ↓
Convert 3-axis acceleration into magnitude
      ↓
Collect 256 time-domain samples
      ↓
Remove average value and apply deadband
      ↓
Convert signal to frequency domain using FFT
      ↓
Normalize spectrum into 256-byte vector
      ↓
Button A trains vector as a new category
      ↓
Monitoring mode classifies live vector against trained prototypes
      ↓
Button B clears all learned prototypes
  
```

Overall, this program demonstrates an MCU-based Edge-AI vibration monitoring workflow. The M5StickS3 acquires vibration data, extracts frequency-domain features locally, trains vibration prototypes using the custom `nmaicore.py` library, and performs real-time vibration pattern recognition without relying on cloud processing.

7.2.3 Image Learning and Recognition

(a) Introduction to Thermal Imaging with a Thermopile Sensors Array

The M5StickS3 with the Panasonic AMG8833 thermopile sensors array (TSA) as shown in Figure 7-4 can be used as a compact, low-resolution thermal image acquisition system for monitoring the non-contact surface temperature of a target object. The AMG8833 detects infrared radiation emitted from object surfaces and converts it into an 8×8 thermal pixel array, giving 64 temperature readings. The M5StickS3 acts as the controller by reading the thermal data through I2C, processing the pixel values, and displaying the heat pattern on its built-in LCD using a color map.

Although the resolution is much lower than a commercial thermal camera, the system is suitable for basic temperature monitoring, hotspot detection, object presence detection, thermal pattern observation, and Edge-

AI prototyping. The readings may be affected by distance, viewing angle, surface material, emissivity, ambient temperature, and airflow, so the target object should be positioned consistently for more reliable results.

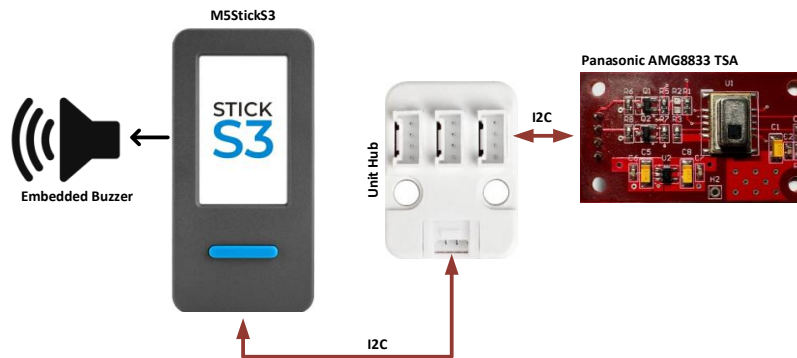


Figure 7-4 – Low-resolution thermal imaging

(b) Program Overview

The thermal-image learning and recognition program `thermal_img_learn_recog_ai.py` implements an **Edge-AI thermal image learning and recognition system** using the M5StickS3, an AMG8833 8×8 thermal sensor, a **Log-FFT feature extraction library**, and a **prototype-based Neuromorphic AI library**. The system captures low-resolution thermal images, converts each image into a compact feature vector, and uses the AI model to learn or recognize thermal patterns locally on the M5StickS3.

The program operates mainly in **recognition mode**. It continuously reads the latest thermal image, converts the 64 thermal pixels into a Log-FFT byte vector, and compares the vector against previously learned thermal-image patterns. Button A is used to learn the current thermal image as a new category, while Button B clears all learned patterns and resets the learning category back to 1.

(c) Required Custom Libraries

The program uses three important custom libraries:

```
from amg8833 import AMG8833
from tlogfft8 import ThermalLogFFT8
from nmaicore import NMAICore, NORM_L1, CLASSIFIER_RBF
```

The first library, **amg8833.py**, provides the driver for the AMG8833 thermal sensor. It allows the M5StickS3 to communicate with the sensor through I2C, read the flattened 8×8 thermal pixel data, apply moving average filtering, and draw the thermal image. In this program, the sensor is configured at I2C address 0x69 with emissivity set to 0.95 and frame rate set to 10.

The second library, **tlogfft8.py**, provides the **ThermalLogFFT8** class. This library converts the 64 flattened thermal pixel values into a **64-value Log-FFT feature vector**. Before the FFT process, the thermal image is normalized using **min-max** normalization. The output is converted into byte values from 0 to 255, making it suitable for the Neuromorphic AI model. The program also removes the DC component using **remove_dc=True**, which helps reduce the effect of the overall average temperature and focuses more on the thermal pattern.

The third library, **nmaicore.py**, provides the NMAICore class, which is used as the **prototype-based Neuromorphic AI model**. The AI network is configured with a network size of 32 neurons and a vector size of 64,

matching the 64-value Log-FFT feature vector. It uses **NORM_L1** distance measurement and the **CLASSIFIER_RBF** classifier. In this setup, each learned thermal image becomes a stored prototype with an assigned category.

(d) Program Operational Logic

During setup, the program first initializes the M5StickS3, configures the LCD, and creates the I2C connection. It then initializes the AMG8833 thermal sensor, enables moving average filtering, creates the Log-FFT feature extractor, and prepares the Neuromorphic AI network. The AI network is set with a context value of 1, L1 distance norm, minimum influence field of 2, and maximum influence field of 8192.

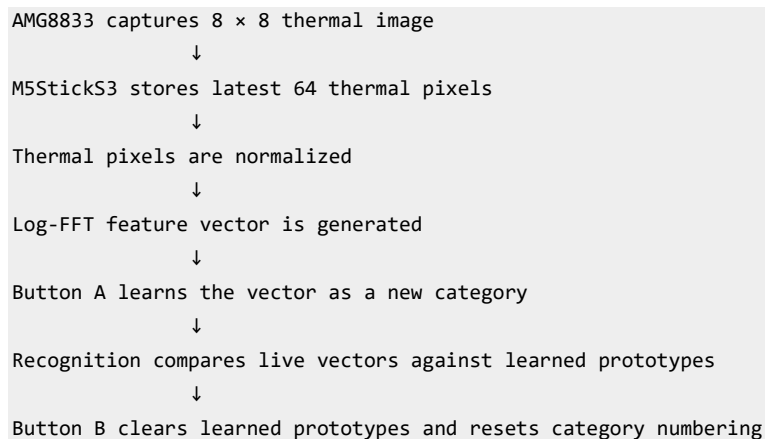
The main loop runs continuously and performs three main tasks. First, it updates the thermal frame by reading the AMG8833 sensor at a fixed display interval. The latest 8×8 thermal frame is stored in **latest_thermal_pixels**, so the same recent frame can be reused for learning and recognition. This avoids learning or recognizing from a different frame than the one most recently acquired.

Second, the program checks the button states. Button A is used for learning. When Button A is pressed, the program captures the latest thermal frame, converts it into a Log-FFT byte vector, and stores it in the Neuromorphic AI model using the current category number. The first learned image is assigned category 1, the second is assigned category 2, and so on. The category number automatically increments after each learning operation. Learning is limited to a maximum of 10 images.

Button B is used to forget all learned thermal images. When Button B is pressed, the program clears the AI network by calling **ai.forget()**. It then resets **learned_count** to 0 and **next_category** back to 1. This allows the user to restart the learning process from the first category again.

Third, the program performs recognition periodically based on the recognition interval. If no image has been learned yet, recognition is skipped because there is no stored prototype to compare against. Once at least one image has been learned, the latest thermal frame is converted into a Log-FFT byte vector and passed to **ai.classify()**. The AI model compares the current vector with the learned prototypes and returns the closest matching category, distance, identified status, and uncertainty status.

In summary, the program follows this operational flow:



Overall, this program demonstrates a complete MCU-based Edge-AI workflow. The M5StickS3 captures thermal data, extracts features locally, learns thermal-image patterns as prototypes, and recognizes live thermal patterns without needing cloud processing.

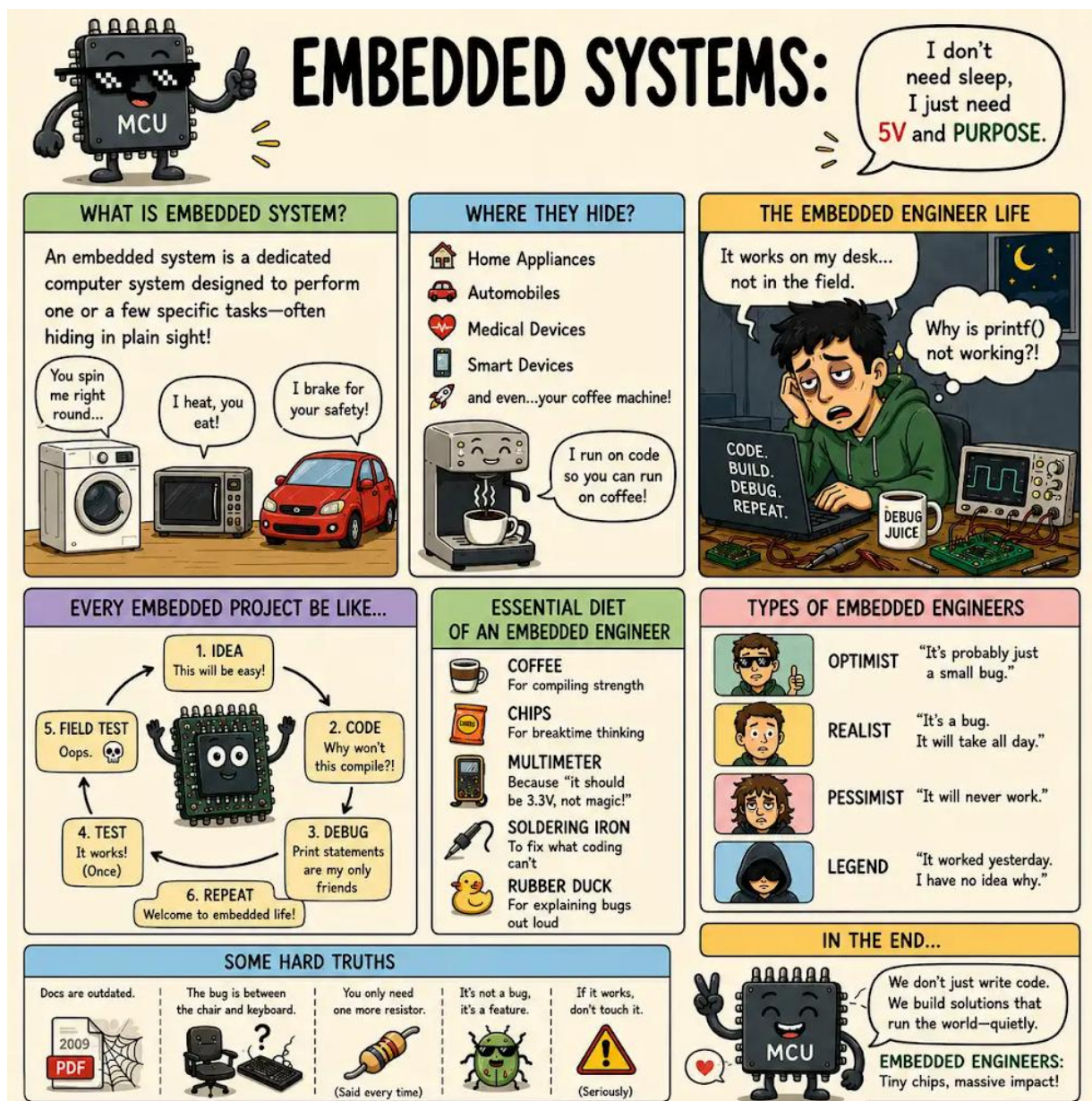
8. Key Takeaways

- 1) MicroPython provides a beginner-friendly way to program microcontrollers using readable Python-style syntax while still supporting practical hardware control. This makes it easier for learners to understand programming concepts and apply them directly to embedded systems, such as reading sensors, controlling outputs, displaying information, and responding to user inputs.
- 2) The M5StickS3 can operate as a compact standalone MCU platform because it includes many built-in features within a small device. These include an LCD display, push buttons, speaker, microphone, IMU, real-time clock, wireless connectivity, battery power, and expansion interfaces. With these built-in peripherals, learners can create interactive MCU applications without immediately requiring additional external hardware.
- 3) Thonny IDE and M5Burner are important tools in the M5StickS3 development workflow. M5Burner is used to prepare the M5StickS3 by installing the required UIFlow 2.0 firmware, while Thonny is used to write, upload, run, and debug MicroPython programs. Together, these tools provide the software environment needed to develop and test MCU applications.
- 4) A typical M5StickS3 program follows a clear and repeatable structure. It begins by importing the required libraries and defining global objects. The `setup()` function is used to initialise the hardware and prepare the system, while the `loop()` function contains the repeated application logic. The program should call `M5.update()` regularly to refresh device status, especially when using buttons and other M5Stack functions. An infinite loop with error handling is commonly used so that the MCU application can continue running safely.
- 5) Core programming concepts such as variables, data types, if/elif/else decisions, for and while loops, time delays, and reusable functions are not treated as isolated theory. They are directly applied in practical MCU tasks such as sensor reading, actuator control, display updates, button detection, monitoring logic, and alert generation. This helps learners connect basic programming concepts with real embedded-system applications.
- 6) The M5StickS3 built-in LCD display, speaker, buttons, RTC, microphone, and IMU allow learners to build useful and interactive applications using only the device itself. These features support activities such as displaying text and graphics, producing audio alerts, detecting button presses, keeping time, capturing sound, measuring motion, detecting tilt, and monitoring vibration.
- 7) PaHUB and PbHUB expand the number of available I2C and GPIO ports on the M5StickS3. This allows the device to support larger systems with multiple sensors, input devices, and output modules connected at the same time. With these expansion hubs, learners can move beyond simple single-sensor examples and develop more integrated MCU applications.

- 8) External peripherals further broaden the practical application scope of the M5StickS3. GPIO-based and I2C-based modules allow learners to explore motion detection, light measurement, servo control, relay switching, distance measurement, environmental monitoring, accelerometer-based vibration sensing, and RFID-based identification. These peripherals provide hands-on experience in connecting hardware, reading sensor data, processing conditions, and controlling outputs.
- 9) The integrated projects demonstrate the complete embedded-system workflow. Learners practise sensing data from the environment, processing the readings, making decisions, controlling output devices, showing local feedback on the display, producing audible alerts, and implementing safe shutdown behaviour. This helps learners understand how individual hardware and software concepts come together in a complete MCU application.
- 10) Machine vibration monitoring and thermal-image recognition show how sensor data can be transformed into feature vectors for local AI learning and classification. Vibration signals can be converted from time-domain data into frequency-domain features, while thermal images can be normalised and represented as compact feature vectors. These processed vectors can then be used by an AI model to learn and recognise different operating patterns.
- 11) Edge-AI reduces dependence on cloud processing by allowing the MCU to perform feature extraction, learning, recognition, and real-time decision-making locally. This enables faster response, lower bandwidth usage, improved privacy, and better operation in environments where internet connectivity may be limited or unavailable.
- 12) Prototype-based Neuromorphic AI is suitable for MCU-based learning because it can learn from a small number of examples and operate with limited processing power and memory. Each learned pattern becomes a stored prototype with an assigned category. During recognition, a new input is compared against the stored prototypes, and the system returns an interpretable category and distance. This makes the approach useful for classroom demonstrations, rapid prototyping, and practical Edge-AI applications on microcontrollers.

9. Concluding Note

The course provides a progression from basic programming and MCU setup to practical embedded-system applications and introductory Edge-AI. It helps learners understand how a microcontroller can sense the environment, process data, make decisions, control outputs, and perform lightweight AI recognition locally. These skills provide a strong foundation for future IoT, automation, sensing, smart-device, and proof-of-concept development projects.



Appendix A – MicroPython Examples

This list includes only named Python example programs from the training document. Supporting custom libraries such as `nmaicore.py`, `amg8833.py`, and `tlogfft8.py` are not included because they are libraries rather than example programs.

No.	Python example program	Short description
1	test1.py	First M5StickS3 MicroPython program that draws simple shapes, displays a greeting, and plays a tone.
2	myprint.py	Introduces the <code>print()</code> function by displaying simple text messages in the Thonny Shell.
3	mydata.py	Demonstrates variables and basic data types such as string, integer, float, and Boolean values.
4	app_template.py	Provides the standard M5StickS3 MicroPython program structure using <code>setup()</code> , <code>loop()</code> , <code>M5.update()</code> , and error handling.
5	lcd_basic.py	Shows basic LCD setup, including screen brightness, background colour, and display rotation.
6	lcd_write_text.py	Displays and updates text labels on the M5StickS3 LCD using different colours, positions, and font sizes.
7	lcd_draw.py	Draws simple graphical objects on the LCD, including a title and circles to create a basic face graphic.
8	buzzer_buttons.py	Uses the built-in buttons to trigger actions and demonstrate event-driven interaction with the device.
9	alarm_clock.py	Turns the M5StickS3 into a 24-hour digital clock with an alarm function using the real-time clock and speaker.
10	countdown_timer.py	Implements a preset countdown timer controlled by the buttons, with an alarm when the timer reaches zero.
11	simple_spl_meter.py	Uses the built-in microphone to estimate sound level and classify the reading as low, medium, or high.
12	sound_recorder.py	Records sound using the built-in microphone and plays back the recorded audio through the built-in speaker.
13	tilt_meter.py	Uses the built-in IMU to detect tilt direction, estimate tilt angle, and monitor device orientation.
14	vibration_sensor.py	Uses the built-in IMU to detect vibration and trigger an alarm when vibration exceeds a selected threshold.
15	pedometer.py	Uses the built-in IMU to detect walking steps and estimate calories burned.
16	app_template_exp.py	Extends the standard program template to include PaHUB and PbHUB support for external peripherals.
17	dual_button_terminal.py	Reads the external Dual-Button Unit through PbHUB and detects red and blue button presses.
18	pir_motion_terminal.py	Reads the PIR Motion Sensor through PbHUB to detect whether human motion is present.
19	light_intensity_terminal.py	Reads the Light Sensor through PbHUB, converts the analog reading into estimated lux, and monitors brightness.

20	angle_servo_control.py	Uses the Angle Sensor as a manual control input to drive a 360-degree servo motor through PbHUB.
21	simple_relay.py	Controls the 4-Relay Unit through PaHUB by switching relay channels on and off in sequence.
22	color_sensor.py	Reads RGB values from the Color Sensor Unit and displays the detected colour on the LCD.
23	measure_distance.py	Uses the I2C Ultrasonic Distance Unit to measure distance to a nearby object and detect range status.
24	env_gas_monitor.py	Reads temperature, humidity, pressure, and gas resistance from the ENV-Pro Unit for environmental monitoring.
25	machine_vibration.py	Uses the ADXL345 accelerometer to monitor machine vibration and classify vibration level based on thresholds.
26	rfid_security_access.py	Uses RFID tags and the 4-Relay Unit to demonstrate RFID-based identification and access control.
27	securehome.py	Combines RFID, PIR motion detection, dual buttons, relay output, and buzzer to create a simple home security system.
28	smart_bathroom.py	Integrates ENV-Pro, PIR, Light Sensor, relays, and buzzer to automate bathroom lighting, ventilation, and alerts.
29	smart_bathroom_ai.py	Adds Edge-AI classification to the smart bathroom system using sensor features and prototype-based condition recognition.
30	vibration_mon_ai.py	Implements NMAI-based vibration monitoring by converting accelerometer vibration data into frequency-domain vectors for learning and recognition.
31	thermal_img_learn_recog_ai.py	Implements thermal image learning and recognition using AMG8833 thermal pixels, Log-FFT feature extraction, and prototype-based Neuromorphic AI.

Appendix B - List of MicroPython Commands

The following command reference consolidates the Python language constructs, functions, classes, and methods used across the training document. Commands are grouped by purpose to make them easier to review and reuse in future MCU programs.

B.1 Import and Library Commands

Command / Syntax	Purpose in the Training Document
import os, sys, io	Imports general system and input/output support libraries.
import M5	Imports the M5Stack device library.
from M5 import *	Imports M5Stack classes and objects such as Widgets, Speaker, BtnA, BtnB, Mic, and Imu into the program namespace.
import time	Imports timing functions such as sleep(), sleep_ms(), ticks_ms(), ticks_diff(), and time().
import math	Imports mathematical functions such as sqrt().
from hardware import Pin	Imports GPIO pin support for configuring hardware pins.
from hardware import I2C	Imports I2C communication support for external modules.
from unit import PAHUBUnit	Imports the PaHUB class for selecting I2C expansion ports.
from unit import PBHUBUnit	Imports the PbHUB class for reading analog/digital GPIO expansion ports and controlling servo outputs.
from unit import Relay4Unit	Imports the 4-Relay Unit class.
from unit import ColorUnit	Imports the RGB Color Sensor Unit class.
from unit import UltrasoundI2CUnit	Imports the I2C Ultrasonic Distance Unit class.
from unit import ENVPROUnit	Imports the ENV Pro environmental sensor class.
from unit import AccelUnit	Imports the ADXL345 3-axis accelerometer unit class.
from unit import RFIDUnit	Imports the RFID Unit class.
from nmaicore import NMAICore, NORM_L1, CLASSIFIER_RBF	Imports the prototype-based Neuromorphic AI core and related classifier settings.
from amg8833 import AMG8833	Imports the AMG8833 thermal sensor library.
from tlogfft8 import ThermalLogFFT8	Imports the 8 x 8 thermal Log-FFT feature extraction library.
from utility import print_error_msg	Imports the error message helper used in exception handling.

B.2 Program Structure and Python Basics

Command / Syntax	Purpose in the Training Document
variable = value	Creates or updates a variable, such as temperature = 28.
None	Represents an empty or unassigned object reference.
True / False	Boolean values used for system states and conditions.
global variable_name	Allows a function to modify a global variable.
def function_name():	Defines a reusable function.
function_name()	Calls a function.
return value	Returns a value from a function to the caller.
if condition:	Runs a block of code when a condition is true.
elif condition:	Checks another condition if the previous if/elif condition is false.
else:	Runs a block when no previous condition is true.
for variable in range(n):	Repeats a block a known number of times.
while condition:	Repeats a block while the condition remains true.
while True:	Creates an infinite loop for continuous MCU operation.
try:	Starts an error-handling block.
except (Exception, KeyboardInterrupt) as e:	Catches runtime errors or keyboard interruption.
except ImportError:	Handles missing import/library errors.
print()	Displays messages or values in the Thonny Shell.
range()	Generates a sequence of integer values for loops.
str()	Converts a value into a string.
int()	Converts a value into an integer.
round()	Rounds a numeric value.
len()	Returns the length of a sequence, list, bytearray, or collection.
abs()	Returns the absolute value.
min()	Returns the smallest value in a sequence.
max()	Returns the largest value in a sequence.
sum()	Returns the sum of numeric values in a sequence.
bytearray()	Creates a mutable byte buffer for audio, vector, or data storage.
list.append()	Adds a new item to a list.
"{}".format()	Formats values into strings for display or printing.
//	Integer division, used for whole-number calculations such as remaining seconds.
%	Remainder operator, used for time formatting and cyclic calculations.
<<	Bit-shift operator used to place RGB values into color bytes.
	Bitwise OR operator used to combine RGB components into 0xRRGGBB color format.

B.3 Timing Commands

Command / Syntax	Purpose in the Training Document
<code>time.sleep(seconds)</code>	Pauses the program for a number of seconds.
<code>time.sleep_ms(milliseconds)</code>	Pauses the program for a number of milliseconds.
<code>time.time()</code>	Returns system time in seconds, used for countdown timing.
<code>time.ticks_ms()</code>	Returns a millisecond counter for non-blocking timing.
<code>time.ticks_diff(current, previous)</code>	Calculates elapsed time safely using tick values.

B.4 M5StickS3 System and LCD Commands

Command / Syntax	Purpose in the Training Document
<code>M5.begin()</code>	Initializes the M5StickS3 hardware and peripherals.
<code>M5.update()</code>	Refreshes device status and should be called regularly in <code>loop()</code> .
<code>Widgets.setBrightness(value)</code>	Sets LCD brightness from 0 to 255.
<code>Widgets.fillScreen(0xRRGGBB)</code>	Fills the LCD background with a color.
<code>Widgets.setRotation(value)</code>	Sets LCD orientation.
<code>Widgets.Label(text, x, y, scale, fg, bg, font)</code>	Creates a text label on the LCD.
<code>label.setText(text)</code>	Updates the text displayed by a label.
<code>label.setColor(fg, bg)</code>	Changes the text and background colour of a label.
<code>Widgets.Title(text, style, fg, bg, font)</code>	Creates a title bar on the LCD.
<code>Widgets.Circle(x, y, r, outline, fill)</code>	Draws a circle on the LCD.
<code>Widgets.Rectangle(x, y, w, h, outline, fill)</code>	Draws a rectangle on the LCD.
<code>Widgets.Triangle(x1, y1, x2, y2, x3, y3, outline, fill)</code>	Draws a triangle on the LCD.
<code>M5.Lcd.drawString(text, x, y)</code>	Draws text directly at a specific LCD position.
<code>M5.Lcd.fillRect(x, y, w, h, color)</code>	Fills a rectangular LCD area.
<code>M5.Lcd.drawRect(x, y, w, h, color)</code>	Draws a rectangle outline.
<code>M5.Lcd.drawLine(x1, y1, x2, y2, color)</code>	Draws a line on the LCD.
<code>M5.Lcd.setTextColor(fg, bg)</code>	Sets direct LCD text foreground and background colors.

B.5 Button and Speaker Commands

Command / Syntax	Purpose in the Training Document
BtnA.setCallback(type=..., cb=...)	Registers a callback function for Button A events.
BtnB.setCallback(type=..., cb=...)	Registers a callback function for Button B events.
BtnA.CB_TYPE.WAS_PRESSED / BtnB.CB_TYPE.WAS_PRESSED	Callback event type that runs once when the button is pressed.
BtnA.wasClicked()	Checks whether Button A was clicked.
BtnA.isPressed() / BtnB.isPressed()	Checks whether the built-in button is currently pressed.
Speaker.begin()	Initializes the built-in speaker.
Speaker.setVolume(value)	Sets speaker volume using numeric value.
Speaker.setVolumePercentage(value)	Sets speaker volume as a percentage.
Speaker.tone(frequency, duration)	Plays a tone at the given frequency and duration.
Speaker.stop()	Stops speaker output.
Speaker.playRaw(data, sample_rate)	Plays raw audio data.
Speaker.isPlaying()	Checks whether audio playback is still active.
Speaker.end()	Ends speaker operation.

B.6 RTC, Microphone, and IMU Commands

Command / Syntax	Purpose in the Training Document
rtc.datetime(tuple)	Sets or reads the real-time clock date and time.
Mic.begin()	Initializes the built-in microphone.
Mic.record(buffer, sample_rate, blocking)	Records microphone samples into a buffer.
Mic.isRecording()	Checks whether microphone recording is still active.
Mic.end()	Ends microphone operation.
Imu.getAccel()	Reads acceleration values from the built-in IMU.
math.sqrt(value)	Calculates square root, used for vibration, tilt, and magnitude calculations.

B.7 I2C, PaHUB, PbHUB, and External Peripheral Commands

Command / Syntax	Purpose in the Training Document
Pin(pin_number)	Identifies a GPIO pin for I2C or other hardware use.
I2C(bus, scl=Pin(...), sda=Pin(...), freq=...)	Creates an I2C bus using selected SCL and SDA pins.
PAHUBUnit(i2c=i2c0, channel=n)	Selects a PaHUB I2C expansion port.
PBHUBUnit(i2c0, 0x61)	Initializes the PbHUB GPIO expansion unit.
pbhub_0.digital_read(port, pin)	Reads a digital input from a PbHUB port and pin.
pbhub_0.analog_read(port)	Reads an analog input from a PbHUB port.
pbhub_0.set_servo_angle(port, channel, angle)	Controls a servo motor connected to a PbHUB port.
Relay4Unit(pahub_port)	Creates a 4-Relay Unit object.
relay4_0.set_mode(Relay4Unit.SYNC_MODE)	Sets relay module mode so indicator LEDs follow relay states.
relay4_0.set_relay_all(state)	Switches all relay channels on or off.
relay4_0.set_relay_state(channel, state)	Switches one relay channel on or off.
ColorUnit(pahub_port)	Creates a Color Sensor object.
color_0.get_color_rgb_bytes()	Reads RGB colour values as bytes.
UltrasonicI2CUnit(pahub_port)	Creates an ultrasonic distance sensor object.
ultrasonic_0.get_target_distance()	Reads target distance from the ultrasonic sensor.
ENVPROUnit(pahub_port)	Creates an ENV Pro sensor object.
envpro_0.get_temperature()	Reads temperature from ENV Pro.
envpro_0.get_humidity()	Reads relative humidity from ENV Pro.
envpro_0.get_pressure()	Reads atmospheric pressure from ENV Pro.
envpro_0.get_gas_resistance()	Reads gas resistance from ENV Pro.
AccelUnit(pahub_port, 0x53)	Creates an ADXL345 accelerometer unit object.
accel_0.get_accel()	Reads X, Y, Z acceleration values from the accelerometer.
RFIDUnit(pahub_port)	Creates an RFID reader object.
rfid_0.read_card_uid()	Reads the UID of an RFID card or tag.

B.8 Application and AI-Related Commands

Command / Syntax	Purpose in the Training Document
NMAICore(network_size, vector_size)	Creates a prototype-based Neuromorphic AI model.
ai.set_context(context, norm, minif, maxif)	Configures AI context, distance norm, and influence field limits.
ai.set_classifier_type(CLASSIFIER_RBF)	Sets the AI model to RBF classifier mode.
ai.learn(vector, category)	Learns a vector as a new prototype with a category.
ai.classify(vector)	Classifies a live vector against learned prototypes.
ai.forget()	Clears all learned prototypes.
ai.count_committed_neurons()	Returns the number of learned/committed neurons.
AMG8833(i2c, address, emissivity, frame_rate)	Creates an AMG8833 thermal sensor object.
sensor.get_flattened_data(refresh=True/False)	Reads flattened thermal pixel data.
sensor.draw_thermal_image(...)	Draws a thermal image on the M5StickS3 LCD.
ThermalLogFFT8(...)	Creates an 8 x 8 thermal Log-FFT feature extractor.
logfft.transform_to_bytes(pixels, input_normalization, shift)	Converts thermal pixels into a byte-based Log-FFT vector.
process_buttons()	Application function used to handle button actions.
process_motion_sensor()	Application function used to process PIR motion detection.
process_rfid_tag()	Application function used to read and process RFID cards.
process_countdown()	Application function used to manage alarm countdown timing.
process_buzzer()	Application function used to generate repeating warning tones.
process_occupancy()	Application function used to update occupied/vacant state.
process_motion_activity()	Application function used to count motion events over time.
read_sensors()	Application function used to read environmental and light sensors.
process_air_quality()	Application function used to evaluate gas/air-quality condition.
process_light_off_countdown()	Application function used to switch light off after a delay.
update_display()	Application function used to refresh display information.

Appendix C - Hardware List

This list includes only the M5StickS3 MCU module and the GPIO-based and I2C-based external peripherals used in the course document. Built-in peripherals, software tools, cables, computers, loads, and demonstration target objects are excluded.

C.1 Core MCU Module

Hardware	Purpose	Type / Category
M5Stack M5StickS3 MCU module	Main ESP32-S3-based microcontroller development device used throughout the workshop.	Controller / MCU board

C.2 GPIO-based External Peripherals

Hardware	Purpose	Type / Category
M5Stack Dual-Button Unit	Provides two independent external push-button inputs.	Digital input
M5Stack PIR Motion Sensor Unit	Detects human movement by sensing changes in infrared radiation.	Digital motion sensor
M5Stack Light Sensor Unit	Measures ambient brightness through an analog output.	Analog light sensor
M5Stack Angle Sensor Unit	Uses a rotary potentiometer to provide an analog position value.	Analog input
M5Stack 360-degree Servo Motor	Provides continuous-rotation actuator output for motor or motion-control demonstrations.	Actuator

C.3 I2C-based External Peripherals

Hardware	Purpose	Type / Category
M5Stack 4-Relay Unit	Provides four software-controlled relay channels for switching external loads.	I2C output / actuator control
M5Stack Color Sensor RGB Unit	Measures red, green, blue, and clear-light components from nearby objects or surfaces.	I2C color sensor
M5Stack Ultrasonic Distance Unit	Measures distance to nearby objects using ultrasonic sensing.	I2C distance sensor
M5Stack ENV Pro Unit	Measures temperature, humidity, atmospheric pressure, and gas resistance.	I2C environmental sensor
M5Stack 3-Axis Digital Accelerometer Unit (ADXL345)	Measures acceleration along the X, Y, and Z axes for motion and vibration monitoring.	I2C motion / vibration sensor
M5Stack RFID 2 Unit	Reads RFID cards, key fobs, and tags for identification and access-control applications.	I2C identification sensor
Panasonic AMG8833 Thermopile Sensor Array	Captures an 8 x 8 low-resolution thermal image for non-contact temperature and thermal pattern monitoring.	I2C thermal sensor